ONLINE HEALTH DIAGNOSIS SYSTEM

BCA PROJECT REPORT

A full-stack application for preliminary symptom triage, health guidance, and doctor appointment workflows.

Submitted by
[Student Full Name]

Under the guidance of
[Mentor Name]

[College/University Name]

[Company Name] | Academic Year: [Academic Year]

Index / Table of Contents

Index / Table of Contents 2

1. Certificates 5

Self Certificate . . . 5

Certificate by Guide . . . 5

Certificate of Approval . . . 5

2. About the Company 5

3. Introduction 6

2A. Objective . . . 6

2B. Scope . . . 6

4. System Analysis 6

3A. Identification of Need . . . 6

3B. Feasibility Study . . . 6

3C. Workflow — Iterative Waterfall . . . 7

3D. Study of the System: Modules and Pages . . . 7

3E. Software Requirement Specification . . . 7

Functional Requirements . . . 7

Non-Functional Requirements . . . 8

5. Software Engineering Paradigm Applied 8

6. Diagrams 8

7. Schema Design 10

Indexes and relationships . . . 11

8. Front End 11

Home / Symptom Input . . . 11

Prediction Report . . . 11

Doctor Consultations and Booking Details . . . 11

Chatbot . . . 12

Sign In and Sign Up . . . 12

frontend/src/pages/Dashboard.jsx . . . 12

frontend/src/pages/DiagnosisForm.jsx . . . 16

frontend/src/pages/Appointments.jsx . . . 26

frontend/src/pages/Chatbot.jsx . . . 29

frontend/src/pages/Login.jsx . . . 34

frontend/src/pages/Signup.jsx . . . 35

frontend/src/routes/AppRouter.jsx . . . 37

9. Back End 39

API entry point and utility . . . 39

backend/server.js . . . 39

backend/utils/apiResponse.js . . . 40
Controllers . . . 40
backend/controllers/authController.js . . . 40
backend/controllers/userController.js . . . 41
backend/controllers/diagnosisController.js . . . 42
backend/controllers/doctorController.js . . . 44
backend/controllers/chatController.js . . . 48
Services . . . 49
backend/services/predictionService.js . . . 49
backend/services/geminiService.js . . . 51
backend/services/mlService.js . . . 62
backend/services/chatEngine.js . . . 66
Mongoose models . . . 70
backend/models/User.js . . . 70
backend/models/Doctor.js . . . 70
backend/models/Appointment.js . . . 71
backend/models/Diagnosis.js . . . 71
Routes and middleware . . . 73
backend/routes/authRoutes.js . . . 73
backend/routes/userRoutes.js . . . 74
backend/routes/diagnosisRoutes.js . . . 74
backend/routes/doctorRoutes.js . . . 74
backend/routes/chatRoutes.js . . . 75
backend/middleware/authMiddleware.js . . . 76
backend/middleware/errorHandler.js . . . 76
backend/middleware/rateLimiter.js . . . 76
backend/middleware/validateMiddleware.js . . . 77
backend/config/constants.js . . . 77
Python ML service and classifier . . . 78
ml-service/app.py . . . 78
ml-service/model/classifier.py . . . 82
ml-service/model/train.py . . . 84
ml-service/model/evaluate.py . . . 84
Safe control-file examples . . . 86
backend/config/doctors-control.example.json . . . 86
backend/config/bookings-control.example.json . . . 86
10. Testing 87
Node backend tests . . . 87
Python ML tests . . . 87

Frontend checks 87
Test coverage represented in source 87
backend/tests/geminiService.test.js 87
backend/tests/mlService.test.js 89
backend/tests/predictionService.test.js 91
ml-service/tests/test_predict.py 94
11. Conclusion 95
Future Scope 95
Bibliography 96

1. Certificates

Self Certificate

I, [Student Full Name], certify that this project report titled “Online Health Diagnosis System” is my original work completed as part of the Bachelor of Computer Applications programme. Sources and external tools used are acknowledged in the bibliography. The system provides preliminary educational triage only and is not a medical diagnosis.

Student: [Student Full Name] Guide: [Mentor Name]

Student Signature: _____

Guide Signature: _____

Company / Department Seal: _____

Date: _____ Place: _____

Certificate by Guide

This is to certify that [Student Full Name] completed the project report and implementation work entitled “Online Health Diagnosis System” under my guidance at [Company Name]. The work is submitted for academic review to [College/University Name].

Student: [Student Full Name] Guide: [Mentor Name]

Student Signature: _____

Guide Signature: _____

Company / Department Seal: _____

Date: _____ Place: _____

Certificate of Approval

This project report has been examined and approved as partial fulfilment of the Bachelor of Computer Applications requirements of [College/University Name]. Approval is subject to the institution’s evaluation and academic rules.

Student: [Student Full Name] Guide: [Mentor Name]

Student Signature: _____

Guide Signature: _____

Company / Department Seal: _____

Date: _____ Place: _____

2. About the Company

[Company Name] is an organization focused on software development, digital services, and technology-enabled solutions. Replace this sample paragraph with the approved company profile, location, business domains, and mentoring context before submission.

Project context: this report documents an academic implementation. It does not imply clinical certification, a production deployment, or an association with a healthcare provider.

3. Introduction

Online Health Diagnosis System (also named Carepath / MedInsight in parts of the code and documentation) is a browser-based application for recording symptoms and receiving preliminary, AI-assisted triage guidance. Its interface is implemented in React and Vite. An Express API validates requests and coordinates Mongoose persistence, a symptom-processing engine, Gemini integration, and a Python ML service. The application also includes account, profile, doctor, and appointment features.

The project is informational and educational. It does not provide a confirmed diagnosis, does not replace a clinician, and preserves a medical disclaimer in health-related responses.

2A. Objective

- Build a responsive full-stack flow for entering symptoms and presenting preliminary assessment information.
- Separate browser presentation, API orchestration, persistence, and Python classification into maintainable components.
- Support authentication, diagnosis history, doctor discovery, and appointment booking.
- Apply input validation, standardized API envelopes, emergency handling, and graceful fallback behavior.

2B. Scope

Included scope: user registration and sign-in; protected dashboard and profile; symptom assessment; conversational chatbot; diagnosis persistence and history; doctor list and appointment booking; API validation and rate limits; an ML service bridge with local fallback; unit tests for service logic.

Limitations: triage output is not a medical diagnosis; live provider behavior depends on deployment configuration; the report does not claim that external services or database connectivity were available during this build. Features are described from checked-in source rather than assumed deployment state.

4. System Analysis

3A. Identification of Need

Users often need an accessible starting point to organize symptoms and decide whether to seek professional help. The project responds with a structured symptom form and conversational intake, paired with explicit uncertainty and emergency guidance. A single API also supports profiles, stored assessment history, provider discovery, and appointment requests.

3B. Feasibility Study

Dimension	Assessment
Technical	Feasible with the repository stack: React/Vite, Node/Express, MongoDB/Mongoose, Python/Flask classifier, and optional Gemini API integration. Provider and database credentials are external deployment requirements.
Operational	Browser workflows cover assessment, account/profile, chat, doctor lookup, and appointment handling. Medical disclaimers and explicit emergency wording set user expectations.
Economic	The application uses open-source runtime libraries and can be run locally; hosting, database, model-serving, and third-party API costs depend on selected providers and usage.
Schedule	An iterative waterfall approach fits a student project: define requirements, design modules, implement vertical slices, test, and refine based on test results.
Legal / safety	This is preliminary educational triage only. A clinical review, privacy review, and jurisdiction-specific compliance work would be required before real clinical use.

3C. Workflow — Iterative Waterfall

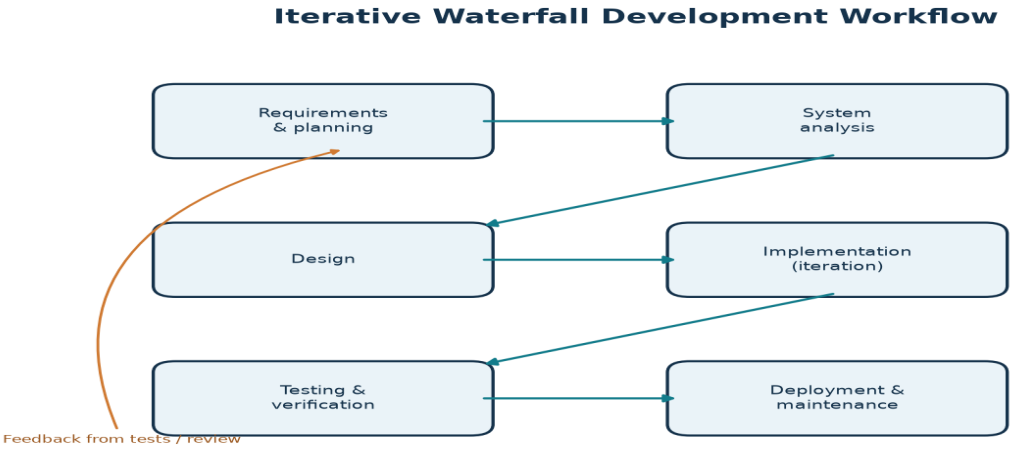


Figure 1. Iterative waterfall with feedback from test and review stages.

The project follows sequential requirement, analysis, design, implementation, and verification stages, while allowing feedback from tests to revisit earlier decisions. It is not a claim that the repository enforces a formal lifecycle toolchain.

3D. Study of the System: Modules and Pages

Module / route	Implemented responsibilities
Home / dashboard (`/dashboard`)	Landing workspace for an authenticated user; links to assessment, chat, profile, and appointments; displays diagnosis/appointment context.
Symptom input and report (`/diagnosis`)	DiagnosisForm collects and submits symptom input, then renders the returned triage/diagnosis result with safety information. The report view is part of this page, not a separately routed screen.
Doctor consultations (`/appointments`)	Appointments page loads doctors and availability, selects a provider/date/time, and submits a booking. There is no separate doctor consultation page route.
Booking details	Appointment list/status and booking form are handled in the Appointments page; appointment records are linked to User and Doctor.
Chatbot (`/chatbot`)	Conversational intake, symptom detection, emergency rules, optional inference, suggestions, and appointment handoff.
Sign in (`/login`)	Login form calls auth service, updates AuthContext, and routes to dashboard.
Sign up (`/signup`)	Registration form validates basic fields/password and creates an authenticated session.
Profile (`/profile`)	Reads and updates profile information through protected user endpoints.

3E. Software Requirement Specification

Functional Requirements

ID	Requirement reflected in implementation
FR-01	Register and sign in with validated credentials; password values are not selected by default in User queries.
FR-02	Protect user, diagnosis, and appointment routes with JWT authentication; chat supports optional authentication.
FR-03	Submit symptoms through a diagnosis endpoint and return an assessment envelope including disclaimer and triage details.
FR-04	Persist authenticated diagnoses and retrieve paginated user diagnosis history.
FR-05	Retrieve and update user profile fields.
FR-06	List doctors and create/list appointments with date, slot, reason, and status.
FR-07	Accept chatbot messages and optional history; classify symptoms, handle emergencies, and optionally run prediction.
FR-08	Return consistent success/error response envelope `{ success, message, data, error }`.

Non-Functional Requirements

ID	Requirement
NFR-01 Security	JWT auth, bcrypt-based credential handling in auth code, Helmet, explicit CORS, input validators, and rate limiting.
NFR-02 Safety	Preliminary triage language, preserved medical disclaimer, conservative urgent-care recommendations, no replacement for medical advice.
NFR-03 Reliability	Typed Gemini errors, request timeouts, local symptom/clinical fallback, API error middleware, and safe controller handling.
NFR-04 Maintainability	Layered routes/controllers/services/models; shared constants; module-separated Python inference service.
NFR-05 Performance	Asynchronous API calls, indexed user history queries, and bounded request payloads.
NFR-06 Privacy	Gemini prompts are constructed in backend services; keys are server-side environment configuration and excluded from this report.
NFR-07 Accessibility	Semantic labels and responsive UI behavior should be reviewed in browser testing; no accessibility audit is claimed here.

5. Software Engineering Paradigm Applied

The codebase uses a layered, service-oriented full-stack design with an iterative waterfall development approach. React components own page interaction and local state; AuthContext carries shared authentication state; a centralized Axios service calls Express endpoints. Express routers apply middleware and validation before controllers. Controllers orchestrate domain services and Mongoose models. The Python service isolates feature encoding and classifier inference.

- Separation of concerns: routes, middleware, controllers, services, models, and UI pages have distinct responsibilities.
- Service orchestration: predictionService controls provider selection and fallback; mlService bridges to the Python service; geminiService parses and sanitizes provider output.
- Defensive design: request bounds, schema validation, safe response envelopes, typed provider errors, and retry/timeouts are present.
- Testability: Node services accept dependency overrides in key prediction paths; Python endpoint logic is covered by pytest tests.

The implementation is a modular monolith on the web/API side with a separately deployable ML microservice. The exact live prediction order follows the current code: default Gemini preliminary prediction for non-empty symptoms, then symptom-engine/ML fallback; Gemini can also enrich engine results. In `ml` mode, local explanations are used.

6. Diagrams

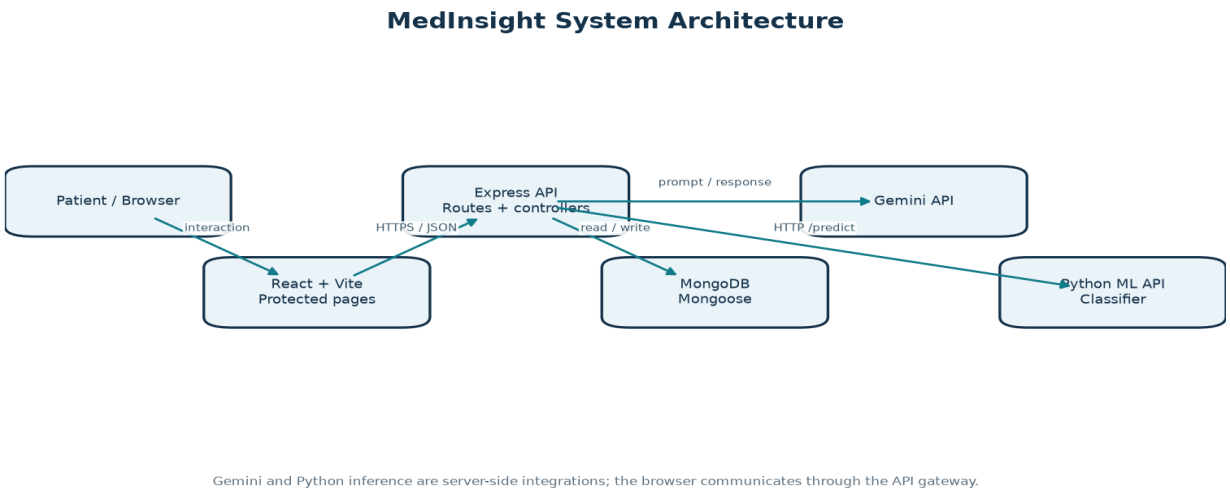


Figure 2. System architecture based on the checked-in frontend, API, database, Gemini, and Python ML service.

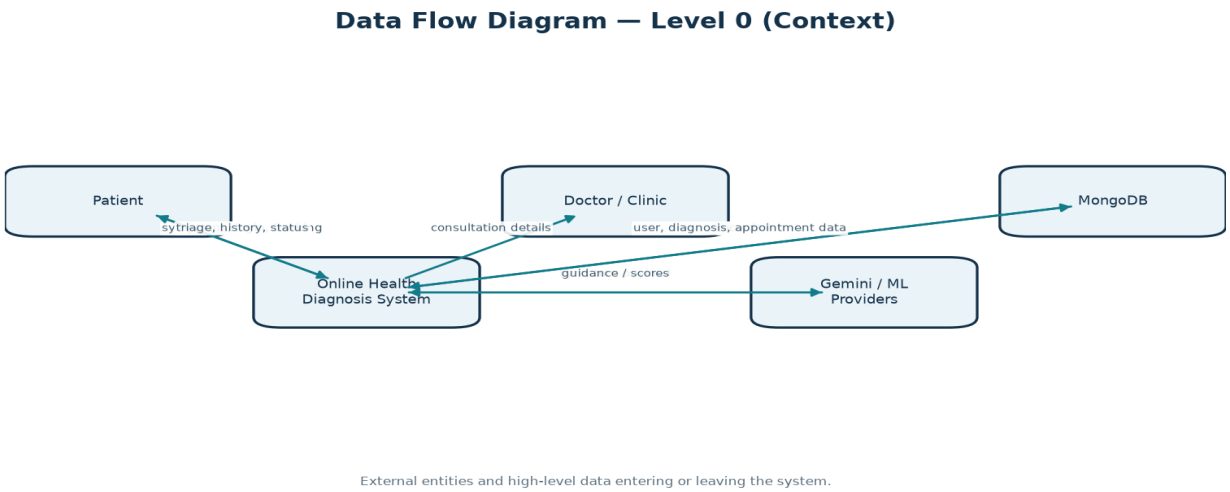


Figure 3. DFD Level 0: system context and external data exchange.

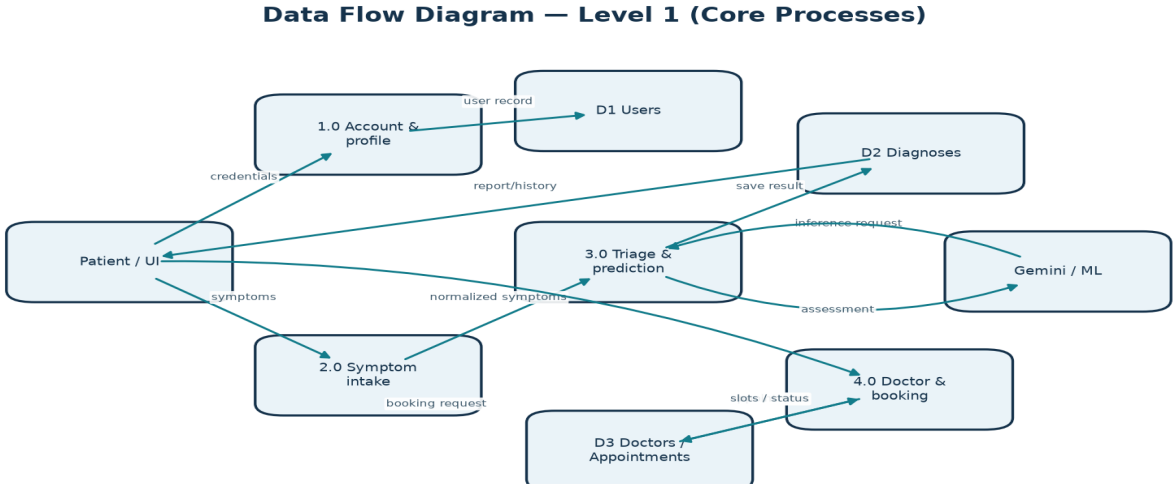


Figure 4. DFD Level 1: account, triage, prediction, provider, and data-store processes.

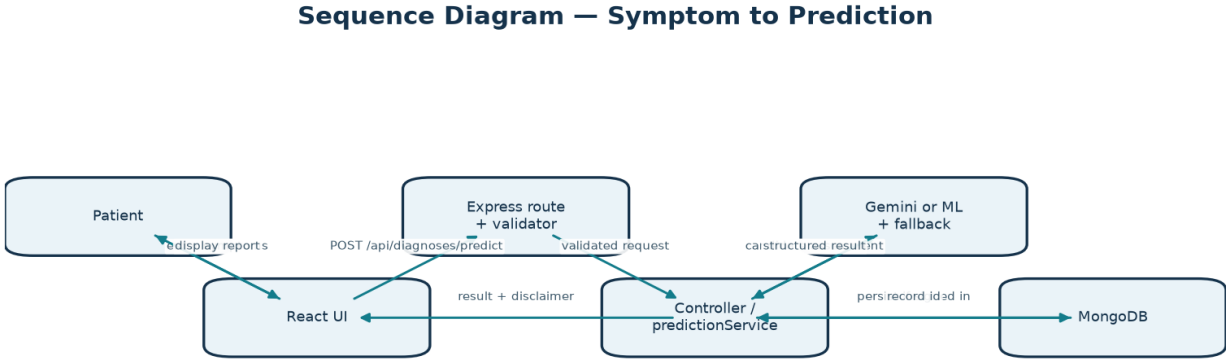


Figure 5. Sequence for symptom intake, candidate prediction, persistence, and response.

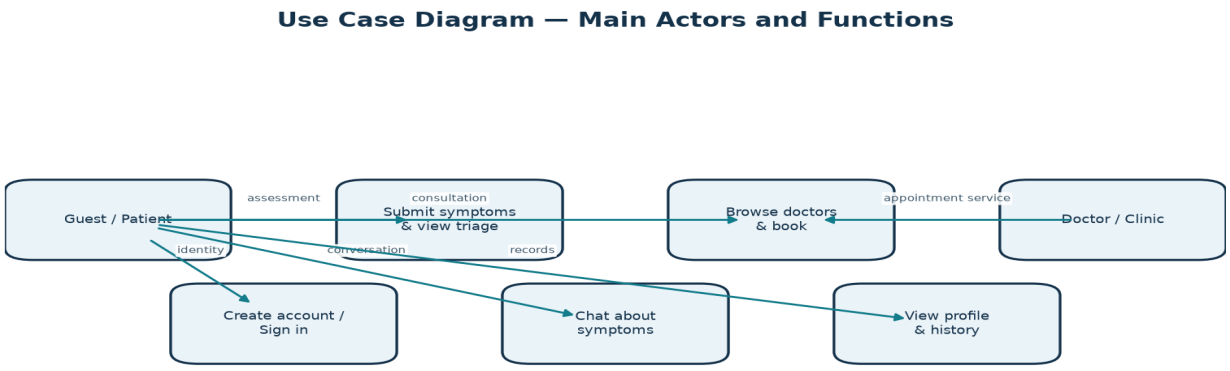


Figure 6. Main patient and provider use cases represented by current routes and pages.

7. Schema Design

The following summary is derived from the four Mongoose models. ObjectId references describe application relationships; timestamps are added by Mongoose where enabled. Nested diagnosis insight and top-candidate schemas are summarized separately.

Model	Field	Mongoose type / rule	Relationship / purpose
User	name	String; required, 2–50 chars	Account identity
User	email	String; unique, lowercase, validated	Login identifier
User	password	String; required, min 8, select:false	Stored credential hash
User	age, gender	Number; String enum	Optional profile details
User	dateOfBirth, bloodType	Date; String	Optional health profile
User	allergies	[String]	Optional profile list
Doctor	name, specialty	String; required	Provider display and classification
Doctor	email	String; unique, lowercase	Provider contact identifier
Doctor	available	Boolean; default true	Availability state
Doctor	rating	Number 0–5; default 5	Display rating
Doctor	consultationFee	Number ≥0; default constant	Displayed consultation fee
Doctor	availabilitySlots	[String]	Available booking slots
Appointment	user	ObjectId ref User; required, indexed	Many appointments belong to one user
Appointment	doctor	ObjectId ref Doctor; required, indexed	Many appointments reference a doctor
Appointment	doctorName, specialty	String; required	Snapshot fields for the booked provider
Appointment	appointmentDate	Date; required	Scheduled date
Appointment	timeSlot	String; required	Scheduled slot
Appointment	reason	String; max 300	Patient’s booking reason
Appointment	status	String enum; pending/confirmed/completed/cancelled	Booking lifecycle
Diagnosis	user	ObjectId ref User; required, indexed	Many diagnoses belong to one user
Diagnosis	symptoms	[String]	Submitted symptom identifiers
Diagnosis	predictedDisease	String; required	Primary preliminary condition
Diagnosis	confidence	Number 0–100	Displayed score with confidenceBasis
Diagnosis	confidenceBasis	Enum: model_probability, gemini_estimate, heuristic_overlap, rule_based, none	Meaning of score

Model	Field	Mongoose type / rule	Relationship / purpose
Diagnosis	severity	Enum: mild/moderate/severe	Triage urgency category
Diagnosis	topPredictions	[subdocument]	Ranked candidate conditions
Diagnosis	description, recommendation	String	Explanation and next-step guidance
Diagnosis	duration, notes	String	User-provided assessment context
Diagnosis	disclaimer	String	Mandatory clinical disclaimer
Diagnosis	geminiInsights	Nested subdocument	Explanation, warning signs, care and metadata
Diagnosis	predictionProvider, isFallback, status	String enum; Boolean; String enum	Provenance and lifecycle

Indexes and relationships

- Appointment has a compound index on user and appointmentDate descending.
- Diagnosis has a compound index on user and createdAt descending; it also exposes userId and possibleConditions virtuals.
- User email and Doctor email are unique. Appointment references both User and Doctor; Diagnosis references User.

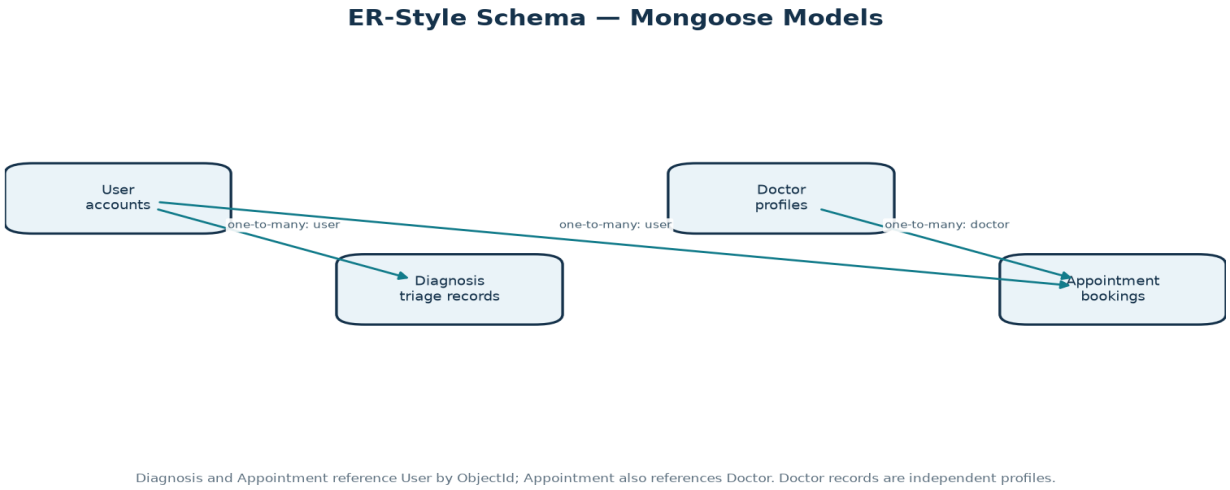


Figure 7. Mongoose relationship summary. Embedded diagnosis subdocuments are omitted for readability.

8. Front End

The route map is defined in `frontend/src/routes/AppRouter.jsx`. `/` redirects to `/dashboard` for authenticated users and `/login` otherwise. The dashboard, diagnosis, profile, chatbot, and appointments pages are protected. No browser screenshot was generated in this report build; clearly marked screenshot placeholders are provided below.

Home / Symptom Input

The home workspace is the Dashboard page. Symptom entry and returned assessment are implemented by DiagnosisForm; the project does not have a distinct home or report component file.

[Screenshot: Home / Dashboard]

[Screenshot: Symptom Input and Prediction Report]

Prediction Report

DiagnosisForm renders the API assessment response, candidate information, safety advice, and disclaimer in the symptom workflow. It is not a separate route.

Doctor Consultations and Booking Details

The appointments route combines doctor discovery, slot selection, booking submission, and appointment details/status. There is no separate doctor directory page in the route map.

[Screenshot: Doctor Consultations]

[Screenshot: Booking Details]

Chatbot

The chatbot page keeps local message, symptom, and conversation state; it sends requests to the chat API and supports a handoff to appointments.

[Screenshot: Chatbot]

Sign In and Sign Up

Login and Signup share the AuthShell component from Login.jsx and use AuthContext plus authService. They navigate into the dashboard after successful authentication.

[Screenshot: Sign In]

[Screenshot: Sign Up]

frontend/src/pages/Dashboard.jsx

Dashboard workspace, health history, navigation, and page-level state.

```
import { useState, useEffect } from 'react';
import { useAuth } from '../context/AuthContext.jsx';
import { appointmentService, diagnosisService } from '../services/api.js';

const formatHistoryDate = (timestamp) => {
  if (!timestamp) return 'Recent check-in';
  const date = new Date(timestamp);
  if (Number.isNaN(date.getTime())) return timestamp;

  const now = new Date();
  const diffDays = Math.floor((now - date) / (1000 * 60 * 60 * 24));

  if (diffDays === 0) {
    return `Today, ${date.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })}`;
  }
  if (diffDays === 1) {
    return `Yesterday, ${date.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })}`;
  }
  return date.toLocaleDateString('en-US', {
    month: 'short',
    day: 'numeric',
    year: 'numeric'
  });
};

const formatBookingDate = (value) => {
  if (!value) return 'Date not available';
  const date = new Date(value);
  if (Number.isNaN(date.getTime())) return 'Date not available';
  return new Intl.DateTimeFormat('en-US', { month: 'short', day: 'numeric', year: 'numeric' }).format(date);
};

const getBookingTimestamp = (booking) => {
  const timestamp = new Date(booking.appointmentDate || booking.date || 0).getTime();
  return Number.isNaN(timestamp) ? 0 : timestamp;
};

export default function Dashboard({ onNewDiagnosis, onChatbot, onProfile, onAppointments }) {
  const { user, signOut } = useAuth();

  const [diagnoses, setDiagnoses] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [selectedDiagnosis, setSelectedDiagnosis] = useState(null);
  const [deletingId, setDeletingId] = useState(null);
  const [showBookingDetails, setShowBookingDetails] = useState(false);
  const [bookings, setBookings] = useState([]);
  const [bookingsLoading, setBookingsLoading] = useState(false);
  const [bookingsError, setBookingsError] = useState('');

  const fetchHistory = async () => {
    try {
      setLoading(true);
      setError('');
      const res = await diagnosisService.getHistory(1, 10);
      const items = res?.data?.diagnoses || [];
      setDiagnoses(items);
    } catch (err) {
      setError(err?.message || 'Unable to load your check-in history.');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchHistory();
  }, []);

  const handleDelete = async (id, e) => {
    if (e) e.stopPropagation();
    if (!window.confirm('Are you sure you want to delete this check-in record?')) {
      return;
```

```
  }

  try {
    setDeletingId(id);
    await diagnosisService.delete(id);
    setDiagnoses((prev) => prev.filter((d) => (d._id || d.id) !== id));
    if ((selectedDiagnosis?._id || selectedDiagnosis?.id) === id) {
      setSelectedDiagnosis(null);
    }
  } catch (err) {
    alert(err?.message || 'Could not delete check-in record.');
  } finally {
    setDeletingId(null);
  }
};

const handleBookingDetails = async () => {
  if (showBookingDetails) {
    setShowBookingDetails(false);
    return;
  }

  setShowBookingDetails(true);
  setBookingsLoading(true);
  setBookingsError('');
  try {
    const response = await appointmentService.getAppointments();
    const items = response?.data?.appointments || [];
    setBookings([...items].sort((a, b) => getBookingTimestamp(b) - getBookingTimestamp(a)));
  } catch (err) {
    setBookings([]);
    setBookingsError(err?.message || 'Unable to load your bookings.');
  } finally {
    setBookingsLoading(false);
  }
};

return (
  <main className="app-shell">
    {/* TOP NAVIGATION BAR */}
    <header className="topbar">
      <div className="brand">
        <span className="brand-mark">+</span> carepath
      </div>
      <nav>
        <button className="nav-active" type="button">
          Overview
        </button>
        <button type="button" onClick={onProfile}>
          Health profile
        </button>
        <button type="button" onClick={onAppointments}>
          Consultations
        </button>
        <button type="button" onClick={onChatbot}>
          AI chat
        </button>
      </nav>
      <div className="user-menu">
        <div className="avatar">{user?.name?.[0] || 'U'}</div>
        <button className="signout" onClick={signOut} type="button">
          Sign out
        </button>
      </div>
    </header>

    {/* DASHBOARD MAIN CONTENT */}
    <section className="dashboard-content">
      {/* Welcome Section */}
      <div className="welcome">
        <div>
          <p className="eyebrow">HEALTH CHECK-IN & PREDICTION</p>
          <h1>Welcome, {user?.name?.split(' ')[0] || 'there'}.</h1>
          <p className="muted">
            Record symptoms and get AI-assisted differential diagnosis and triage support.
          </p>
        </div>
        <div style={{ display: 'flex', gap: '10px', flexWrap: 'wrap' }}>
          <button className="primary large" onClick={onNewDiagnosis} type="button">
            + Start a check-in
          </button>
          <button
            className="secondary-action-btn"
            onClick={handleBookingDetails}
            type="button"
            aria-expanded={showBookingDetails}
          >
            Booking Details
          </button>
        </div>
      </div>

      {showBookingDetails && (
        <section className="appointments-list" aria-live="polite">
          <h2>Booking Details</h2>
          {bookingsLoading ? (
            <p className="muted">Loading bookings...</p>
          ) : bookingsError ? (
            <p className="error">{bookingsError}</p>
          ) : bookings.length === 0 ? (
            <p className="muted">No bookings yet</p>
          ) : (
            bookings.map((booking, index) => {
              const status = booking.status === 'approved' ? 'approved' : 'pending';
              return (
```

```
            <article className="appointment-card" key={booking._id || booking.bookingId || index}>
              <div className="appointment-meta">
                <strong>{booking.doctorName || 'Doctor'}</strong>
                <span className="status green">{status}</span>
              </div>
              <p>{booking.specialty || 'Specialty not available'}</p>
              <small>
                {formatBookingDate(booking.appointmentDate || booking.date)} · {booking.timeSlot || booking.time || 'Time not available'}
              </small>
              <span className="appointment-reason">{booking.reason || 'No reason provided'}</span>
            </article>
          );
        })
      )}
    </section>
  )}

  {/* Safety Disclaimer Banner */}
  <div className="notice">
    <div className="notice-icon">i</div>
    <div>
      <strong>Carepath is here to support, not replace, your doctor.</strong>
      <p>For urgent or life-threatening symptoms, contact emergency services immediately.</p>
    </div>
  </div>

  {/* Activity / Diagnoses Section */}
  <div className="section-heading">
    <div>
      <p className="eyebrow">YOUR RECENT ASSESSMENTS</p>
      <h2>Diagnosis Check-Ins</h2>
    </div>
    {diagnoses.length > 0 && (
      <button className="text-button" onClick={onNewDiagnosis} type="button">
        + New check-in <span>&rarr;</span>
      </button>
    )}
  </div>

  {/* UI STATE TRIAD: Loading / Error / Empty / Data */}
  {loading && (
    <div style={{ padding: '36px 0', display: 'flex', flexDirection: 'column', gap: '14px' }}>
      <div style={{ height: '54px', background: '#eaf0eb', borderRadius: '8px' }} />
      <div style={{ height: '54px', background: '#eaf0eb', borderRadius: '8px', opacity: 0.6 }} />
    </div>
  )}

  {error && (
    <div
      style={{
        padding: '16px 20px',
        background: '#fdf3f2',
        border: '1px solid #f8d2ce',
        borderRadius: '8px',
        color: '#a83c30',
        marginBottom: '20px',
        display: 'flex',
        justifyContent: 'space-between',
        alignItems: 'center'
      }}
    >
      <span>{error}</span>
      <button className="text-button" onClick={fetchHistory} type="button">
        Retry
      </button>
    </div>
  )}

  {!loading && !error && diagnoses.length === 0 && (
    <div
      style={{
        padding: '48px 24px',
        background: '#ffffff',
        border: '1px dashed var(--line)',
        borderRadius: '12px',
        textAlign: 'center',
        marginBottom: '32px'
      }}
    >
      <p className="eyebrow">NO CHECK-INS RECORDED</p>
      <h3 style={{ font: '600 24px Fraunces, serif', margin: '8px 0 10px' }}>
        You haven't run any health predictions yet.
      </h3>
      <p className="muted" style={{ maxWidth: '420px', margin: '0 auto 24px' }}>
        Select your current symptoms to evaluate potential conditions with our diagnostic
        classifier.
      </p>
      <button className="primary" onClick={onNewDiagnosis} type="button">
        Find Health Prediction Now &rarr;
      </button>
    </div>
  )}

  {!loading && diagnoses.length > 0 && (
    <div className="history-list">
      {diagnoses.map((item) => {
        const itemId = item._id || item.id;
        const dateLabel = formatHistoryDate(item.createdAt);
        const disease = item.predictedDisease || 'General symptom pattern';
        const symptomCount = item.symptoms?.length || 0;
        const confidence = item.confidence || 0;
        const severity = item.severity || 'moderate';

        return (
          <article
```

```
              className="history-item"
              key={itemId}
              style={{ cursor: 'pointer' }}
              onClick={() => setSelectedDiagnosis(item)}
            >
              <div className="history-icon">+</div>
              <div className="history-main">
                <p>{dateLabel}</p>
                <h3>{disease}</h3>
                <span>
                  {symptomCount} symptom{symptomCount !== 1 ? 's' : ''} evaluated &bull;{' '}
                  {confidence}% confidence
                </span>
              </div>
              <span className={`status ${severity === 'mild' ? 'green' : 'blue'}`}>
                {severity.toUpperCase()}
              </span>
              <button
                className="arrow-button"
                aria-label={`Open details for ${disease}`}
                onClick={(e) => {
                  e.stopPropagation();
                  setSelectedDiagnosis(item);
                }}
                type="button"
              >
                &rarr;
              </button>
            </article>
          );
        })}
      </div>
    )}

    {/* MODAL DIALOG FOR INSPECTING A DIAGNOSIS */}
    {selectedDiagnosis && (
      <div
        className="modal-backdrop"
        onClick={() => setSelectedDiagnosis(null)}
        role="dialog"
        aria-modal="true"
      >
        <div className="modal-dialog" onClick={(e) => e.stopPropagation()}>
          <div className="modal-header">
            <div>
              <span className={`severity-pill ${selectedDiagnosis.severity || 'moderate'}`}>
                {(selectedDiagnosis.severity || 'moderate').toUpperCase()}
              </span>
              <h2 style={{ marginTop: '6px' }}>{selectedDiagnosis.predictedDisease}</h2>
              <p className="muted" style={{ fontSize: '13px', margin: '4px 0 0' }}>
                Recorded {formatHistoryDate(selectedDiagnosis.createdAt)} &bull;{' '}
                <strong>{selectedDiagnosis.confidence}%</strong> confidence
              </p>
            </div>
            <button
              type="button"
              className="modal-close-btn"
              onClick={() => setSelectedDiagnosis(null)}
              aria-label="Close dialog"
            >
              &times;
            </button>
          </div>

          {/* Condition Summary */}
          {selectedDiagnosis.description && (
            <p style={{ fontSize: '14px', color: '#3d4f48', margin: 0, lineHeight: 1.6 }}>
              {selectedDiagnosis.description}
            </p>
          )}

          {/* Evaluated Symptoms */}
          <div>
            <span className="detail-label">Evaluated Symptoms:</span>
            <div className="selected-chips-grid" style={{ marginTop: '8px' }}>
              {selectedDiagnosis.symptoms?.map((s) => (
                <span className="selected-chip" key={s}>
                    {s.replace(/_/g, ' ')}
                </span>
              ))}
            </div>
          </div>

          {/* Differential Candidates */}
          {selectedDiagnosis.topPredictions?.length > 0 && (
            <div className="differential-section" style={{ padding: '16px' }}>
              <h3 style={{ fontSize: '12px', marginBottom: '10px' }}>Differential Possibilities</h3>
              <div className="diff-list">
                {selectedDiagnosis.topPredictions.map((pred, i) => (
                  <div className="diff-row" key={pred.disease || i}>
                    <div className="diff-meta">
                      <span>{pred.disease}</span>
                      <strong>{pred.confidence}%</strong>
                    </div>
                    <div className="progress-track">
                      <div
                        className="progress-fill"
                        style={{
                          width: `${Math.min(100, Math.max(5, pred.confidence))}%`
                        }}
                      />
                    </div>
                  </div>
                ))}
              </div>
```

```
              </div>
            )}

            {/* Clinical Care Recommendation */}
            {selectedDiagnosis.recommendation && (
              <aside className="recommendation-panel" style={{ padding: '14px 16px' }}>
                <h4 style={{ fontSize: '11px' }}>Care Recommendation</h4>
                <p style={{ fontSize: '12px' }}>{selectedDiagnosis.recommendation}</p>
              </aside>
            )}

            {/* Medical Safety Disclaimer */}
            <div className="medical-disclaimer-box" style={{ padding: '12px' }}>
              <p className="disclaimer-text" style={{ fontSize: '11px' }}>
                <strong>Disclaimer: </strong>
                {selectedDiagnosis.disclaimer ||
                  'MedInsight provides preliminary, AI-assisted symptom triage guidance for informational purposes only. It is not a clinical medical
    diagnosis.'}
              </p>
            </div>

            {/* Modal Actions */}
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginTop: '10px' }}>
              <button
                type="button"
                style={{ color: '#b24e43', fontSize: '12px', fontWeight: 600 }}
                disabled={deletingId === (selectedDiagnosis._id || selectedDiagnosis.id)}
                onClick={(e) => handleDelete(selectedDiagnosis._id || selectedDiagnosis.id, e)}
              >
                {deletingId === (selectedDiagnosis._id || selectedDiagnosis.id) ? 'Deleting...' : '   Delete record'}
              </button>

              <button
                className="primary"
                type="button"
                onClick={() => setSelectedDiagnosis(null)}
              >
                Close
              </button>
            </div>
          </div>
        </div>
      )}

      {/* Dashboard Grid Panels */}
      <div className="dashboard-grid">
        <div className="soft-panel">
          <p className="eyebrow">YOUR CARE CIRCLE</p>
          <h2>Keep your health story close.</h2>
          <p className="muted">Complete your health profile so your future check-ins have more context.</p>
          <button className="text-button" onClick={onProfile} type="button">
            Complete profile <span>&rarr;</span>
          </button>
        </div>

        <div className="soft-panel accent-panel">
          <p className="eyebrow">FOLLOW-UP CARE</p>
          <h2>Need a doctor review?</h2>
          <p className="muted">Book a consultation with a specialist best matched to your recent symptom pattern.</p>
          <button className="text-button" onClick={onAppointments} type="button">
            Book a consultation <span>&rarr;</span>
          </button>
        </div>

        <div className="soft-panel">
          <p className="eyebrow">CONVERSATIONAL CHECK-IN</p>
          <h2>Talk through how you feel.</h2>
          <p className="muted">
            Share symptoms in your own words and get supportive triage guidance.
          </p>
          <button className="text-button" onClick={onChatbot} type="button">
            Start an AI chat <span>&rarr;</span>
          </button>
        </div>

        <div className="quote-panel">
          <span className="quote-mark">“</span>
          <p>Health is not a destination. It is a way of travelling.</p>
          <small>— Carepath</small>
        </div>
      </div>
    </section>
  </main>
 );
}
```

frontend/src/pages/DiagnosisForm.jsx

Symptom input, request flow, assessment/report rendering, and handoff.

```
import { useState, useEffect, useMemo, useRef } from 'react';
import { useAuth } from '../context/AuthContext.jsx';
import { diagnosisService } from '../services/api.js';

// Fallback symptom taxonomy in case network is disconnected
const defaultPopularSymptoms = [
  { id: 'headache', name: 'Headache' },
  { id: 'high_fever', name: 'High Fever' },
  { id: 'cough', name: 'Cough' },
  { id: 'fatigue', name: 'Fatigue' },
  { id: 'chills', name: 'Chills' },
  { id: 'nausea', name: 'Nausea' },
```

```
  { id: 'breathlessness', name: 'Shortness of Breath' },
  { id: 'joint_pain', name: 'Joint Pain' },
  { id: 'chest_pain', name: 'Chest Pain' },
  { id: 'dizziness', name: 'Dizziness' },
  { id: 'skin_rash', name: 'Skin Rash' },
  { id: 'throat_irritation', name: 'Sore / Irritated Throat' },
  { id: 'stomach_pain', name: 'Stomach Pain' },
  { id: 'muscle_pain', name: 'Muscle Pain' }
];

export default function DiagnosisForm({ onBack, onBookAppointment }) {
  const { user } = useAuth();

  const [availableSymptoms, setAvailableSymptoms] = useState(defaultPopularSymptoms);
  const [selectedSymptoms, setSelectedSymptoms] = useState([]);
  const [searchQuery, setSearchQuery] = useState('');
  const [dropdownOpen, setDropdownOpen] = useState(false);
  const [form, setForm] = useState({
    duration: '1–3 days',
    severity: 'moderate',
    notes: ''
  });

  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [prediction, setPrediction] = useState(null);

  const searchContainerRef = useRef(null);

  // Fetch full 130+ symptoms master catalog from API
  useEffect(() => {
    let isMounted = true;
    const loadSymptoms = async () => {
      try {
        const response = await diagnosisService.getSymptoms();
        const symptomsList = response?.data?.symptoms;
        if (isMounted && Array.isArray(symptomsList) && symptomsList.length > 0) {
          setAvailableSymptoms(symptomsList);
        }
      } catch (_e) {
        // Retain default popular list if offline
      }
    };
    loadSymptoms();
    return () => {
      isMounted = false;
    };
  }, []);

  // Close dropdown when clicking outside
  useEffect(() => {
    const handleClickOutside = (event) => {
      if (searchContainerRef.current && !searchContainerRef.current.contains(event.target)) {
        setDropdownOpen(false);
      }
    };
    document.addEventListener('mousedown', handleClickOutside);
    return () => document.removeEventListener('mousedown', handleClickOutside);
  }, []);

  // Fuzzy/partial search: matches if any word in query starts with any word in the symptom name
  // This allows "heand" to match "headache", "sto" to match "stomach pain", etc.
  const fuzzyMatch = (symptomName, symptomId, query) => {
    const name = symptomName.toLowerCase();
    const id = symptomId.toLowerCase().replace(/_/g, ' ');
    // Exact substring match first (fast path)
    if (name.includes(query) || id.includes(query)) return true;
    // Word-start prefix match: each query word must be a prefix of some word in the symptom name
    const queryWords = query.split(/\s+/).filter(Boolean);
    const nameWords = (name + ' ' + id).split(/\s+/);
    return queryWords.every((qw) => nameWords.some((nw) => nw.startsWith(qw)));
  };

  const filteredSymptoms = useMemo(() => {
    const query = searchQuery.trim().toLowerCase();
    if (!query) return [];
    return availableSymptoms
      .filter((s) => !selectedSymptoms.includes(s.id))
      .filter((s) => fuzzyMatch(s.name, s.id, query))
      .slice(0, 8);
  }, [availableSymptoms, selectedSymptoms, searchQuery]);

  // Converts free-text into a symptom-id style key and adds it to selected symptoms
  const handleAddFreeText = (rawText) => {
    const text = rawText.trim();
    if (!text) return;
    // Convert to underscore-id: "leg break" → "leg_break"
    const id = text.toLowerCase().replace(/\s+/g, '_').replace(/[^a-z0-9_]/g, '');
    if (id && !selectedSymptoms.includes(id)) {
      setSelectedSymptoms((current) => [...current, id]);
    }
    setSearchQuery('');
    setDropdownOpen(false);
    setError('');
  };

  const toggleSymptom = (symptomId) => {
    setError('');
    setSelectedSymptoms((current) =>
      current.includes(symptomId)
        ? current.filter((id) => id !== symptomId)
        : [...current, symptomId]
    );
  };
```

```
const handleSelectFromSearch = (symptomId) => {
  if (!selectedSymptoms.includes(symptomId)) {
    setSelectedSymptoms((current) => [...current, symptomId]);
  }
  setSearchQuery('');
  setDropdownOpen(false);
};

const removeSymptom = (symptomId) => {
  setSelectedSymptoms((current) => current.filter((id) => id !== symptomId));
};

const clearAllSymptoms = () => {
  setSelectedSymptoms([]);
  setError('');
};

const formatSymptomName = (id) => {
  const found = availableSymptoms.find((s) => s.id === id);
  if (found) return found.name;
  // For free-text custom symptoms, format nicely: "leg_break" → "Leg Break"
  return id.replace(/_/g, ' ').replace(/\b\w/g, (c) => c.toUpperCase());
};

const handleSubmit = async (e) => {
  e.preventDefault();

  let symptomsToSubmit = [...selectedSymptoms];

  // If user typed into the input (e.g. "leg break") and didn't manually click the dropdown, auto-include it!
  if (searchQuery.trim()) {
    const cleanId = searchQuery.trim().toLowerCase().replace(/\s+/g, '_').replace(/[^a-z0-9_]/g, '');
    if (cleanId && !symptomsToSubmit.includes(cleanId)) {
      symptomsToSubmit.push(cleanId);
      setSelectedSymptoms(symptomsToSubmit);
    }
    setSearchQuery('');
    setDropdownOpen(false);
  }

  if (symptomsToSubmit.length === 0) {
    setError('Please type or select at least one symptom or health problem before submitting.');
    return;
  }

  setLoading(true);
  setError('');

  try {
    const payload = {
      symptoms: symptomsToSubmit,
      duration: form.duration,
      severity: form.severity,
      notes: form.notes
    };

    const response = await diagnosisService.predict(payload);
    if (response?.data) {
      setPrediction(response.data);
    } else {
      throw new Error(response?.message || 'Inference engine did not return diagnostic data.');
    }
  } catch (err) {
    setError(
      err.message ||
        'Diagnostic evaluation encountered an unexpected error. Please check your network and try again.'
    );
  } finally {
    setLoading(false);
  }
};

const resetAssessment = () => {
  setPrediction(null);
  setSelectedSymptoms([]);
  setForm({ duration: '1–3 days', severity: 'moderate', notes: '' });
  setError('');
};

// ---------------------------------------------------------------------------
// RENDER: PREDICTION RESULT VIEW
// ---------------------------------------------------------------------------
if (prediction) {
  const topPredictions = prediction.topPredictions || [];
  const primaryConfidence = prediction.confidence || 0;
  const severity = prediction.severity || 'moderate';
  // Use the low_confidence flag from the ML service (set when only 1 symptom is recognized)
  const isLowConfidence = prediction.low_confidence === true;
  // The predicted condition is always grounded by the ML/heuristic engine;
  // Gemini only supplies the candidate explanations rendered below.
  const isGeminiPrimary = prediction.predictionProvider === 'gemini';
  // What the displayed confidence number is: a validated model probability
  // vs a preliminary (non-probability) score — controls the text labels.
  const isModelProbability = prediction.confidenceBasis === 'model_probability';

  return (
    <main className="app-shell">
      <header className="topbar">
        <div className="brand">
          <span className="brand-mark">+</span> carepath
        </div>
        <button className="signout" onClick={onBack} type="button">
          &larr; Back to overview
        </button>
      </header>
```

```
    <section className="dashboard-content" style={{ maxWidth: '820px', paddingBlock: '48px' }}>
      <div className="section-heading" style={{ marginBottom: '24px' }}>
        <div>
          <p className="eyebrow">DIAGNOSTIC TRIAGE ANALYSIS</p>
          <h1 style={{ font: '600 42px/1.1 Fraunces, serif', margin: 0 }}>
            Health Prediction Report
          </h1>
        </div>
        <button className="text-button" onClick={resetAssessment} type="button">
          + New check-in
        </button>
      </div>

      <div className="prediction-result-wrapper">
        {/* Low-confidence advisory banner (shown for single-symptom results) */}
        {isLowConfidence && prediction.low_confidence_message && (
          <div style={{
            padding: '12px 16px',
            background: '#fffbea',
            border: '1px solid #f0d060',
            borderRadius: '10px',
            color: '#7a6000',
            fontSize: '13px',
            marginBottom: '16px',
            display: 'flex',
            gap: '10px',
            alignItems: 'flex-start'
          }}>
            <span style={{ fontSize: '16px' }}>  </span>
            <span>{prediction.low_confidence_message}</span>
          </div>
        )}

        {/* Dual AI ensemble consensus card — shown only in ML-primary (dual) mode */}
        {prediction.geminiInsights && !isGeminiPrimary && (
          <section
            style={{
              background: 'linear-gradient(135deg, #f7faf8 0%, #edf5f0 100%)',
              border: '1px solid #c2decb',
              borderRadius: '14px',
              padding: '24px',
              marginBottom: '20px',
              position: 'relative'
            }}
          >
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', flexWrap: 'wrap', gap: '10px', marginBottom:
'16px' }}>
              <div style={{ display: 'flex', alignItems: 'center', gap: '8px' }}>
                <span style={{ fontSize: '18px' }}> </span>
                <strong style={{ fontSize: '14px', color: '#1d543e', textTransform: 'uppercase', letterSpacing: '0.06em' }}>
                  {prediction.accuracyMetrics
                    ? `Dual AI Clinical Consensus (ML + ${prediction.geminiInsights?.poweredBy || 'Gemini'})`
                    : `ML Prediction + ${prediction.geminiInsights?.poweredBy || 'Gemini'} Clinical Explanation`}
                </strong>
              </div>
              <span
                style={{
                  background: '#1d543e',
                  color: '#ffffff',
                  fontSize: '11px',
                  fontWeight: 700,
                  padding: '4px 10px',
                  borderRadius: '20px',
                  letterSpacing: '0.04em'
                }}
              >
                {prediction.accuracyMetrics?.validationStatus || 'Symptom-Grounded Assessment'}
              </span>
            </div>

            {/* Combined Consensus Banner */}
            {prediction.combinedPrediction && (
              <div
                style={{
                  background: '#ffffff',
                  border: '1px solid #d4e8da',
                  borderRadius: '10px',
                  padding: '16px',
                  marginBottom: '16px',
                  display: 'flex',
                  justifyContent: 'space-between',
                  alignItems: 'center',
                  flexWrap: 'wrap',
                  gap: '12px'
                }}
              >
                <div>
                  <small style={{ color: '#688273', fontSize: '11px', textTransform: 'uppercase', fontWeight: 700 }}>
                    Synthesized Consensus Prediction
                  </small>
                  <h3 style={{ margin: '2px 0 0', fontSize: '20px', color: '#163f2f', fontFamily: 'Fraunces, serif' }}>
                    {prediction.combinedPrediction.predictedDisease}
                  </h3>
                  <small style={{ color: '#4d6959', fontSize: '12px' }}>
                    {prediction.combinedPrediction.method}
                  </small>
                </div>
                <div style={{ textAlign: 'right' }}>
                  <span style={{ fontSize: '24px', fontWeight: 700, color: '#1d543e' }}>
                    {prediction.combinedPrediction.confidence > 0 ? `${prediction.combinedPrediction.confidence}%` : '—'}
                  </span>
                  <small style={{ display: 'block', fontSize: '10px', color: '#688273', textTransform: 'uppercase' }}>
                    {isModelProbability ? 'Model Probability' : 'Preliminary Score'}
                  </small>
```

```
            </div>
          </div>
        ))}

        {/* Dual Model Side-by-Side Comparison */}
        <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(240px, 1fr))', gap: '14px', marginBottom: '16px' }}>
          {/* ML Model Card */}
          <div style={{ background: '#ffffff', border: '1px solid #e1ebe4', borderRadius: '10px', padding: '14px' }}>
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '6px' }}>
              <small style={{ fontWeight: 700, color: '#426854', textTransform: 'uppercase', fontSize: '11px' }}>
                  ML Classifier
              </small>
              <span style={{ fontWeight: 700, color: '#1d543e', fontSize: '13px' }}>
                {prediction.confidence > 0 ? `${prediction.confidence}%` : '—'}
              </span>
            </div>
            <strong style={{ display: 'block', fontSize: '16px', color: '#163f2f', marginBottom: '4px' }}>
              {prediction.predictedDisease}
            </strong>
            <small style={{ color: '#748b7d', fontSize: '11px' }}>
              {prediction.isFallback
                ? 'Heuristic Rule Fallback'
                : isModelProbability
                ? 'Trained Probabilistic Classifier'
                : 'Awaiting More Symptoms'}
            </small>
          </div>

          {/* Gemini model card (dual mode) */}
          <div style={{ background: '#ffffff', border: '1px solid #e1ebe4', borderRadius: '10px', padding: '14px' }}>
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '6px' }}>
              <small style={{ fontWeight: 700, color: '#2563eb', textTransform: 'uppercase', fontSize: '11px' }}>
                  {prediction.geminiInsights?.poweredBy || 'Gemini'}
              </small>
              <span style={{ fontWeight: 700, color: '#2563eb', fontSize: '13px' }}>
                {prediction.geminiInsights.geminiConfidence > 0 ? `${prediction.geminiInsights.geminiConfidence}%` : '—'}
              </span>
            </div>
            <strong style={{ display: 'block', fontSize: '16px', color: '#1e293b', marginBottom: '4px' }}>
              {prediction.geminiInsights.geminiPredictedDisease}
            </strong>
            <small style={{ color: '#64748b', fontSize: '11px' }}>
              {prediction.accuracyMetrics?.modelAgreement
                ? '  Agreed with ML model'
                : prediction.geminiEnriched
                ? 'Explains supporting & missing symptoms for ML candidates'
                : 'Local rule-based explanation of ML candidates'}
            </small>
          </div>
        </div>

        {/* Accuracy Metrics Bar */}
        {prediction.accuracyMetrics && (
          <div
            style={{
              background: 'rgba(255, 255, 255, 0.7)',
              border: '1px solid #d4e8da',
              borderRadius: '8px',
              padding: '10px 14px',
              display: 'flex',
              justifyContent: 'space-around',
              flexWrap: 'wrap',
              gap: '12px',
              textAlign: 'center'
            }}
          >
            <div>
              <small style={{ display: 'block', fontSize: '10px', color: '#688273', textTransform: 'uppercase' }}>
                Agreement Concordance
              </small>
              <strong style={{ color: '#1d543e', fontSize: '14px' }}>
                {prediction.accuracyMetrics.agreementScore}%
              </strong>
            </div>
            <div>
              <small style={{ display: 'block', fontSize: '10px', color: '#688273', textTransform: 'uppercase' }}>
                Core Symptom Coverage
              </small>
              <strong style={{ color: '#1d543e', fontSize: '14px' }}>
                {prediction.accuracyMetrics.symptomCoverageRatio}%
              </strong>
            </div>
            <div>
              <small style={{ display: 'block', fontSize: '10px', color: '#688273', textTransform: 'uppercase' }}>
                Consensus Level
              </small>
              <strong style={{ color: '#1d543e', fontSize: '14px', textTransform: 'capitalize' }}>
                {prediction.accuracyMetrics.consensusLevel}
              </strong>
            </div>
          </div>
        )}
      </section>
    )}

    {/* Hero Condition Card */}
    <article className="prediction-hero">
      <div className="prediction-hero-header">
        <div className="condition-title-group">
          <div style={{ display: 'flex', gap: '8px', alignItems: 'center', flexWrap: 'wrap' }}>
            <span className={`severity-pill ${severity}`}>
              {severity === 'severe'
                ? '   Needs Medical Attention'
                : severity === 'moderate'
                ? '  Moderate Impact'
```

```
                : '  Mild Condition'}
              </span>
              {prediction.geminiInsights?.urgencyAssessment && (
                <span
                  style={{
                    fontSize: '11px',
                    fontWeight: 700,
                    textTransform: 'uppercase',
                    letterSpacing: '0.06em',
                    padding: '3px 8px',
                    borderRadius: '12px',
                    background:
                      prediction.geminiInsights.urgencyAssessment === 'high'
                        ? '#fee2e2'
                        : prediction.geminiInsights.urgencyAssessment === 'low'
                        ? '#dcfce7'
                        : '#fef3c7',
                    color:
                      prediction.geminiInsights.urgencyAssessment === 'high'
                        ? '#b91c1c'
                        : prediction.geminiInsights.urgencyAssessment === 'low'
                        ? '#15803d'
                        : '#b45309'
                  }}
                >
                  Urgency: {prediction.geminiInsights.urgencyAssessment}
                </span>
              )}
            </div>
            <h2>{prediction.primary_condition || prediction.predictedDisease || 'Unknown Condition'}</h2>
            <p className="eyebrow" style={{ color: '#8a9c92', margin: '4px 0 0' }}>
              Primary Condition Identified
            </p>
          </div>

          <div className="confidence-score-badge">
            <span className="score">
              {primaryConfidence > 0 ? `${primaryConfidence}%` : '—'}
            </span>
            <span className="score-label">
              {isModelProbability ? 'Model Probability' : primaryConfidence > 0 ? 'Preliminary Score' : 'Preliminary Assessment'}
            </span>
          </div>
        </div>

        <p className="condition-summary-text">
          {prediction.description ||
            'The diagnostic model analyzed your symptom pattern and matched clinical markers associated with this condition.'}
        </p>

        {/* Analyzed Symptoms — API returns 'recognized_symptoms' array */}
        <div style={{ marginTop: '14px', borderTop: '1px solid var(--line)', paddingTop: '14px' }}>
          <span className="detail-label">Analyzed Symptoms:</span>
          <div className="selected-chips-grid" style={{ marginTop: '8px' }}>
            {(prediction.recognized_symptoms || prediction.symptoms || []).map((s) => (
              <span className="selected-chip" key={s}>
                  {formatSymptomName(s)}
              </span>
            ))}
          </div>
        </div>
      </article>

      {/* Gemini Clinical Explanation & Pattern Insights */}
      {prediction.geminiInsights && (
        <section
          style={{
            background: '#ffffff',
            border: '1px solid var(--line)',
            borderRadius: '12px',
            padding: '24px',
            display: 'flex',
            flexDirection: 'column',
            gap: '16px'
          }}
        >
          <div>
            <div style={{ display: 'flex', alignItems: 'center', gap: '8px', marginBottom: '8px' }}>
              <span style={{ fontSize: '18px' }}> </span>
              <h3 style={{ margin: 0, fontSize: '18px', color: 'var(--ink)' }}>
                Clinical AI Explanation & Pattern Analysis
              </h3>
            </div>
            <p style={{ margin: '0 0 12px', color: 'var(--ink)', lineHeight: '1.6', fontSize: '14px' }}>
              {prediction.geminiInsights.clinicalExplanation}
            </p>
          </div>

          {prediction.geminiInsights.symptomPatternInsight && (
            <div style={{ background: '#f8faf9', borderLeft: '4px solid #236b58', padding: '12px 16px', borderRadius: '4px' }}>
              <strong style={{ display: 'block', fontSize: '12px', color: '#236b58', textTransform: 'uppercase', marginBottom: '4px' }}>
                Symptom Pattern Insight
              </strong>
              <span style={{ fontSize: '13px', color: '#2e493b', lineHeight: '1.5' }}>
                {prediction.geminiInsights.symptomPatternInsight}
              </span>
            </div>
          )}

          {prediction.geminiInsights.differentialReasoning && (
            <div style={{ background: '#f8faf9', borderLeft: '4px solid #4a7a8c', padding: '12px 16px', borderRadius: '4px' }}>
              <strong style={{ display: 'block', fontSize: '12px', color: '#2b5768', textTransform: 'uppercase', marginBottom: '4px' }}>
                Differential Reasoning
              </strong>
              <span style={{ fontSize: '13px', color: '#2b4754', lineHeight: '1.5' }}>
```

```
                  {prediction.geminiInsights.differentialReasoning}
                </span>
              </div>
            )}

            {prediction.geminiInsights.confidenceInterpretation && (
              <small style={{ color: 'var(--muted)', fontSize: '12px', fontStyle: 'italic' }}>
                  {prediction.geminiInsights.confidenceInterpretation}
              </small>
            )}
          </section>
        )}

        {/* Differential Diagnosis (Top-3 Predictions) */}
        {topPredictions.length > 0 && (
          <section className="differential-section">
            <h3>Differential Diagnostic Possibilities (Top 3 Candidates)</h3>
            <div className="diff-list">
              {topPredictions.map((candidate, idx) => (
                <div className="diff-row" key={candidate.disease || idx}>
                  <div className="diff-meta">
                    <strong>
                      {idx + 1}. {candidate.disease}
                    </strong>
                    <span>
                      {candidate.confidence}% {isModelProbability ? 'probability' : 'preliminary score'}
                    </span>
                  </div>
                  <div className="progress-track">
                    <div
                      className="progress-fill"
                      style={{
                        width: `${Math.min(100, Math.max(5, candidate.confidence))}%`,
                        background:
                          idx === 0
                            ? 'linear-gradient(90deg, #236b58, #3baa84)'
                            : '#8ea99d'
                      }}
                    />
                  </div>
                  {candidate.description && (
                    <small style={{ color: 'var(--muted)', fontSize: '11px', marginTop: '2px' }}>
                      {candidate.description}
                    </small>
                  )}
                </div>
              ))}
            </div>
          </section>
        )}

        {/* Gemini Guided Self-Care & Red Flags */}
        {prediction.geminiInsights && (prediction.geminiInsights.selfCareSteps?.length > 0 || prediction.geminiInsights.warningSignsToWatch?.length
> 0) && (
          <section style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(280px, 1fr))', gap: '16px' }}>
            {prediction.geminiInsights.selfCareSteps?.length > 0 && (
              <div style={{ background: '#f4f9f6', border: '1px solid #d1e7da', borderRadius: '12px', padding: '18px' }}>
                <h4 style={{ margin: '0 0 10px', color: '#1e5942', fontSize: '14px', textTransform: 'uppercase', letterSpacing: '0.04em' }}>
                    Supportive Self-Care Steps
                </h4>
                <ul style={{ margin: 0, paddingLeft: '20px', fontSize: '13px', color: '#274b39', lineHeight: '1.6' }}>
                  {prediction.geminiInsights.selfCareSteps.map((step, idx) => (
                    <li key={idx} style={{ marginBottom: '6px' }}>{step}</li>
                  ))}
                </ul>
              </div>
            )}

            {prediction.geminiInsights.warningSignsToWatch?.length > 0 && (
              <div style={{ background: '#fdf6f5', border: '1px solid #f6cfc9', borderRadius: '12px', padding: '18px' }}>
                <h4 style={{ margin: '0 0 10px', color: '#991b1b', fontSize: '14px', textTransform: 'uppercase', letterSpacing: '0.04em' }}>
                     Warning Signs to Watch
                </h4>
                <ul style={{ margin: 0, paddingLeft: '20px', fontSize: '13px', color: '#7f1d1d', lineHeight: '1.6' }}>
                  {prediction.geminiInsights.warningSignsToWatch.map((sign, idx) => (
                    <li key={idx} style={{ marginBottom: '6px' }}>{sign}</li>
                  ))}
                </ul>
              </div>
            )}
          </section>
        )}

        {/* Clinical Recommendation Box */}
        <aside className="recommendation-panel">
          <h4>Recommended Care Steps</h4>
          <p>
            {prediction.recommendation ||
              'Monitor your condition carefully. Maintain proper hydration and rest, and consult a qualified healthcare professional if your
discomfort continues.'}
          </p>
        </aside>

        {/* Non-dismissible Mandatory Medical Disclaimer */}
        <aside className="medical-disclaimer-box" role="alert">
          <span className="disclaimer-icon">  </span>
          <p className="disclaimer-text">
            <strong>Mandatory Medical Disclaimer: </strong>
            {prediction.disclaimer ||
              'MedInsight provides preliminary, AI-assisted symptom triage guidance for informational purposes only. It is not a clinical medical
diagnosis. In case of emergency or severe discomfort, please consult a certified doctor or contact your local emergency services immediately.'}
          </p>
        </aside>

        {/* Action Buttons */}
```

```
          <div className="prediction-button-row">
            <button
              className="primary"
              type="button"
              onClick={() => {
                if (onBookAppointment) {
                  const diseaseName = prediction.primary_condition || prediction.predictedDisease || 'health condition';
                  const conf = prediction.confidence || 0;
                  onBookAppointment({
                    reason: `Follow-up for AI health triage: ${diseaseName} (${conf}% confidence)`
                  });
                } else {
                  onBack();
                }
              }}
            >
              Book Specialist Consultation &rarr;
            </button>

            <button className="secondary-action-btn" onClick={resetAssessment} type="button">
              Start Another Check-In
            </button>

            {isLowConfidence && (
              <button
                className="secondary-action-btn"
                onClick={() => {
                  setPrediction(null);
                  setError('');
                }}
                type="button"
              >
                Add More Symptoms
              </button>
            )}

            <button className="secondary-action-btn" onClick={onBack} type="button">
              Return to Overview
            </button>
          </div>
        </div>
      </section>
    </main>
  );
}

// ---------------------------------------------------------------------------
// RENDER: LOADING STATE (INFERENCE ANALYSIS IN PROGRESS)
// ---------------------------------------------------------------------------
if (loading) {
  return (
    <main className="app-shell">
      <header className="topbar">
        <div className="brand">
          <span className="brand-mark">+</span> carepath
        </div>
      </header>

      <section className="form-page" style={{ maxWidth: '640px', paddingBlock: '80px' }}>
        <div className="analysis-loader-card">
          <div className="pulse-spinner" />
          <h2 style={{ font: '600 32px/1.2 Fraunces, serif', margin: '0 0 8px' }}>
            Evaluating Health Prediction...
          </h2>
          <p className="muted" style={{ maxWidth: '440px', margin: '0 auto' }}>
            Our probabilistic classifier is analyzing your symptom pattern and cross-referencing
            differential diagnosis models.
          </p>

          <div className="analysis-steps">
            <span>  Vectorizing multi-hot symptom indicators...</span>
            <span>  Running calibrated Bayesian classification...</span>
            <span>  Generating clinical triage guidance...</span>
          </div>
        </div>
      </section>
    </main>
  );
}

// ---------------------------------------------------------------------------
// RENDER: SYMPTOM SELECTION FORM
// ---------------------------------------------------------------------------
return (
  <main className="app-shell">
    <header className="topbar">
      <div className="brand">
        <span className="brand-mark">+</span> carepath
      </div>
      <button className="signout" onClick={onBack} type="button">
        Cancel
      </button>
    </header>

    <section className="form-page" style={{ maxWidth: '720px' }}>
      <div className="form-intro">
        <p className="eyebrow">AI HEALTH PREDICTION & TRIAGE</p>
        <h1>Find your health prediction.</h1>
        <p className="muted">
          Search or select the symptoms you are currently experiencing to run our diagnostic inference
          engine. This provides supportive triage guidance, not a medical diagnosis.
        </p>
      </div>

      {error && (
```

```
          <div
            style={{
              padding: '14px 16px',
              background: '#fdf2f0',
              border: '1px solid #f6cfc9',
              borderRadius: '8px',
              color: '#a83c30',
              fontSize: '13px',
              marginBottom: '20px'
            }}
          >
            {error}
          </div>
        )}

        <form className="checkin-form" onSubmit={handleSubmit}>
          {/* SYMPTOM PICKER SECTION */}
          <fieldset>
            <legend style={{ fontSize: '18px', marginBottom: '4px' }}>
              What symptoms are you experiencing?
            </legend>
            <p className="field-hint">
              Search across 130+ symptoms or select common indicators below.
            </p>

            {/* Live Autocomplete Search Input */}
            <div className="symptom-search-container" ref={searchContainerRef}>
              <div style={{ display: 'flex', gap: '8px', alignItems: 'center' }}>
                <div className="symptom-search-box" style={{ flex: 1, position: 'relative' }}>
                  <span className="symptom-search-icon">🔍</span>
                  <input
                    type="text"
                    className="symptom-search-input"
                    placeholder="Type any symptom, injury, or problem (e.g. leg break, headache, vomiting)..."
                    value={searchQuery}
                    onChange={(e) => {
                      setSearchQuery(e.target.value);
                      setDropdownOpen(true);
                    }}
                    onFocus={() => setDropdownOpen(true)}
                    onKeyDown={(e) => {
                      // Press Enter to add the typed text as a symptom (even if no dropdown match)
                      if (e.key === 'Enter') {
                        e.preventDefault();
                        if (searchQuery.trim()) {
                          handleAddFreeText(searchQuery);
                        }
                      }
                    }}
                    aria-label="Search symptoms"
                  />
                  {searchQuery && (
                    <button
                      type="button"
                      className="symptom-search-clear"
                      onClick={() => setSearchQuery('')}
                      aria-label="Clear search"
                    >
                      ×
                    </button>
                  )}
                </div>
                {searchQuery.trim() && (
                  <button
                    type="button"
                    className="primary"
                    style={{
                      padding: '10px 16px',
                      whiteSpace: 'nowrap',
                      fontSize: '13px',
                      borderRadius: '8px',
                      fontWeight: 600,
                      cursor: 'pointer',
                      height: '44px'
                    }}
                    onClick={() => handleAddFreeText(searchQuery)}
                  >
                    + Add to Assessment
                  </button>
                )}
              </div>

              {/* Suggestions Dropdown */}
              {dropdownOpen && searchQuery.trim() && (
                <div className="symptom-dropdown" role="listbox">
                  {filteredSymptoms.length > 0 ? (
                    filteredSymptoms.map((symptom) => (
                      <button
                        type="button"
                        key={symptom.id}
                        className="symptom-dropdown-item"
                        onClick={() => handleSelectFromSearch(symptom.id)}
                        role="option"
                        aria-selected="false"
                      >
                        <span>{symptom.name}</span>
                        <small style={{ color: 'var(--muted)' }}>+ Add</small>
                      </button>
                    ))
                  ) : (
                    /* No match found — allow adding as free-text symptom */
                    <button
                      type="button"
                      className="symptom-dropdown-item"
                      onClick={() => handleAddFreeText(searchQuery)}
```

```
                role="option"
                aria-selected="false"
                style={{ borderTop: 'none' }}
              >
                <span>
                  <span style={{ opacity: 0.5, marginRight: '6px' }}> </span>
                  Add <strong>&ldquo;{searchQuery.trim()}&rdquo;</strong> as symptom
                </span>
                <small style={{ color: 'var(--muted)' }}>Press Enter or click</small>
              </button>
            )}
          </div>
        )}
      </div>

      {/* Selected Symptoms Chips Bar */}
      {selectedSymptoms.length > 0 && (
        <div className="selected-tags-section">
          <div className="selected-tags-header">
            <span>
              Selected Symptoms (<strong>{selectedSymptoms.length}</strong>)
            </span>
            <button type="button" className="clear-all-link" onClick={clearAllSymptoms}>
              Clear all
            </button>
          </div>
          <div className="selected-chips-grid">
            {selectedSymptoms.map((id) => (
              <span className="selected-chip" key={id}>
                  {formatSymptomName(id)}
                <button
                  type="button"
                  className="chip-remove-btn"
                  onClick={() => removeSymptom(id)}
                  aria-label={`Remove ${formatSymptomName(id)}`}
                >
                  ×
                </button>
              </span>
            ))}
          </div>
        </div>
      )}

      {/* Quick-Pick Popular Symptoms */}
      <div className="quick-suggestions">
        <p className="quick-title">Common Symptoms (Quick Select):</p>
        <div className="quick-pills">
          {defaultPopularSymptoms.map((symptom) => {
            const isSelected = selectedSymptoms.includes(symptom.id);
            return (
              <button
                type="button"
                key={symptom.id}
                className={`quick-pill ${isSelected ? 'selected' : ''}`}
                onClick={() => toggleSymptom(symptom.id)}
              >
                {isSelected ? '  ' : '+ '}
                {symptom.name}
              </button>
            );
          })}
        </div>
      </div>
    </fieldset>

    {/* DURATION */}
    <label>
      How long have you felt these symptoms?
      <select
        value={form.duration}
        onChange={(e) => setForm({ ...form, duration: e.target.value })}
      >
        <option>Less than 24 hours</option>
        <option>1–3 days</option>
        <option>4–7 days</option>
        <option>More than a week</option>
      </select>
    </label>

    {/* PERCEIVED SEVERITY */}
    <fieldset>
      <legend>How would you describe your overall discomfort?</legend>
      <div className="severity-options">
        {[
          { level: 'mild', label: 'Mild', sub: 'Noticeable but manageable' },
          { level: 'moderate', label: 'Moderate', sub: 'Impacting daily activities' },
          { level: 'severe', label: 'Severe', sub: 'Significant or distressing' }
        ].map(({ level, label, sub }) => (
          <label
            className={`severity ${form.severity === level ? 'selected' : ''}`}
            key={level}
          >
            <input
              type="radio"
              name="severity"
              value={level}
              checked={form.severity === level}
              onChange={(e) => setForm({ ...form, severity: e.target.value })}
            />
            <span>
              <strong>{label}</strong>
              <small>{sub}</small>
            </span>
          </label>
```

```
            ))}
          </div>
        </fieldset>

        {/* CLINICAL NOTES */}
        <label>
          Additional context or notes
          <span className="optional">Optional</span>
          <textarea
            value={form.notes}
            onChange={(e) => setForm({ ...form, notes: e.target.value })}
            placeholder="E.g., Started after working outside, worse in the morning, accompanied by chills..."
          />
        </label>

        {/* SUBMIT BUTTON */}
        <div style={{ display: 'flex', alignItems: 'center', gap: '16px', flexWrap: 'wrap' }}>
          <button
            className="primary submit-button"
            type="submit"
            disabled={selectedSymptoms.length === 0 && !searchQuery.trim()}
          >
            Analyze & Find Health Prediction <span>&rarr;</span>
          </button>
          {selectedSymptoms.length === 0 && !searchQuery.trim() && (
            <span style={{ fontSize: '13px', color: 'var(--muted)' }}>
              Type your symptom or health problem above to analyze
            </span>
          )}
          {selectedSymptoms.length === 0 && searchQuery.trim() && (
            <span style={{ fontSize: '13px', color: '#1d543e', fontWeight: 600 }}>
                Ready to analyze: &ldquo;{searchQuery.trim()}&rdquo;
            </span>
          )}
        </div>
      </form>
    </section>
  </main>
 );
}
```

frontend/src/pages/Appointments.jsx

Doctor list, appointment form, and booking details/status.

```
import { useEffect, useMemo, useState } from 'react';
import { appointmentService } from '../services/api.js';

const defaultReason = 'I would like to follow up on my recent symptom check-in';

const formatDisplayDate = (value) => {
  if (!value) return 'Not scheduled';
  const date = new Date(value);
  if (Number.isNaN(date.getTime())) return value;
  return new Intl.DateTimeFormat('en-US', { month: 'short', day: 'numeric', year: 'numeric' }).format(date);
};

const formatTimeLabel = (value) => (value ? value.replace(':', ':') : 'TBD');

export default function Appointments({ onBack, initialReason }) {
  const [activeTab, setActiveTab] = useState('booking');
  const [doctors, setDoctors] = useState([]);
  const [appointments, setAppointments] = useState([]);
  const [selectedDoctorId, setSelectedDoctorId] = useState('');
  const [date, setDate] = useState('');
  const [timeSlot, setTimeSlot] = useState('');
  const [reason, setReason] = useState(initialReason || defaultReason);
  const [error, setError] = useState('');
  const [message, setMessage] = useState('');
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    if (initialReason) {
      setReason(initialReason);
    }
  }, [initialReason]);

  const selectedDoctor = useMemo(
    () => doctors.find((doctor) => doctor._id === selectedDoctorId) || null,
    [doctors, selectedDoctorId]
  );

  const fetchAllData = async () => {
    try {
      setLoading(true);
      const [doctorResponse, appointmentResponse] = await Promise.all([
        appointmentService.getDoctors(),
        appointmentService.getAppointments()
      ]);

      const nextDoctors = doctorResponse?.data?.doctors || [];
      const nextAppointments = appointmentResponse?.data?.appointments || [];

      setDoctors(nextDoctors);
      setAppointments(nextAppointments);
      if (nextDoctors[0] && !selectedDoctorId) {
        setSelectedDoctorId(nextDoctors[0]._id);
        setTimeSlot(nextDoctors[0].availabilitySlots?.[0] || '09:00');
      }
    } catch (loadError) {
      setError(loadError?.message || 'Unable to load doctors right now.');
    } finally {
```

```
    setLoading(false);
  }
};

useEffect(() => {
  fetchAllData();
}, []);

useEffect(() => {
  if (!selectedDoctor) return;
  const nextTime = selectedDoctor.availabilitySlots?.[0] || '09:00';
  if (!timeSlot || !selectedDoctor.availabilitySlots?.includes(timeSlot)) {
    setTimeSlot(nextTime);
  }
}, [selectedDoctor, timeSlot]);

const handleSubmit = async (event) => {
  event.preventDefault();
  setError('');
  setMessage('');

  if (!selectedDoctorId || !date || !timeSlot) {
    setError('Please choose a doctor, date, and appointment time.');
    return;
  }

  if (selectedDoctor && selectedDoctor.available === false) {
    setError(`Dr. ${selectedDoctor.name} is currently marked as unavailable in the database. Please select an available doctor.`);
    return;
  }

  try {
    setLoading(true);
    const response = await appointmentService.bookAppointment({
      doctorId: selectedDoctorId,
      date,
      timeSlot,
      reason
    });

    const newAppointment = response?.data?.appointment;
    setAppointments((prev) =>
      [...prev, newAppointment].sort((a, b) => new Date(a.appointmentDate) - new Date(b.appointmentDate))
    );
    setMessage('Appointment request sent successfully.');
    setReason(defaultReason);
    setDate('');
    if (selectedDoctor) {
      setTimeSlot(selectedDoctor.availabilitySlots?.[0] || '09:00');
    }
  } catch (submitError) {
    setError(submitError?.message || 'Unable to book this appointment.');
  } finally {
    setLoading(false);
  }
};

return (
  <main className="app-shell">
    <header className="topbar">
      <div className="brand"><span className="brand-mark">+</span> carepath</div>
      <nav>
        <button type="button" onClick={onBack}>Overview</button>
        <button className="nav-active" type="button">Consultations</button>
      </nav>
    </header>

    <section className="appointments-page">
      <div className="appointments-header">
        <div>
          <p className="eyebrow">CARE COORDINATION & DOCTOR MANAGEMENT</p>
          <h1>Doctor Consultations</h1>
        </div>
        <button className="primary" onClick={onBack} type="button">Back to dashboard</button>
      </div>

      {/* Consultation tab */}
      <div
        style={{
          display: 'flex',
          gap: '12px',
          marginBottom: '28px',
          borderBottom: '1px solid var(--line)',
          paddingBottom: '12px',
          flexWrap: 'wrap'
        }}
      >
        <button
          type="button"
          className={activeTab === 'booking' ? 'primary' : 'secondary-action-btn'}
          style={{
            padding: '10px 20px',
            borderRadius: '8px',
            fontWeight: 700,
            fontSize: '14px',
            cursor: 'pointer',
            background: activeTab === 'booking' ? 'var(--green)' : '#ffffff',
            color: activeTab === 'booking' ? '#ffffff' : 'var(--ink)',
            border: activeTab === 'booking' ? 'none' : '1px solid var(--line)'
          }}
          onClick={() => setActiveTab('booking')}
        >
            Book Consultation
        </button>
      </div>
```

```
{message && (
  <div
    style={{
      padding: '14px 18px',
      background: '#e8f5ec',
      border: '1px solid #c2decb',
      borderRadius: '8px',
      color: '#1d543e',
      fontSize: '14px',
      fontWeight: 600,
      marginBottom: '20px',
      display: 'flex',
      justifyContent: 'space-between',
      alignItems: 'center'
    }}
  >
    <span>  {message}</span>
    <button
      type="button"
      onClick={() => setMessage('')}
      style={{ background: 'none', border: 'none', fontSize: '16px', color: '#1d543e', cursor: 'pointer' }}
    >
      ×
    </button>
  </div>
)}

{error && (
  <div
    style={{
      padding: '14px 18px',
      background: '#fdf2f0',
      border: '1px solid #f6cfc9',
      borderRadius: '8px',
      color: '#a83c30',
      fontSize: '14px',
      fontWeight: 600,
      marginBottom: '20px'
    }}
  >
       {error}
  </div>
)}

{/* TAB 2: BOOKING CONSULTATION */}
{activeTab === 'booking' && (
  <div className="appointments-grid">
    <form className="booking-form" onSubmit={handleSubmit}>
      <h2>Choose a specialist</h2>

      <label>
        Doctor
        <select
          value={selectedDoctorId}
          onChange={(event) => setSelectedDoctorId(event.target.value)}
        >
          {doctors.map((doctor) => {
            const isAvail = doctor.available !== false;
            return (
              <option key={doctor._id} value={doctor._id}>
                {doctor.name} — {doctor.specialty} {isAvail ? '  (Available)' : '  [UNAVAILABLE]'}
              </option>
            );
          })}
        </select>
      </label>

      {/* Selected Doctor Preview Card */}
      {selectedDoctor && (
        <div
          className="doctor-card"
          style={{
            borderColor: selectedDoctor.available === false ? '#f6cfc9' : 'var(--line)',
            background: selectedDoctor.available === false ? '#fff5f5' : '#f3f8f4'
          }}
        >
          <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
            <strong>{selectedDoctor.name}</strong>
            <span
              style={{
                padding: '3px 8px',
                borderRadius: '12px',
                fontSize: '11px',
                fontWeight: 700,
                background: selectedDoctor.available === false ? '#fee2e2' : '#e2f1e5',
                color: selectedDoctor.available === false ? '#b91c1c' : '#276943'
              }}
            >
              {selectedDoctor.available === false ? '  Marked Unavailable' : '  Available'}
            </span>
          </div>
          <span>{selectedDoctor.specialty}</span>
          <p>{selectedDoctor.description}</p>
          <small>
            {selectedDoctor.consultationFee
              ? `$${selectedDoctor.consultationFee} consultation fee`
              : 'Consultation available'}
          </small>

          {/* Unavailable warning & quick toggle */}
          {selectedDoctor.available === false && (
            <div
              style={{
                marginTop: '10px',
```

```
                      padding: '10px 12px',
                      background: '#fff',
                      border: '1px solid #f6cfc9',
                      borderRadius: '6px',
                      display: 'flex',
                      justifyContent: 'space-between',
                      alignItems: 'center',
                      flexWrap: 'wrap',
                      gap: '8px'
                    }}
                  >
                    <span style={{ fontSize: '12px', color: '#b91c1c', fontWeight: 600 }}>
                         This doctor is currently marked as unavailable in the database.
                    </span>
                  </div>
                )}
              </div>
            )}

            <div className="booking-row">
              <label>
                Preferred date
                <input
                  type="date"
                  value={date}
                  onChange={(event) => setDate(event.target.value)}
                  min={new Date().toISOString().split('T')[0]}
                />
              </label>
              <label>
                Time
                <select value={timeSlot} onChange={(event) => setTimeSlot(event.target.value)}>
                  {(selectedDoctor?.availabilitySlots || []).map((slot) => (
                    <option key={slot} value={slot}>{formatTimeLabel(slot)}</option>
                  ))}
                </select>
              </label>
            </div>

            <label>
              Reason for visit
              <textarea value={reason} onChange={(event) => setReason(event.target.value)} rows="4" />
            </label>

            <button
              className="primary large"
              type="submit"
              disabled={loading || selectedDoctor?.available === false}
              style={{
                opacity: selectedDoctor?.available === false ? 0.6 : 1,
                cursor: selectedDoctor?.available === false ? 'not-allowed' : 'pointer'
              }}
            >
              {loading
                ? 'Booking…'
                : selectedDoctor?.available === false
                ? 'Doctor Unavailable in Database'
                : 'Book appointment'}
              </button>
          </form>

          <aside className="appointments-list">
            <h2>Upcoming requests</h2>
            {appointments.length === 0 ? (
              <div className="empty-appointments">
                <p>No visits scheduled yet.</p>
              </div>
            ) : (
              appointments.map((item) => (
                <article className="appointment-card" key={item._id || `${item.appointmentDate}-${item.doctorName}`}>
                  <div className="appointment-meta">
                    <span className="status green">{item.status}</span>
                    <strong>{item.doctorName}</strong>
                  </div>
                  <p>{item.specialty}</p>
                  <small>{formatDisplayDate(item.appointmentDate)} • {formatTimeLabel(item.timeSlot)}</small>
                  <span className="appointment-reason">{item.reason}</span>
                </article>
              ))
            )}
          </aside>
        </div>
      )}
    </section>
  </main>
 );
}
```

frontend/src/pages/Chatbot.jsx

Chat interface, history, requests, suggestions, and triage result display.

```
import { Fragment, useState, useRef, useEffect, useCallback } from 'react';
import { chatService } from '../services/api';
import { CHAT_MAX_MESSAGE_LENGTH, MEDICAL_DISCLAIMER } from '../utils/constants.js';
import './Chatbot.css';

const WELCOME_MSG = {
  id: 'welcome',
  role: 'bot',
  text: "  Hi! I'm **Carepath AI**, your personal health assistant.\n\nDescribe your symptoms in plain language — for example, *\"I've had a fever and
    sore throat for 2 days\"* — and I'll help assess your condition and suggest next steps.\n\nHow are you feeling today?",
```

```
  time: new Date(),
  type: 'welcome',
};

const SEVERITY_CONFIG = {
  mild: { label: 'Mild', color: '#22c55e', bg: 'rgba(34,197,94,0.12)', icon: ' ' },
  moderate: { label: 'Moderate', color: '#f59e0b', bg: 'rgba(245,158,11,0.12)', icon: ' ' },
  severe: { label: 'Severe', color: '#ef4444', bg: 'rgba(239,68,68,0.12)', icon: ' ' },
};

let msgCounter = 0;
const uid = () => `msg-${++msgCounter}-${Date.now()}`;

/* ———————————————— markdown renderer ———————————————— */
function renderMarkdown(text) {
  if (!text) return null;
  return text.split('\n').map((line, lineIndex) => (
    <Fragment key={lineIndex}>
      {line.split(/(\*\*[^*]+\*\*|\*[^*]+\*)/g).map((part, partIndex) => {
        if (part.startsWith('**') && part.endsWith('**')) {
          return <strong key={partIndex}>{part.slice(2, -2)}</strong>;
        }
        if (part.startsWith('*') && part.endsWith('*')) {
          return <em key={partIndex}>{part.slice(1, -1)}</em>;
        }
        return part;
      })}
      {lineIndex < text.split('\n').length - 1 && <br />}
    </Fragment>
  ));
}

/* ———————————————— DiagnosisCard ———————————————— */
function DiagnosisCard({ result, onBookAppointment }) {
  if (!result) return null;
  const sev = SEVERITY_CONFIG[result.severity] || SEVERITY_CONFIG.moderate;
  return (
    <div className="chat-diag-card">
      <div className="chat-diag-header">
        <span className="chat-diag-icon"> </span>
        <div>
          <div className="chat-diag-title">AI Triage Result</div>
          <div className="chat-diag-disease">{result.predictedDisease}</div>
        </div>
        <div className="chat-diag-confidence">
          <svg viewBox="0 0 36 36" className="conf-ring">
            <circle cx="18" cy="18" r="15.9" fill="none" stroke="#1e293b" strokeWidth="3" />
            <circle
              cx="18" cy="18" r="15.9" fill="none"
              stroke={sev.color} strokeWidth="3"
              strokeDasharray={`${result.confidence} ${100 - result.confidence}`}
              strokeLinecap="round"
              transform="rotate(-90 18 18)"
            />
          </svg>
          <span style={{ color: sev.color }}>{result.confidence}%</span>
        </div>
      </div>

      <div className="chat-diag-severity" style={{ background: sev.bg, borderColor: sev.color }}>
        {sev.icon} Severity: <strong style={{ color: sev.color }}>{sev.label}</strong>
      </div>

      {result.topPredictions?.length > 1 && (
        <div className="chat-diag-section">
          <div className="chat-diag-section-label">Differential Diagnoses</div>
          {result.topPredictions.slice(0, 3).map((p, i) => (
            <div key={i} className="chat-diag-bar-row">
              <span className="chat-diag-bar-label">{p.condition || p.disease}</span>
              <div className="chat-diag-bar-track">
                <div
                  className="chat-diag-bar-fill"
                  style={{
                    width: `${p.probability ?? p.confidence ?? 0}%`,
                    background: i === 0 ? sev.color : '#475569'
                  }}
                />
              </div>
              <span className="chat-diag-bar-pct">{p.probability ?? p.confidence ?? 0}%</span>
            </div>
          ))}
        </div>
      )}

      {result.geminiInsights && (
        <div style={{ marginTop: '10px', background: 'rgba(37, 99, 235, 0.08)', border: '1px solid rgba(37, 99, 235, 0.2)', borderRadius: '8px',
    padding: '10px 12px' }}>
          <div style={{ display: 'flex', alignItems: 'center', gap: '6px', marginBottom: '4px' }}>
            <span style={{ fontSize: '13px' }}> </span>
            <strong style={{ fontSize: '11px', color: '#60a5fa', textTransform: 'uppercase', letterSpacing: '0.04em' }}>
              {result.geminiInsights?.poweredBy || 'Gemini'} Clinical Insight
            </strong>
          </div>
          <p style={{ margin: 0, fontSize: '12px', color: '#e2e8f0', lineHeight: '1.5' }}>
            {result.geminiInsights.clinicalExplanation}
          </p>
          {result.accuracyMetrics && (
            <div style={{ marginTop: '8px', display: 'flex', gap: '10px', fontSize: '10px', color: '#94a3b8' }}>
              <span>Agreement: <strong style={{ color: '#60a5fa' }}>{result.accuracyMetrics.agreementScore}%</strong></span>
              <span>Dataset Coverage: <strong style={{ color: '#34d399' }}>{result.accuracyMetrics.symptomCoverageRatio}%</strong></span>
            </div>
          )}
        </div>
      )}
```

```
      {result.recommendation && (
        <div className="chat-diag-rec">
          <strong>Care Advice:</strong> {result.recommendation}
        </div>
      )}

      {result.isFallback && (
        <div className="chat-diag-fallback">
             ML service is offline — result generated by fallback heuristic engine.
        </div>
      )}

      <div className="chat-diag-disclaimer">{MEDICAL_DISCLAIMER}</div>

      {onBookAppointment && (
        <button
          className="chat-book-btn"
          onClick={() => onBookAppointment(result)}
          id="chat-book-appointment-btn"
        >
            Book Specialist Appointment
        </button>
      )}
    </div>
  );
}

/* ———————————————— SymptomPills ———————————————— */
function SymptomPills({ symptoms, onAdd }) {
  if (!symptoms?.length) return null;
  return (
    <div className="chat-symptom-pills">
      <span className="chat-pills-label">Also experiencing?</span>
      {symptoms.map((s) => (
        <button
          key={s.id}
          className="chat-pill"
          onClick={() => onAdd(s)}
          id={`suggest-${s.id}`}
        >
          + {s.name}
        </button>
      ))}
    </div>
  );
}

/* ———————————————— Message Bubble ———————————————— */
function Message({ msg, onBookAppointment, onAddSymptom }) {
  const isBot = msg.role === 'bot';
  return (
    <div className={`chat-bubble-row ${isBot ? 'bot-row' : 'user-row'}`}>
      {isBot && (
        <div className="chat-avatar bot-avatar" aria-label="AI assistant">
          <span> </span>
        </div>
      )}
      <div className={`chat-bubble ${isBot ? 'bot-bubble' : 'user-bubble'}`}>
        {msg.isTyping ? (
          <div className="typing-indicator">
            <span /><span /><span />
          </div>
        ) : (
          <>
            <div
              className="bubble-text"
            >
              {renderMarkdown(msg.text)}
            </div>
            {msg.isEmergency && (
              <div className="chat-emergency-banner">
                  <strong>EMERGENCY WARNING:</strong> Your symptoms may require immediate medical attention. Call emergency services (112/108) now.
              </div>
            )}
            {msg.diagnosisResult && (
              <DiagnosisCard
                result={msg.diagnosisResult}
                onBookAppointment={onBookAppointment}
              />
            )}
            {msg.suggestedSymptoms?.length > 0 && (
              <SymptomPills symptoms={msg.suggestedSymptoms} onAdd={onAddSymptom} />
            )}
            <div className="bubble-time">
              {msg.time?.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })}
            </div>
          </>
        )}
      </div>
      {!isBot && (
        <div className="chat-avatar user-avatar" aria-label="You">
          <span> </span>
        </div>
      )}
    </div>
  );
}

/* ———————————————— Main Chatbot ———————————————— */
export default function Chatbot({ onBack, onBookAppointment }) {
  const [messages, setMessages] = useState([WELCOME_MSG]);
  const [input, setInput] = useState('');
  const [loading, setLoading] = useState(false);
  const [currentSymptoms, setCurrentSymptoms] = useState([]);
  const [conversationHistory, setConversationHistory] = useState([]);
```

```
  const [sessionDone, setSessionDone] = useState(false);
  const bottomRef = useRef(null);
  const inputRef = useRef(null);

  useEffect(() => {
    bottomRef.current?.scrollIntoView({ behavior: 'smooth' });
  }, [messages]);

  const addMessage = useCallback((msg) => {
    setMessages((prev) => [...prev, { id: uid(), time: new Date(), ...msg }]);
  }, []);

  const handleSend = useCallback(async (text) => {
    const trimmed = (text || input).trim();
    if (!trimmed || loading) return;

    setInput('');

    addMessage({ role: 'user', text: trimmed });
    const typingId = uid();
    setMessages((prev) => [
      ...prev,
      { id: typingId, role: 'bot', isTyping: true, time: new Date() }
    ]);
    setLoading(true);

    const newHistory = [...conversationHistory, { role: 'user', content: trimmed }];

    try {
      const resp = await chatService.sendMessage({
        message: trimmed,
        conversationHistory: newHistory,
        currentSymptoms
      });

      const data = resp.data;
      const updatedSymptoms = data.detectedSymptoms?.length > 0
        ? [...new Set([...currentSymptoms, ...data.detectedSymptoms])]
        : currentSymptoms;

      setCurrentSymptoms(updatedSymptoms);
      setConversationHistory([
        ...newHistory.slice(-40),
        { role: 'assistant', content: data.reply }
      ]);

      if (data.diagnosisResult) setSessionDone(true);

      setMessages((prev) =>
        prev.map((m) =>
          m.id === typingId
            ? {
                ...m,
                isTyping: false,
                text: data.reply,
                diagnosisResult: data.diagnosisResult,
                suggestedSymptoms: data.suggestedSymptoms,
                isEmergency: data.isEmergency
              }
            : m
        )
      );
    } catch (err) {
      setMessages((prev) =>
        prev.map((m) =>
          m.id === typingId
            ? {
                ...m,
                isTyping: false,
                text: `  ${err.message || 'Something went wrong. Please try again.'}`
              }
            : m
        )
      );
    } finally {
      setLoading(false);
      inputRef.current?.focus();
    }
  }, [input, loading, conversationHistory, currentSymptoms, addMessage]);

  const handleEvaluate = useCallback(async () => {
    if (loading || currentSymptoms.length === 0) return;
    setLoading(true);
    const typingId = uid();
    setMessages((prev) => [
      ...prev,
      { id: typingId, role: 'bot', isTyping: true, time: new Date() }
    ]);

    try {
      const resp = await chatService.sendMessage({
        message: '',
        action: 'evaluate',
        conversationHistory,
        currentSymptoms
      });
      const data = resp.data;
      if (data.diagnosisResult) setSessionDone(true);
      setMessages((prev) =>
        prev.map((m) =>
          m.id === typingId
            ? {
                ...m,
                isTyping: false,
                text: data.reply,
```

```
              diagnosisResult: data.diagnosisResult,
              suggestedSymptoms: data.suggestedSymptoms,
              isEmergency: data.isEmergency
            }
          : m
      )
    );
  } catch (err) {
    setMessages((prev) =>
      prev.map((m) =>
        m.id === typingId
          ? { ...m, isTyping: false, text: `  ${err.message}` }
          : m
      )
    );
  } finally {
    setLoading(false);
  }
}, [loading, currentSymptoms, conversationHistory]);

const handleAddSymptom = useCallback((symptom) => {
  handleSend(`Yes, I also have ${symptom.name}`);
}, [handleSend]);

const handleReset = () => {
  setMessages([WELCOME_MSG]);
  setInput('');
  setCurrentSymptoms([]);
  setConversationHistory([]);
  setSessionDone(false);
  inputRef.current?.focus();
};

const quickPrompts = [
  'I have fever and headache',
  'Chest pain and shortness of breath',
  'Stomach ache and nausea',
  'Skin rash and itching',
];

return (
  <div className="chatbot-page">
    {/* Header */}
    <header className="chatbot-header">
      <button className="chatbot-back-btn" onClick={onBack} id="chatbot-back-btn" aria-label="Go back">
        <svg width="20" height="20" fill="none" stroke="currentColor" strokeWidth="2" viewBox="0 0 24 24">
          <path strokeLinecap="round" strokeLinejoin="round" d="M19 12H5m7-7-7 7 7 7" />
        </svg>
      </button>
      <div className="chatbot-header-info">
        <div className="chatbot-avatar-ring">
          <span className="chatbot-header-avatar"> </span>
          <span className="chatbot-status-dot" />
        </div>
        <div>
          <div className="chatbot-header-name">Carepath AI</div>
          <div className="chatbot-header-sub">Health Assessment Assistant</div>
        </div>
      </div>
      <div className="chatbot-header-actions">
        {currentSymptoms.length > 0 && !sessionDone && (
          <button
            className="chat-eval-btn"
            onClick={handleEvaluate}
            disabled={loading}
            id="chat-run-evaluation-btn"
            title="Run full diagnosis with collected symptoms"
          >
            <svg width="16" height="16" fill="none" stroke="currentColor" strokeWidth="2" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round" d="M9 12l2 2 4-4m6 2a9 9 0 11-18 0 9 9 0 0118 0z" />
            </svg>
            Evaluate ({currentSymptoms.length})
          </button>
        )}
        <button className="chat-reset-btn" onClick={handleReset} id="chat-reset-btn" title="Start new session">
          <svg width="16" height="16" fill="none" stroke="currentColor" strokeWidth="2" viewBox="0 0 24 24">
            <path strokeLinecap="round" strokeLinejoin="round" d="M4 4v5h.582m15.356 2A8.001 8.001 0 004.582 9m0 0H9m11 11v-5h-.581m0 0a8.003 8.003 0
01-15.357-2m15.357 2H15" />
          </svg>
        </button>
      </div>
    </header>

    {/* Symptoms tracker bar */}
    {currentSymptoms.length > 0 && (
      <div className="chatbot-symptom-bar">
        <span className="symptom-bar-label">Tracking:</span>
        {currentSymptoms.slice(0, 6).map((s) => (
          <span key={s} className="symptom-bar-tag">
            {s.replace(/_/g, ' ')}
          </span>
        ))}
        {currentSymptoms.length > 6 && (
          <span className="symptom-bar-more">+{currentSymptoms.length - 6} more</span>
        )}
      </div>
    )}

    {/* Messages area */}
    <div className="chatbot-messages" id="chatbot-messages-area" role="log" aria-live="polite">
      {messages.map((msg) => (
        <Message
          key={msg.id}
          msg={msg}
          onBookAppointment={onBookAppointment}
```

```
            onAddSymptom={handleAddSymptom}
          />
        ))}
        <div ref={bottomRef} />
      </div>

      {/* Quick prompt chips (only when fresh) */}
      {messages.length === 1 && !loading && (
        <div className="chatbot-quick-prompts">
          {quickPrompts.map((p) => (
            <button
              key={p}
              className="quick-prompt-chip"
              onClick={() => handleSend(p)}
              id={`quick-prompt-${p.replace(/\s+/g, '-').toLowerCase()}`}
            >
              {p}
            </button>
          ))}
        </div>
      )}

      {/* Input bar */}
      <div className="chatbot-input-area">
        <div className="chatbot-input-wrap">
          <textarea
            ref={inputRef}
            className="chatbot-input"
            id="chatbot-text-input"
            placeholder="Describe your symptoms…"
            value={input}
            rows={1}
            maxLength={CHAT_MAX_MESSAGE_LENGTH}
            onChange={(e) => setInput(e.target.value)}
            onKeyDown={(e) => {
              if (e.key === 'Enter' && !e.shiftKey) {
                e.preventDefault();
                handleSend();
              }
            }}
            disabled={loading}
            aria-label="Type your message"
          />
          <button
            className={`chatbot-send-btn ${loading ? 'loading' : ''}`}
            onClick={() => handleSend()}
            disabled={loading || !input.trim()}
            id="chatbot-send-btn"
            aria-label="Send message"
          >
            {loading ? (
              <div className="send-spinner" />
            ) : (
              <svg width="20" height="20" fill="none" stroke="currentColor" strokeWidth="2" viewBox="0 0 24 24">
                <path strokeLinecap="round" strokeLinejoin="round" d="M12 19l9 2-9-18-9 18 9-2zm0 0v-8" />
              </svg>
            )}
          </button>
        </div>
        <p className="chatbot-footer-note">{MEDICAL_DISCLAIMER}</p>
      </div>
    </div>
  );
}
```

frontend/src/pages/Login.jsx

Login view and reusable AuthShell.

```
import { useState } from 'react';
import { useAuth } from '../context/AuthContext.jsx';
import { authService } from '../services/api.js';

export default function Login({ onSignup, onSuccess }) {
  const { signIn } = useAuth();
  const [form, setForm] = useState({ email: '', password: '[REDACTED]' });
  const [error, setError] = useState('');
  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleSubmit = async (event) => {
    event.preventDefault();
    setError('');

    if (!form.email || !form.password) {
      setError('Please provide both email and password');
      return;
    }

    setIsSubmitting(true);
    try {
      const response = await authService.login(form);
      signIn(response);
      onSuccess();
    } catch (err) {
      setError(err.message || 'Could not log in. Please check your credentials.');
    } finally {
      setIsSubmitting(false);
    }
  };

  return (
    <AuthShell
```

```
      title="A calmer way to understand your health."
      subtitle="Private guidance for the moments when you need a little more clarity."
    >
      <h1>Welcome back</h1>
      <p className="muted">Sign in to continue your health check-ins.</p>

      <form onSubmit={handleSubmit} className="form-stack">
        <label>
          Email address
          <input
            type="email"
            required
            autoComplete="email"
            value={form.email}
            disabled={isSubmitting}
            onChange={(e) => setForm({ ...form, email: e.target.value })}
            placeholder="you@example.com"
          />
        </label>

        <label>
          Password
          <input
            type="password"
            required
            autoComplete="current-password"
            value={form.password}
            disabled={isSubmitting}
            onChange={(e) => setForm({ ...form, password: e.target.value })}
            placeholder="Your password"
          />
        </label>

        {error && (
          <div className="error-banner" role="alert" style={{
            padding: '10px 14px',
            backgroundColor: '#fee2e2',
            border: '1px solid #f87171',
            borderRadius: '6px',
            color: '#b91c1c',
            fontSize: '0.9rem',
            marginBottom: '1rem'
          }}>
            {error}
          </div>
        )}

        <button className="primary" type="submit" disabled={isSubmitting}>
          {isSubmitting ? 'Signing in...' : (
            <>
              Sign in <span>-&gt;</span>
            </>
          )}
        </button>
      </form>

      <p className="switch">
        New to Carepath?{' '}
        <button type="button" onClick={onSignup} disabled={isSubmitting}>
          Create an account
        </button>
      </p>
    </AuthShell>
  );
}

export function AuthShell({ title, subtitle, children }) {
  return (
    <main className="auth-page">
      <section className="auth-story">
        <div className="brand">
          <span className="brand-mark">+</span> carepath
        </div>
        <div className="story-copy">
          <p className="eyebrow">YOUR HEALTH, IN FOCUS</p>
          <h2>{title}</h2>
          <p>{subtitle}</p>
        </div>
        <div className="story-footer">
          Built around your wellbeing <span>2026</span>
        </div>
      </section>
      <section className="auth-panel">
        <div className="form-card">{children}</div>
      </section>
    </main>
  );
}
```

frontend/src/pages/Signup.jsx

Registration view and client-side input checks.

```
import { useState } from 'react';
import { useAuth } from '../context/AuthContext.jsx';
import { authService } from '../services/api.js';
import { AuthShell } from './Login.jsx';

export default function Signup({ onLogin, onSuccess }) {
  const { signIn } = useAuth();
  const [form, setForm] = useState({
    name: '',
```

```
  email: '',
  password: '[REDACTED]',
  age: '',
  gender: 'prefer_not_to_say'
});
const [error, setError] = useState('');
const [isSubmitting, setIsSubmitting] = useState(false);

const isPasswordValid = form.password.length >= 8 && /\d/.test(form.password);

const handleSubmit = async (event) => {
  event.preventDefault();
  setError('');

  if (!form.name.trim() || !form.email.trim() || !form.password) {
    setError('Please fill out all required fields');
    return;
  }

  if (!isPasswordValid) {
    setError('Password must be at least 8 characters long and contain at least one number');
    return;
  }

  setIsSubmitting(true);
  try {
    const payload = {
      name: form.name.trim(),
      email: form.email.trim(),
      password: form.password,
      age: form.age ? Number(form.age) : undefined,
      gender: form.gender
    };

    const response = await authService.register(payload);
    signIn(response);
    onSuccess();
  } catch (err) {
    setError(err.message || 'Could not create your account. Please try again.');
  } finally {
    setIsSubmitting(false);
  }
};

return (
  <AuthShell
    title="Make space for better health habits."
    subtitle="A simple, private place to check in with yourself and find your next step."
  >
    <h1>Create your account</h1>
    <p className="muted">It takes less than a minute.</p>

    <form onSubmit={handleSubmit} className="form-stack">
      <label>
        Full name *
        <input
          required
          autoComplete="name"
          value={form.name}
          disabled={isSubmitting}
          onChange={(e) => setForm({ ...form, name: e.target.value })}
          placeholder="Jordan Lee"
        />
      </label>

      <label>
        Email address *
        <input
          type="email"
          required
          autoComplete="email"
          value={form.email}
          disabled={isSubmitting}
          onChange={(e) => setForm({ ...form, email: e.target.value })}
          placeholder="you@example.com"
        />
      </label>

      <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '0.75rem' }}>
        <label>
          Age (optional)
          <input
            type="number"
            min="0"
            max="120"
            value={form.age}
            disabled={isSubmitting}
            onChange={(e) => setForm({ ...form, age: e.target.value })}
            placeholder="e.g. 28"
          />
        </label>

        <label>
          Gender (optional)
          <select
            value={form.gender}
            disabled={isSubmitting}
            onChange={(e) => setForm({ ...form, gender: e.target.value })}
            style={{
              width: '100%',
              padding: '0.75rem',
              borderRadius: '6px',
              border: '1px solid #d1d5db',
              backgroundColor: '#fff',
              fontSize: '0.95rem'
```

```
            }}
          >
            <option value="prefer_not_to_say">Prefer not to say</option>
            <option value="female">Female</option>
            <option value="male">Male</option>
            <option value="other">Other</option>
          </select>
        </label>
      </div>

      <label>
        Password *
        <input
          type="password"
          required
          autoComplete="new-password"
          value={form.password}
          disabled={isSubmitting}
          onChange={(e) => setForm({ ...form, password: e.target.value })}
          placeholder="Min 8 chars, at least 1 number"
        />
      </label>

      {form.password && !isPasswordValid && (
        <p style={{ fontSize: '0.8rem', color: '#e11d48', marginTop: '-0.25rem' }}>
          Requires at least 8 characters and 1 number.
        </p>
      )}

      {error && (
        <div
          role="alert"
          style={{
            padding: '10px 14px',
            backgroundColor: '#fee2e2',
            border: '1px solid #f87171',
            borderRadius: '6px',
            color: '#b91c1c',
            fontSize: '0.9rem',
            marginBottom: '1rem'
          }}
        >
          {error}
        </div>
      )}

      <button className="primary" type="submit" disabled={isSubmitting}>
        {isSubmitting ? 'Creating account...' : (
          <>
            Get started <span>-&gt;</span>
          </>
        )}
      </button>
    </form>

    <p className="switch">
      Already have an account?{' '}
      <button type="button" onClick={onLogin} disabled={isSubmitting}>
        Sign in
      </button>
    </p>
  </AuthShell>
 );
}
```

frontend/src/routes/AppRouter.jsx

Actual page route composition and protected wrappers.

```
import { Navigate, Route, Routes, useLocation, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext.jsx';
import Login from '../pages/Login.jsx';
import Signup from '../pages/Signup.jsx';
import Dashboard from '../pages/Dashboard.jsx';
import DiagnosisForm from '../pages/DiagnosisForm.jsx';
import Profile from '../pages/Profile.jsx';
import Appointments from '../pages/Appointments.jsx';
import Chatbot from '../pages/Chatbot.jsx';
import ProtectedRoute from './ProtectedRoute.jsx';

function RootRedirect() {
  const { isAuthenticated } = useAuth();
  return <Navigate to={isAuthenticated ? '/dashboard' : '/login'} replace />;
}

function AuthScreen({ mode = 'login' }) {
  const navigate = useNavigate();
  const { isAuthenticated } = useAuth();

  if (isAuthenticated) {
    return <Navigate to="/dashboard" replace />;
  }

  if (mode === 'signup') {
    return (
      <Signup
        onLogin={() => navigate('/login')}
        onSuccess={() => navigate('/dashboard')}
      />
    );
  }
```

```
  return (
    <Login
      onSignup={() => navigate('/signup')}
      onSuccess={() => navigate('/dashboard')}
    />
  );
}

function ProtectedDashboard() {
  const navigate = useNavigate();

  return (
    <Dashboard
      onNewDiagnosis={() => navigate('/diagnosis')}
      onChatbot={() => navigate('/chatbot')}
      onProfile={() => navigate('/profile')}
      onAppointments={() => navigate('/appointments')}
    />
  );
}

function ProtectedDiagnosis() {
  const navigate = useNavigate();

  return (
    <DiagnosisForm
      onBack={() => navigate('/dashboard')}
      onBookAppointment={({ reason }) => {
        const appointmentReason = reason || 'Follow-up consultation';
        navigate('/appointments', { state: { reason: appointmentReason } });
      }}
    />
  );
}

function ProtectedProfile() {
  const navigate = useNavigate();

  return <Profile onBack={() => navigate('/dashboard')} />;
}

function ProtectedChatbot() {
  const navigate = useNavigate();

  return (
    <Chatbot
      onBack={() => navigate('/dashboard')}
      onBookAppointment={(result) => {
        const reason = `Follow-up for AI health triage: ${result.predictedDisease} (${result.confidence}% confidence)`;
        navigate('/appointments', { state: { reason } });
      }}
    />
  );
}

function ProtectedAppointments() {
  const navigate = useNavigate();
  const location = useLocation();
  const initialReason = location.state?.reason || '';

  return (
    <Appointments
      onBack={() => navigate('/dashboard')}
      initialReason={initialReason}
    />
  );
}

export default function AppRouter() {
  return (
    <Routes>
      <Route path="/" element={<RootRedirect />} />
      <Route path="/login" element={<AuthScreen mode="login" />} />
      <Route path="/signup" element={<AuthScreen mode="signup" />} />

      <Route
        path="/dashboard"
        element={
          <ProtectedRoute>
            <ProtectedDashboard />
          </ProtectedRoute>
        }
      />
      <Route
        path="/diagnosis"
        element={
          <ProtectedRoute>
            <ProtectedDiagnosis />
          </ProtectedRoute>
        }
      />
      <Route
        path="/profile"
        element={
          <ProtectedRoute>
            <ProtectedProfile />
          </ProtectedRoute>
        }
      />
      <Route
        path="/chatbot"
        element={
          <ProtectedRoute>
            <ProtectedChatbot />
          </ProtectedRoute>
```

```
        }
      />
      <Route
        path="/appointments"
        element={
          <ProtectedRoute>
            <ProtectedAppointments />
          </ProtectedRoute>
        }
      />
      <Route path="*" element={<RootRedirect />} />
    </Routes>
  );
}
```

9. Back End

The following appendix-style listings are copied directly from the checked-in source files at report-generation time. They are grouped by path with a brief role statement. Secret-bearing environment files, package lock files, binary models, private control files, build output, and dependency folders are excluded. The two control `.example.json` files are represented only by one harmless example each.

API entry point and utility

backend/server.js

Express application setup, routes, security middleware, database startup, and shutdown.

```
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import connectDB from './config/db.js';
import authRoutes from './routes/authRoutes.js';
import userRoutes from './routes/userRoutes.js';
import diagnosisRoutes from './routes/diagnosisRoutes.js';
import doctorRoutes from './routes/doctorRoutes.js';
import chatRoutes from './routes/chatRoutes.js';
import { getSymptoms } from './controllers/diagnosisController.js';
import { syncBookingControl } from './controllers/doctorController.js';
import { validatePredictionEnvironment } from './services/predictionService.js';
import { errorHandler } from './middleware/errorHandler.js';
import { sendSuccess, sendError } from './utils/apiResponse.js';

const app = express();
const basePort = Number(process.env.PORT) || 5000;

// Security & Parsing Middleware (helmet sets safe HTTP security headers per CODING_STANDARDS §5)
app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_URL || 'http://localhost:5173' }));
app.use(express.json());

// Health Check Probe
app.get('/api/health', (_req, res) => {
  return sendSuccess(res, 200, 'Health diagnosis API service is healthy', {
    status: 'ok',
    service: 'health-diagnosis-api',
    timestamp: new Date().toISOString()
  });
});

// API Routes
app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);
app.use('/api/diagnoses', diagnosisRoutes);
app.use('/api/diagnosis', diagnosisRoutes);
app.get('/api/symptoms', getSymptoms);
app.use('/api/chat', chatRoutes);
app.use('/api', doctorRoutes);

// 404 Route Handler
app.use((req, res) => {
  return sendError(res, 404, `Route ${req.method} ${req.originalUrl} not found`, {
    code: 'ROUTE_NOT_FOUND'
  });
});

// Centralized Error Handler
app.use(errorHandler);

let server;
const start = async () => {
  // Environment integrity (ARCHITECTURE §6.4): fail fast on missing required secrets
  if (!process.env.JWT_SECRET) {
    console.error('[API Gateway] JWT_SECRET is missing. Create backend/.env from backend/.env.example.');
    process.exit(1);
  }

  // Validate prediction provider configuration up front (non-fatal warnings —
  // requests degrade gracefully to the ML/heuristic engine when Gemini is unconfigured)
  const { provider, warnings } = validatePredictionEnvironment();
  console.log(`[API Gateway] Prediction provider: ${provider}`);
  warnings.forEach((warning) => console.warn(`[API Gateway] ${warning}`));

  await connectDB();
  await syncBookingControl();
```

```
  const listenOnPort = (port, attempt = 0) => {
    server = app.listen(port, () => {
      console.log(`[API Gateway] Listening on port ${port}`);
    });

    server.on('error', (error) => {
      if (error.code === 'EADDRINUSE' && attempt < 9) {
        console.warn(`[API Gateway] Port ${port} is busy. Retrying on ${port + 1}...`);
        listenOnPort(port + 1, attempt + 1);
        return;
      }

      console.error('[API Gateway] Failed to start server:', error.message);
      process.exit(1);
    });
  };

  listenOnPort(basePort);
};

// Graceful Shutdown
const handleShutdown = async (signal) => {
  console.log(`Received ${signal}. Shutting down gracefully...`);
  if (server) {
    server.close(() => {
      console.log('HTTP server closed.');
      process.exit(0);
    });
  } else {
    process.exit(0);
  }
};

process.on('SIGINT', () => handleShutdown('SIGINT'));
process.on('SIGTERM', () => handleShutdown('SIGTERM'));

start();
```

backend/utils/apiResponse.js

Standard success and error response envelopes.

```
/**
 * Standardized Success Response Helper
 */
export const sendSuccess = (res, statusCode = 200, message = 'Success', data = null) => {
  return res.status(statusCode).json({
    success: true,
    message,
    data,
    error: null
  });
};

/**
 * Standardized Error Response Helper
 */
export const sendError = (res, statusCode = 400, message = 'Error', error = null) => {
  return res.status(statusCode).json({
    success: false,
    message,
    data: null,
    error
  });
};
```

Controllers

backend/controllers/authController.js

Registration, login, and credential/session orchestration.

```
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import User from '../models/User.js';
import { JWT_EXPIRES_IN, BCRYPT_SALT_ROUNDS } from '../config/constants.js';
import { sendSuccess, sendError } from '../utils/apiResponse.js';

const createToken = (id) => {
  const jwtSecret = process.env.JWT_SECRET;
  if (!jwtSecret) {
    // Fail fast instead of silently signing with an insecure default.
    throw new Error('JWT_SECRET is not configured. Set it in backend/.env.');
  }
  return jwt.sign({ id }, jwtSecret, {
    expiresIn: JWT_EXPIRES_IN
  });
};

const formatPublicUser = (user) => ({
  id: user._id,
  name: user.name,
  email: user.email,
  age: user.age,
  gender: user.gender,
  dateOfBirth: user.dateOfBirth,
  bloodType: user.bloodType,
  allergies: user.allergies,
```

```
  createdAt: user.createdAt
});

/**
 * Register a new user
 * POST /api/auth/register (or /api/auth/signup)
 */
export const register = async (req, res, next) => {
  try {
    const { name, email, password, age, gender } = req.body;

    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return sendError(res, 409, 'An account with this email address already exists', {
        code: 'AUTH_EMAIL_ALREADY_EXISTS'
      });
    }

    const hashedPassword = await bcrypt.hash(password, BCRYPT_SALT_ROUNDS);

    const user = await User.create({
      name,
      email,
      password: hashedPassword,
      age: age ? Number(age) : undefined,
      gender: gender || undefined
    });

    const token = createToken(user._id);

    return sendSuccess(res, 201, 'User registered successfully', {
      token,
      user: formatPublicUser(user)
    });
  } catch (error) {
    next(error);
  }
};

/**
 * Legacy alias for register
 */
export const signup = register;

/**
 * Authenticate existing user
 * POST /api/auth/login
 */
export const login = async (req, res, next) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email }).select('+password');
    if (!user) {
      return sendError(res, 401, 'Invalid email or password', {
        code: 'AUTH_INVALID_CREDENTIALS'
      });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return sendError(res, 401, 'Invalid email or password', {
        code: 'AUTH_INVALID_CREDENTIALS'
      });
    }

    const token = createToken(user._id);

    return sendSuccess(res, 200, 'Login successful', {
      token,
      user: formatPublicUser(user)
    });
  } catch (error) {
    next(error);
  }
};
```

backend/controllers/userController.js

Profile read and update operations.

```
import { sendSuccess, sendError } from '../utils/apiResponse.js';

const formatPublicUser = (user) => ({
  id: user._id,
  name: user.name,
  email: user.email,
  age: user.age,
  gender: user.gender,
  dateOfBirth: user.dateOfBirth,
  bloodType: user.bloodType,
  allergies: user.allergies,
  createdAt: user.createdAt,
  updatedAt: user.updatedAt
});

/**
 * Get authenticated user profile
 * GET /api/users/me (or /api/users/profile)
 */
export const getProfile = async (req, res) => {
  return sendSuccess(res, 200, 'User profile fetched successfully', {
```

```
    user: formatPublicUser(req.user)
  });
};

/**
 * Update authenticated user profile
 * PUT /api/users/me (or /api/users/profile)
 */
export const updateProfile = async (req, res, next) => {
  try {
    const { name, age, gender, dateOfBirth, bloodType, allergies } = req.body;

    if (name) req.user.name = name;
    if (age !== undefined) req.user.age = age ? Number(age) : undefined;
    if (gender !== undefined) req.user.gender = gender;
    if (dateOfBirth !== undefined) req.user.dateOfBirth = dateOfBirth;
    if (bloodType !== undefined) req.user.bloodType = bloodType;
    if (allergies !== undefined) req.user.allergies = allergies;

    const updatedUser = await req.user.save();

    return sendSuccess(res, 200, 'User profile updated successfully', {
      user: formatPublicUser(updatedUser)
    });
  } catch (error) {
    if (error.name === 'ValidationError') {
      return sendError(res, 400, error.message, { details: error.errors });
    }
    next(error);
  }
};
```

backend/controllers/diagnosisController.js

Diagnosis prediction orchestration, persistence, history, detail, deletion, and symptom catalog.

```
import mongoose from 'mongoose';
import Diagnosis from '../models/Diagnosis.js';
import { getSymptomsList, MANDATORY_DISCLAIMER } from '../services/mlService.js';
import { runPrediction } from '../services/predictionService.js';
import { sendSuccess, sendError } from '../utils/apiResponse.js';
import {
  CARE_RECOMMENDATIONS,
  DEFAULT_SYMPTOM_DURATION,
  LOW_CONFIDENCE_MESSAGE,
  SYMPTOMS_NOT_RECOGNIZED_CODE,
  CONFIDENCE_BASIS
} from '../config/constants.js';

/**
 * GET /api/diagnoses/symptoms
 * Retrieves master list of selectable symptoms
 */
export const getSymptoms = async (_req, res, next) => {
  try {
    const symptoms = await getSymptomsList();
    return sendSuccess(res, 200, 'Symptoms retrieved successfully', { symptoms });
  } catch (error) {
    return next(error);
  }
};

/**
 * POST /api/diagnoses/predict
 * Evaluates patient symptoms with the configured prediction provider
 * (Google Gemini by default, ML microservice/heuristic fallback), persists the
 * record into MongoDB Atlas, and returns it inside the standard API envelope.
 */
export const createDiagnosis = async (req, res, next) => {
  try {
    const { symptoms, duration, severity: clientSeverity, notes } = req.body;

    if (!Array.isArray(symptoms) || symptoms.length === 0) {
      return sendError(res, 400, 'At least one symptom must be provided in the symptoms array', {
        field: 'symptoms',
        code: 'EMPTY_SYMPTOMS'
      });
    }

    // 1. Run the symptom-grounded prediction engine (validated ML model, local
    //    heuristic fallback). Gemini only explains candidate conditions.
    const { provider, prediction, insights, fallbackReason } = await runPrediction({
      symptoms,
      duration: duration || DEFAULT_SYMPTOM_DURATION,
      severity: clientSeverity || ''
    });

    // Unrecognized input: ask the user to clarify instead of returning a disease
    if (prediction.needs_clarification) {
      return sendError(res, 400, prediction.clarification_message, {
        code: SYMPTOMS_NOT_RECOGNIZED_CODE,
        unrecognized_symptoms: prediction.unrecognized_symptoms || [],
        suggestions: prediction.suggestions || []
      });
    }

    // 2. Clinical severity and care recommendation alignment
    let severity = clientSeverity || prediction.severity || 'moderate';
    if (insights?.urgencyAssessment === 'high') {
      severity = 'severe';
    }
```

```
    const primaryDisease =
      prediction.primary_condition || insights?.geminiPredictedDisease || 'Preliminary Symptom Evaluation';
    const parsedConfidence = Number(prediction.confidence);
    const confidence = Number.isFinite(parsedConfidence)
      ? Math.min(100, Math.max(0, parsedConfidence))
      : 0;
    const confidenceBasis = prediction.confidence_basis || CONFIDENCE_BASIS.NONE;
    const description = prediction.description || insights?.clinicalExplanation || '';
    const recommendation =
      (insights?.whenToSeeDoctor ? `${insights.whenToSeeDoctor} ` : '') +
      (prediction.recommendation || CARE_RECOMMENDATIONS[severity] || CARE_RECOMMENDATIONS.moderate);

    // Advisory only for engine-flagged low-confidence results — a 1-symptom
    // submission is no longer blocked or auto-flagged (Gemini/ML still
    // produce a real prediction for it).
    const isLowConfidence = prediction.low_confidence === true;

    // 3. Persist diagnosis record into MongoDB Atlas
    const diagnosis = await Diagnosis.create({
      user: req.user._id,
      symptoms,
      predictedDisease: primaryDisease,
      confidence,
      confidenceBasis,
      severity,
      topPredictions: prediction.top_predictions?.length > 0 ? prediction.top_predictions : (insights?.geminiTopPredictions || []),
      description,
      duration: duration || DEFAULT_SYMPTOM_DURATION,
      notes: notes || '',
      recommendation,
      disclaimer: prediction.disclaimer || MANDATORY_DISCLAIMER,
      isFallback: prediction.isFallback || false,
      unrecognizedSymptoms: prediction.unrecognized_symptoms || [],
      status: 'completed',
      predictionProvider: provider,
      providerFallbackReason: fallbackReason,
      geminiInsights: insights || null,
      geminiEnriched: insights?.geminiEnriched === true
    });

    return sendSuccess(res, 201, 'Diagnosis evaluated and saved successfully', {
      id: diagnosis._id,
      _id: diagnosis._id,
      user: diagnosis.user,
      symptoms: diagnosis.symptoms,
      recognized_symptoms: prediction.recognized_symptoms || diagnosis.symptoms,
      unrecognized_symptoms: prediction.unrecognized_symptoms || [],
      predictedDisease: diagnosis.predictedDisease,
      primary_condition: diagnosis.predictedDisease,   // alias for frontend compatibility
      confidence: diagnosis.confidence,
      confidenceBasis: diagnosis.confidenceBasis,
      severity: diagnosis.severity,
      topPredictions: diagnosis.topPredictions,
      description: diagnosis.description,
      duration: diagnosis.duration,
      notes: diagnosis.notes,
      recommendation: diagnosis.recommendation,
      disclaimer: diagnosis.disclaimer,
      isFallback: diagnosis.isFallback,
      predictionProvider: diagnosis.predictionProvider,
      providerFallbackReason: diagnosis.providerFallbackReason || null,
      low_confidence: isLowConfidence,
      low_confidence_message: isLowConfidence
        ? (prediction.low_confidence_message || LOW_CONFIDENCE_MESSAGE)
        : null,
      createdAt: diagnosis.createdAt,
      // Gemini insights (candidate explanations; local rules when unavailable)
      geminiInsights: diagnosis.geminiInsights || null,
      geminiEnriched: diagnosis.geminiEnriched || false,
      combinedPrediction: insights?.combinedPrediction || {
        predictedDisease: diagnosis.predictedDisease,
        confidence: diagnosis.confidence,
        method: confidenceBasis === CONFIDENCE_BASIS.MODEL_PROBABILITY
          ? 'SymptomDiseaseClassifier probability'
          : 'Preliminary symptom assessment (no validated probability available)'
      },
      accuracyMetrics: insights?.accuracyMetrics || null
    });
  } catch (error) {
    return next(error);
  }
};

/**
 * GET /api/diagnoses
 * GET /api/diagnoses/history
 * Returns paginated diagnosis history for the authenticated user.
 */
export const getDiagnoses = async (req, res, next) => {
  try {
    const page = Math.max(1, parseInt(req.query.page, 10) || 1);
    const limit = Math.min(50, Math.max(1, parseInt(req.query.limit, 10) || 10));
    const skip = (page - 1) * limit;

    const filter = { user: req.user._id };

    const [diagnoses, total] = await Promise.all([
      Diagnosis.find(filter)
        .sort({ createdAt: -1 })
        .skip(skip)
        .limit(limit)
        .lean(),
      Diagnosis.countDocuments(filter)
    ]);
```

```
    const totalPages = Math.ceil(total / limit) || 1;

    return sendSuccess(res, 200, 'Diagnoses retrieved successfully', {
      diagnoses,
      pagination: {
        total,
        page,
        limit,
        totalPages
      }
    });
  } catch (error) {
    return next(error);
  }
};

/**
 * GET /api/diagnoses/:id
 * Retrieves a single diagnosis record, enforcing user ownership.
 */
export const getDiagnosis = async (req, res, next) => {
  try {
    const { id } = req.params;

    if (!mongoose.Types.ObjectId.isValid(id)) {
      return sendError(res, 400, 'Invalid diagnosis ID format', { code: 'INVALID_ID' });
    }

    const diagnosis = await Diagnosis.findOne({ _id: id, user: req.user._id }).lean();
    if (!diagnosis) {
      return sendError(res, 404, 'Diagnosis record not found', { code: 'NOT_FOUND' });
    }

    return sendSuccess(res, 200, 'Diagnosis retrieved successfully', { diagnosis });
  } catch (error) {
    return next(error);
  }
};

/**
 * DELETE /api/diagnoses/:id
 * Deletes a diagnosis record, verifying user ownership.
 */
export const deleteDiagnosis = async (req, res, next) => {
  try {
    const { id } = req.params;

    if (!mongoose.Types.ObjectId.isValid(id)) {
      return sendError(res, 400, 'Invalid diagnosis ID format', { code: 'INVALID_ID' });
    }

    const deleted = await Diagnosis.findOneAndDelete({ _id: id, user: req.user._id });
    if (!deleted) {
      return sendError(res, 404, 'Diagnosis record not found or unauthorized', { code: 'NOT_FOUND' });
    }

    return sendSuccess(res, 200, 'Diagnosis deleted successfully', { id });
  } catch (error) {
    return next(error);
  }
};
```

backend/controllers/doctorController.js

Doctor availability and appointment orchestration. File path shown; live private control files are excluded.

```
import fs from 'fs';
import path from 'path';
import { createHash } from 'crypto';
import { fileURLToPath } from 'url';
import Doctor from '../models/Doctor.js';
import Appointment from '../models/Appointment.js';
import { sendSuccess, sendError } from '../utils/apiResponse.js';
import { APPOINTMENT_STATUSES, DEFAULT_DOCTOR_AVAILABILITY_SLOTS } from '../config/constants.js';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const doctorControlPath = path.resolve(__dirname, '../config/doctors-control.json');
const bookingsControlPath = path.resolve(__dirname, '../config/bookings-control.json');

const readBookingControl = () => {
  try {
    const control = JSON.parse(fs.readFileSync(bookingsControlPath, 'utf8'));
    if (!control || typeof control !== 'object' || Array.isArray(control)) return {};
    return control;
  } catch (_error) {
    return {};
  }
};

const hasBookingEntry = (control, bookingId) =>
  Object.prototype.hasOwnProperty.call(control, bookingId);

const toBookingControlEntry = (appointment, patientName = appointment.user?.name || '') => {
  const appointmentDate = appointment.appointmentDate ? new Date(appointment.appointmentDate) : null;
  return {
    bookingId: appointment._id.toString(),
    patientName,
    doctorName: appointment.doctorName,
    specialty: appointment.specialty,
    date: appointmentDate && !Number.isNaN(appointmentDate.getTime())
      ? appointmentDate.toISOString().slice(0, 10)
      : '',
```

```
    time: appointment.timeSlot,
    reason: appointment.reason || '',
    status: 'pending'
  };
};

const writeBookingControl = (control) => {
  try {
    fs.writeFileSync(bookingsControlPath, `${JSON.stringify(control, null, 2)}\n`, 'utf8');
  } catch (_error) {
    // File errors never interrupt booking or appointment requests.
  }
};

const addBookingToControl = (appointment, patientName) => {
  try {
    const control = readBookingControl();
    const bookingId = appointment._id.toString();
    if (hasBookingEntry(control, bookingId)) return;

    control[bookingId] = toBookingControlEntry(appointment, patientName);
    writeBookingControl(control);
  } catch (_error) {
    // Booking creation remains successful if the local approval file is unavailable.
  }
};

export const syncBookingControl = async () => {
  try {
    const appointments = await Appointment.find({}).populate('user', 'name').lean();
    const control = readBookingControl();
    let changed = false;

    appointments.forEach((appointment) => {
      const bookingId = appointment._id.toString();
      if (hasBookingEntry(control, bookingId)) return;
      control[bookingId] = toBookingControlEntry(appointment);
      changed = true;
    });

    if (changed) writeBookingControl(control);
  } catch (_error) {
    // Keep startup and list requests available if the control file cannot be synced.
  }
};

const readDoctorAvailabilityControl = () => {
  try {
    const control = JSON.parse(fs.readFileSync(doctorControlPath, 'utf8'));
    if (!control || typeof control !== 'object' || Array.isArray(control)) return null;
    return control;
  } catch (_error) {
    return null;
  }
};

const getDoctorAvailability = (doctor, control) =>
  control && typeof control[doctor.name] === 'boolean' ? control[doctor.name] : doctor.available;

const ensureControlDoctors = async (control) => {
  if (!control) return;

  for (const [name, available] of Object.entries(control)) {
    if (!name.trim() || typeof available !== 'boolean') continue;

    const nameSlug = name.toLowerCase().replace(/[^a-z0-9]+/g, '.').replace(/^\.|\.$/g, '') || 'doctor';
    const nameHash = createHash('sha256').update(name).digest('hex').slice(0, 10);

    await Doctor.findOneAndUpdate(
      { name },
      {
        $setOnInsert: {
          name,
          specialty: 'General Physician',
          email: `${nameSlug}.${nameHash}@doctors-control.local`,
          available: true,
          rating: 4.5,
          consultationFee: 50,
          availabilitySlots: DEFAULT_DOCTOR_AVAILABILITY_SLOTS,
          description: 'Provides general healthcare consultations for common health concerns.'
        }
      },
      { upsert: true, new: true, setDefaultsOnInsert: true }
    );
  }
};

const defaultDoctors = [
  {
    name: 'Dr. Maya Patel',
    specialty: 'Pulmonologist',
    email: 'maya.patel@carepath.health',
    available: true,
    rating: 4.9,
    consultationFee: 49,
    availabilitySlots: DEFAULT_DOCTOR_AVAILABILITY_SLOTS,
    description: 'Focuses on preventive care, routine checkups, and early symptom triage.'
  },
  {
    name: 'Dr. Aiden Brooks',
    specialty: 'Cardiologist',
    email: 'aiden.brooks@carepath.health',
    available: true,
    rating: 4.8,
    consultationFee: 79,
```

```
    availabilitySlots: ['09:30', '11:00', '15:00'],
    description: 'Supports chest-related concerns, heart rhythm questions, and follow-up monitoring.'
  },
  {
    name: 'Dr. Sofia Nguyen',
    specialty: 'Dermatologist',
    email: 'sofia.nguyen@carepath.health',
    available: true,
    rating: 4.7,
    consultationFee: 64,
    availabilitySlots: ['08:30', '12:00', '13:30'],
    description: 'Reviews skin, allergic, and rash-related symptoms with practical care guidance.'
  },
  {
    name: 'Dr. Liam Hassan',
    specialty: 'Pulmonologist',
    email: 'liam.hassan@carepath.health',
    available: true,
    rating: 4.9,
    consultationFee: 82,
    availabilitySlots: ['10:00', '11:30', '17:00'],
    description: 'Helps with respiratory symptoms, chest discomfort, and breathing pattern concerns.'
  },
  {
    name: 'Dr. SWAPAN JANA',
    specialty: 'Gastroenterologist',
    email: 'swapan.jana@carepath.health',
    available: true,
    rating: 4.9,
    consultationFee: 100,
    availabilitySlots: ['11:00', '13:30', '17:00'],
    description: 'Provides primary healthcare consultations for common illnesses, fever, cold, cough, infections, and general health concerns.'
  },{
  name: 'Dr. Arjun Sharma',
  specialty: 'Neurologist',
  email: 'arjun.sharma@carepath.health',
  available: true,
  rating: 4.8,
  consultationFee: 90,
  availabilitySlots: ['09:00', '12:30', '16:00'],
  description: 'Provides consultations for heart health, blood pressure concerns, and cardiovascular risk assessment.'
},
{
  name: 'Dr. Emily Carter',
  specialty: 'Neurologist',
  email: 'emily.carter@carepath.health',
  available: true,
  rating: 4.8,
  consultationFee: 85,
  availabilitySlots: ['10:00', '13:00', '15:30'],
  description: 'Evaluates headaches, dizziness, nerve-related symptoms, and other neurological concerns.'
},
{
  name: 'Dr. Priya Patel',
  specialty: 'Pediatrician',
  email: 'priya.patel@carepath.health',
  available: true,
  rating: 4.9,
  consultationFee: 70,
  availabilitySlots: ['09:30', '12:00', '17:30'],
  description: 'Provides healthcare consultations for children, including common illnesses, growth, and preventive care.'
},
{
  name: 'Dr. Daniel Kim',
  specialty: 'Orthopedic',
  email: 'daniel.kim@carepath.health',
  available: true,
  rating: 4.7,
  consultationFee: 80,
  availabilitySlots: ['08:30', '11:00', '16:30'],
  description: 'Consults on joint pain, bone injuries, muscle problems, and mobility concerns.'
},
{
  name: 'Dr. Aisha Rahman',
  specialty: 'Gynecologist',
  email: 'aisha.rahman@carepath.health',
  available: true,
  rating: 4.9,
  consultationFee: 85,
  availabilitySlots: ['10:30', '13:30', '18:00'],
  description: 'Provides consultations for menstrual health, reproductive health, and common gynecological concerns.'
},
{
  name: 'Dr. Michael Brown',
  specialty: 'ENT',
  email: 'michael.brown@carepath.health',
  available: true,
  rating: 4.6,
  consultationFee: 75,
  availabilitySlots: ['09:00', '12:30', '15:00'],
  description: 'Evaluates digestive concerns, including acidity, abdominal discomfort, and changes in bowel habits.'
},
{
  name: 'Dr. Olivia Wilson',
  specialty: 'Pediatrician',
  email: 'olivia.wilson@carepath.health',
  available: true,
  rating: 4.8,
  consultationFee: 65,
  availabilitySlots: ['08:00', '11:30', '14:30'],
  description: 'Provides consultations for eye irritation, vision concerns, and common eye conditions.'
},
{
  name: 'Dr. Ethan Davis',
  specialty: 'Psychiatrist',
```

```
  email: 'ethan.davis@carepath.health',
  available: true,
  rating: 4.7,
  consultationFee: 60,
  availabilitySlots: ['09:30', '13:00', '17:00'],
  description: 'Evaluates ear pain, hearing concerns, sinus problems, and throat-related symptoms.'
},
{
  name: 'Dr. Meera Das',
  specialty: 'Dermatologist',
  email: 'meera.das@carepath.health',
  available: true,
  rating: 4.8,
  consultationFee: 90,
  availabilitySlots: ['10:00', '14:00', '18:30'],
  description: 'Provides mental health consultations for stress, anxiety, mood concerns, and emotional well-being.'
},
{
  name: 'Dr. Noah Martin',
  specialty: 'Urologist',
  email: 'noah.martin@carepath.health',
  available: true,
  rating: 4.7,
  consultationFee: 80,
  availabilitySlots: ['09:00', '12:00', '16:00'],
  description: 'Evaluates urinary symptoms, bladder concerns, and other urological health conditions.'
}
];

const ensureDoctorSeed = async () => {
  const count = await Doctor.countDocuments();
  if (count > 0) return;
  await Doctor.insertMany(defaultDoctors);
};

export const getDoctors = async (_req, res, next) => {
  try {
    await ensureDoctorSeed();
    const control = readDoctorAvailabilityControl();
    await ensureControlDoctors(control);
    const doctors = await Doctor.find({}).sort({ specialty: 1, name: 1 }).lean();
    const doctorsWithAvailability = doctors.map((doctor) => ({
      ...doctor,
      available: getDoctorAvailability(doctor, control)
    }));
    return sendSuccess(res, 200, 'Doctors retrieved successfully', { doctors: doctorsWithAvailability });
  } catch (error) {
    next(error);
  }
};

export const getAppointments = async (req, res, next) => {
  try {
    await syncBookingControl();
    const appointments = await Appointment.find({ user: req.user._id })
      .sort({ appointmentDate: 1 })
      .lean();
    const bookingControl = readBookingControl();
    const appointmentsWithStatus = appointments.map((appointment) => ({
      ...appointment,
      status: bookingControl[appointment._id.toString()]?.status === 'approved' ? 'approved' : 'pending'
    }));

    return sendSuccess(res, 200, 'Appointments retrieved successfully', { appointments: appointmentsWithStatus });
  } catch (error) {
    next(error);
  }
};

export const createAppointment = async (req, res, next) => {
  try {
    const { doctorId, date, timeSlot, reason } = req.body;

    if (!doctorId || !date || !timeSlot) {
      return sendError(res, 400, 'Doctor, date, and time slot are required', {
        code: 'APPOINTMENT_MISSING_REQUIRED_FIELDS'
      });
    }

    const doctor = await Doctor.findById(doctorId);
    if (!doctor) {
      return sendError(res, 404, 'Selected doctor was not found', {
        code: 'DOCTOR_NOT_FOUND'
      });
    }

    const control = readDoctorAvailabilityControl();
    if (getDoctorAvailability(doctor, control) === false) {
      return sendError(res, 400, `${doctor.name} is currently marked as unavailable for appointments in the system. Please select an available
    doctor.`, {
        code: 'DOCTOR_UNAVAILABLE'
      });
    }

    if (!doctor.availabilitySlots?.includes(timeSlot)) {
      return sendError(res, 400, 'Selected time slot is not available for this doctor', {
        code: 'APPOINTMENT_TIME_SLOT_UNAVAILABLE'
      });
    }

    const dateString = `${date}T${timeSlot}:00`;
    const appointmentDate = new Date(dateString);

    if (Number.isNaN(appointmentDate.getTime())) {
      return sendError(res, 400, 'Please choose a valid appointment date and time', {
```

```
        code: 'APPOINTMENT_INVALID_DATE'
      });
    }

    const appointment = await Appointment.create({
      user: req.user._id,
      doctor: doctor._id,
      doctorName: doctor.name,
      specialty: doctor.specialty,
      appointmentDate,
      timeSlot,
      reason: reason?.trim() || 'Follow-up consultation',
      status: APPOINTMENT_STATUSES[0]
    });
    addBookingToControl(appointment, req.user.name);

    return sendSuccess(res, 201, 'Appointment booked successfully', { appointment });
  } catch (error) {
    if (error.name === 'ValidationError') {
      return sendError(res, 400, error.message, { details: error.errors });
    }
    next(error);
  }
};
```

backend/controllers/chatController.js

Chat request orchestration and response composition.

```
import { processChatMessage } from '../services/chatEngine.js';
import { MANDATORY_DISCLAIMER } from '../services/mlService.js';
import { generateGeminiChatReply } from '../services/geminiService.js';
import { runPrediction } from '../services/predictionService.js';
import Diagnosis from '../models/Diagnosis.js';
import { sendSuccess, sendError } from '../utils/apiResponse.js';
import { CARE_RECOMMENDATIONS, DEFAULT_SYMPTOM_DURATION } from '../config/constants.js';

/**
 * POST /api/chat/message
 * Processes conversational user messages, extracts clinical symptoms,
 * provides empathetic responses, and optionally runs the prediction flow
 * (Gemini-first with ML/heuristic fallback).
 */
export const handleChatMessage = async (req, res, next) => {
  try {
    const { message, conversationHistory = [], currentSymptoms = [], action, duration, notes } = req.body;

    if (!message && action !== 'evaluate') {
      return sendError(res, 400, 'Message text or evaluate action is required', { field: 'message' });
    }

    const chatResult = await processChatMessage({
      message: message || '',
      conversationHistory,
      currentSymptoms
    });

    let diagnosisResult = null;

    // Run inference if requested by user or triggered by chat engine
    const shouldEvaluate = action === 'evaluate' || chatResult.shouldRunInference;
    const symptomsToEvaluate = chatResult.detectedSymptoms.length > 0 ? chatResult.detectedSymptoms : currentSymptoms;

    if (shouldEvaluate && symptomsToEvaluate.length > 0) {
      // Symptom-grounded prediction (ML engine first; Gemini only explains)
      const { provider, prediction, insights, fallbackReason } = await runPrediction({
        symptoms: symptomsToEvaluate,
        duration: duration || DEFAULT_SYMPTOM_DURATION
      });

      // Unrecognized input → ask the user to clarify, never invent a disease
      if (prediction.needs_clarification) {
        chatResult.reply += `\n\n  **I need a bit more clarity:** ${prediction.clarification_message}`;
        return sendSuccess(res, 200, 'Message processed successfully', {
          reply: chatResult.reply,
          detectedSymptoms: chatResult.detectedSymptoms,
          suggestedSymptoms: chatResult.suggestedSymptoms,
          isEmergency: chatResult.isEmergency,
          diagnosisResult: null,
          clarification: {
            code: 'SYMPTOMS_NOT_RECOGNIZED',
            unrecognized_symptoms: prediction.unrecognized_symptoms || [],
            suggestions: prediction.suggestions || []
          },
          disclaimer: MANDATORY_DISCLAIMER
        });
      }

      let severity = prediction.severity || 'moderate';
      if (insights?.urgencyAssessment === 'high') {
        severity = 'severe';
      }

      const recommendation =
        (insights?.whenToSeeDoctor ? `${insights.whenToSeeDoctor} ` : '') +
        (prediction.recommendation || CARE_RECOMMENDATIONS[severity] || CARE_RECOMMENDATIONS.moderate);

      diagnosisResult = {
        predictedDisease: prediction.primary_condition,
        primary_condition: prediction.primary_condition,
        confidence: prediction.confidence,
```

```
        confidenceBasis: prediction.confidence_basis || 'none',
        severity,
        topPredictions: prediction.top_predictions || [],
        description: prediction.description || '',
        symptoms: symptomsToEvaluate,
        recognized_symptoms: prediction.recognized_symptoms || symptomsToEvaluate,
        unrecognized_symptoms: prediction.unrecognized_symptoms || [],
        recommendation,
        disclaimer: prediction.disclaimer || MANDATORY_DISCLAIMER,
        isFallback: prediction.isFallback || false,
        predictionProvider: provider,
        providerFallbackReason: fallbackReason || null,
        geminiInsights: insights || null,
        geminiEnriched: insights?.geminiEnriched === true,
        combinedPrediction: insights?.combinedPrediction || {
          predictedDisease: prediction.primary_condition,
          confidence: prediction.confidence,
          method: (prediction.confidence_basis === 'model_probability')
            ? 'SymptomDiseaseClassifier probability'
            : 'Preliminary symptom assessment (no validated probability available)'
        },
        accuracyMetrics: insights?.accuracyMetrics || null
      };

      // Persist to MongoDB if authenticated
      if (req.user) {
        const record = await Diagnosis.create({
          user: req.user._id,
          symptoms: symptomsToEvaluate,
          predictedDisease: prediction.primary_condition,
          confidence: Number.isFinite(Number(prediction.confidence)) ? Number(prediction.confidence) : 0,
          confidenceBasis: prediction.confidence_basis || 'none',
          severity,
          topPredictions: prediction.top_predictions || [],
          description: prediction.description || '',
          duration: duration || DEFAULT_SYMPTOM_DURATION,
          notes: notes || 'Generated via AI Chatbot Triage assessment',
          recommendation,
          disclaimer: prediction.disclaimer || MANDATORY_DISCLAIMER,
          isFallback: prediction.isFallback || false,
          unrecognizedSymptoms: prediction.unrecognized_symptoms || [],
          status: 'completed',
          predictionProvider: provider,
          providerFallbackReason: fallbackReason || null,
          geminiInsights: insights || null,
          geminiEnriched: insights?.geminiEnriched === true
        });
        diagnosisResult.id = record._id;
        diagnosisResult._id = record._id;
      }

      const confidenceLabel = prediction.confidence_basis === 'model_probability'
        ? `${prediction.confidence}% model probability`
        : `${prediction.confidence}% preliminary score`;
      chatResult.reply +=
        `\n\n  **Diagnostic Triage Result:**\n` +
        `• **Primary Condition:** ${prediction.primary_condition} (${confidenceLabel})\n` +
        `• **Triage Severity:** ${severity.toUpperCase()}\n` +
        `• **Care Advice:** ${recommendation}`;

      if (insights?.clinicalExplanation) {
        chatResult.reply += `\n\n  **Clinical Insight:** ${insights.clinicalExplanation}`;
      }
    } else if (!chatResult.isEmergency && chatResult.isHealthRelated && message) {
      // Natural Gemini conversational enhancement when not evaluating and not emergency
      const geminiReply = await generateGeminiChatReply({
        userMessage: message,
        detectedSymptoms: chatResult.detectedSymptoms,
        conversationHistory,
        isEmergency: chatResult.isEmergency
      });
      if (geminiReply) {
        chatResult.reply = geminiReply;
      }
    }

    return sendSuccess(res, 200, 'Message processed successfully', {
      reply: chatResult.reply,
      detectedSymptoms: chatResult.detectedSymptoms,
      suggestedSymptoms: chatResult.suggestedSymptoms,
      isEmergency: chatResult.isEmergency,
      diagnosisResult,
      disclaimer: MANDATORY_DISCLAIMER
    });
  } catch (error) {
    return next(error);
  }
};
```

Services

backend/services/predictionService.js

Prediction provider selection, candidate engine, fallback behavior, and environment checks.

```
import { predictDiagnosis as runMlPrediction, INJURY_PATTERN } from './mlService.js';
import {
  enrichDiagnosisWithGemini,
  buildLocalClinicalInsights,
  classifyGeminiError,
  generateGeminiPrediction
```

```
} from './geminiService.js';
import {
  PREDICTION_PROVIDER_GEMINI,
  PREDICTION_PROVIDER_ML,
  PREDICTION_PROVIDER_HEURISTIC,
  PREDICTION_PROVIDER_ENV_DEFAULT,
  DEFAULT_SYMPTOM_DURATION
} from '../config/constants.js';
import { MANDATORY_DISCLAIMER } from './mlService.js';

/**
 * Resolves the configured explanation mode from the environment.
 * - 'gemini' (default): the symptom engine grounds the prediction; Gemini
 *   additionally explains the candidate conditions.
 * - 'ml': same symptom-grounded prediction with local rule-based explanations
 *   only (no external Gemini calls).
 * Unknown values log a safe warning and fall back to the default, keeping the
 * service available instead of failing closed.
 */
export const resolvePredictionProvider = (rawProvider = process.env.PREDICTION_PROVIDER) => {
  const normalized = String(rawProvider ?? '').trim().toLowerCase();

  if (normalized === PREDICTION_PROVIDER_ML) return PREDICTION_PROVIDER_ML;
  if (normalized === PREDICTION_PROVIDER_GEMINI) return PREDICTION_PROVIDER_GEMINI;
  if (normalized !== '') {
    console.warn(`[Prediction Service] Unknown PREDICTION_PROVIDER "${normalized}". Defaulting to "${PREDICTION_PROVIDER_ENV_DEFAULT}".`);
  }
  return PREDICTION_PROVIDER_ENV_DEFAULT;
};

/**
 * Logs explanation-provider failures safely — error codes only, never prompts,
 * API keys, or patient-supplied data.
 */
const logExplanationFallback = (error) => {
  const code = error?.code || classifyGeminiError(error);
  console.warn(`[Prediction Service] Gemini explanations unavailable (${code}). Using local rule-based explanations.`);
  return code;
};

/**
 * Orchestrates the disease-prediction flow.
 *
 * 1. In the default 'gemini' mode, Gemini generates a preliminary top-3
 *    differential for ANY submitted symptom(s) — including a single symptom
 *    or unusual entries the taxonomy cannot resolve. The orthopedic-trauma
 *    safety rule and any Gemini failure/timeout degrade to the symptom
 *    engine below, so the app never crashes.
 * 2. Fallback: the Python ML engine (validated classifier) grounds the
 *    candidates — or the local heuristic overlap engine when the service is
 *    unreachable. Clarification results (unrecognized input) pass through
 *    untouched so callers can ask the user for better input.
 * 3. On the fallback path in 'gemini' mode, Gemini (or local rules) only
 *    EXPLAINS the engine's candidate conditions; 'ml' mode uses local
 *    rule-based explanations with no external calls.
 *
 * @returns {Promise<{provider: string, prediction: object, insights: object|null, fallbackReason: string|null}>}
 *   `provider` reports the candidate-engine slot ('ml' | 'heuristic') — the
 *   UI's dual-consensus layout key; Gemini's role is surfaced through
 *   insights.poweredBy / combinedPrediction.method.
 *   `fallbackReason` records why Gemini was unavailable.
 */
export const runPrediction = async (
  { symptoms = [], duration = DEFAULT_SYMPTOM_DURATION, severity = '' },
  deps = {}
) => {
  const mlPredictor = deps.mlPredictor || runMlPrediction;
  const enricher = deps.enricher || enrichDiagnosisWithGemini;
  const geminiPredictor = deps.geminiPredictor || generateGeminiPrediction;
  const provider = deps.provider || resolvePredictionProvider();

  // Default 'gemini' mode: send every non-empty submitted entry for a
  // preliminary prediction; local injury rules still enforce urgent triage.
  if (provider === PREDICTION_PROVIDER_GEMINI && symptoms.some((item) => String(item).trim())) {
    try {
      const geminiResult = await geminiPredictor({ symptoms, duration, severity });
      if (geminiResult?.prediction && geminiResult?.insights) {
        const emergency = INJURY_PATTERN.test(symptoms.join(' ')) || /chest pain|difficulty breathing|shortness of breath|uncontrolled
    bleeding|stroke|seizure|unconscious|severe allergic/i.test(symptoms.join(' '));
        if (emergency) {
          geminiResult.prediction.severity = 'severe';
          geminiResult.insights.urgencyAssessment = 'high';
        }
        return {
          // 'ml' keeps the frontend's dual-consensus layout stable (it treats
          // the legacy 'gemini' provider value as explanation-only mode and
          // hides the consensus cards). Gemini's role is surfaced truthfully
          // via insights.poweredBy and combinedPrediction.method.
          provider: PREDICTION_PROVIDER_ML,
          prediction: geminiResult.prediction,
          insights: geminiResult.insights,
          fallbackReason: null
        };
      }
    } catch (error) {
      const code = classifyGeminiError(error);
      // Log the safe code only — never the raw SDK message, API key, or prompt.
      console.warn(`[Prediction Service] Gemini prediction unavailable (${code}). Falling back to the symptom engine.`);
    }
  }

  let mlResult;
  try {
    mlResult = await mlPredictor(symptoms);
  } catch (_error) {
```

```
    mlResult = { needs_clarification: true, isFallback: true, unrecognized_symptoms: symptoms };
  }
  const candidateProvider = mlResult.isFallback ? PREDICTION_PROVIDER_HEURISTIC : PREDICTION_PROVIDER_ML;

  // Keep a usable prediction when local engines cannot map a genuine health
  // term into their narrower reference taxonomy.
  if (mlResult.needs_clarification) {
    const input = symptoms.map((item) => String(item).trim()).filter(Boolean).join(', ');
    const isRandom = /^(?:[a-z]{4,}\s*)+$/.test(input) &&
    !/\b(?:pain|ache|fever|rash|cough|diabetes|injury|broken|fracture|fall|hair|headache|leg)\b/i.test(input);
    const emergency = /chest pain|difficulty breathing|shortness of breath|uncontrolled bleeding|stroke|seizure|unconscious|severe
    allergic/i.test(input);
    const prediction = {
      success: true,
      primary_condition: `Preliminary assessment for ${input || 'reported health concern'}`,
      confidence: 5,
      confidence_basis: 'gemini_estimate',
      severity: emergency ? 'severe' : 'mild',
      description: `The reported health concern (${input}) is not covered by the local reference model. This low-confidence result is general triage
    guidance and needs assessment by a qualified clinician.`,
      recommendation: emergency ? 'Contact local emergency services now.' : 'Consider contacting a healthcare professional for an assessment,
    especially if this is new, persistent, worsening, or concerning.',
      top_predictions: [{ disease: `Unspecified condition related to ${input}`, confidence: 5, severity: emergency ? 'severe' : 'mild', description:
    'The local model cannot reliably classify this input; this is a low-confidence placeholder.' }],
      recognized_symptoms: mlResult.recognized_symptoms || [],
      unrecognized_symptoms: mlResult.unrecognized_symptoms || symptoms,
      disclaimer: MANDATORY_DISCLAIMER,
      isFallback: true,
      low_confidence: true,
      low_confidence_message: 'The local reference model could not classify this entry; this is a low-confidence preliminary assessment.'
    };
    if (isRandom) prediction.description = 'The entry could not be interpreted as a health concern. This low-confidence placeholder is not a medical
    assessment.';
    const insights = buildLocalClinicalInsights(symptoms, prediction);
    if (emergency) insights.urgencyAssessment = 'high';
    return {
      provider: candidateProvider,
      prediction,
      insights,
      fallbackReason: null
    };
  }

  if (provider === PREDICTION_PROVIDER_GEMINI) {
    let insights = null;
    let fallbackReason = null;

    try {
      insights = await enricher({ symptoms, mlResult, duration, severity });
    } catch (error) {
      fallbackReason = logExplanationFallback(error);
      insights = null;
    }

    if (insights && insights.geminiEnriched === false && insights.enrichmentErrorCode) {
      fallbackReason = insights.enrichmentErrorCode;
    }

    return {
      provider: candidateProvider,
      prediction: mlResult,
      insights: insights || buildLocalClinicalInsights(symptoms, mlResult),
      fallbackReason
    };
  }

  // PREDICTION_PROVIDER=ml → local rule-based explanations, no Gemini calls
  return {
    provider: candidateProvider,
    prediction: mlResult,
    insights: buildLocalClinicalInsights(symptoms, mlResult),
    fallbackReason: null
  };
};

/**
 * Startup environment validation (non-fatal warnings only — the service must
 * stay available and degrade gracefully at request time).
 */
export const validatePredictionEnvironment = (env = process.env) => {
  const provider = resolvePredictionProvider(env.PREDICTION_PROVIDER);
  const warnings = [];

  if (provider === PREDICTION_PROVIDER_GEMINI && !String(env.GEMINI_API_KEY || '').trim()) {
    warnings.push(
      'GEMINI_API_KEY is not set. Predictions remain grounded in the ML engine, but Gemini candidate explanations are disabled. Create a key at
    https://aistudio.google.com/apikey to enable them.'
    );
  }

  return { provider, warnings };
};

export default {
  resolvePredictionProvider,
  runPrediction,
  validatePredictionEnvironment
};
```

backend/services/geminiService.js

Gemini request construction, output validation, explanation and chat retry/fallback.

```
import { GoogleGenAI } from '@google/genai';
import fs from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';
import { INJURY_PATTERN, MANDATORY_DISCLAIMER, resolveSymptomId } from './mlService.js';
import {
  GEMINI_DEFAULT_MODEL,
  GEMINI_ENRICHMENT_TIMEOUT_MS,
  GEMINI_PREDICTION_TIMEOUT_MS,
  GEMINI_MAX_OUTPUT_TOKENS,
  GEMINI_TEMPERATURE,
  GEMINI_CHAT_MAX_OUTPUT_TOKENS,
  GEMINI_CHAT_TEMPERATURE,
  GEMINI_CHAT_TIMEOUT_MS,
  GEMINI_CHAT_HISTORY_TURNS,
  GEMINI_MAX_TOP_PREDICTIONS,
  GEMINI_MAX_ARRAY_ITEMS,
  GEMINI_MAX_CONDITION_LENGTH,
  GEMINI_MAX_TEXT_LENGTH,
  GEMINI_MAX_SHORT_TEXT_LENGTH,
  GEMINI_MAX_ARRAY_ITEM_LENGTH,
  GEMINI_ERROR_CODES,
  URGENCY_LEVELS,
  CONFIDENCE_BASIS,
  DEFAULT_SYMPTOM_DURATION
} from '../config/constants.js';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

/**
 * Gemini plays two roles, both server-side only (the API key never reaches
 * the frontend):
 * 1. PRELIMINARY PREDICTION (default 'gemini' provider mode): Gemini returns
 *    a top-3 differential with preliminary probability estimates for ANY
 *    submitted symptom(s) — including a single symptom or unusual entries the
 *    taxonomy cannot resolve. Every estimate is labelled with the
 *    'gemini_estimate' confidence basis (never a validated probability), and
 *    any Gemini failure falls back to the symptom-grounded engine.
 * 2. EXPLANATION: on the fallback path Gemini explains the symptom engine's
 *    candidate conditions (supporting / missing symptoms, self-care, warning
 *    signs); it never re-chooses the engine's condition there.
 */

/**
 * Typed Gemini provider failure. `code` is always one of GEMINI_ERROR_CODES and
 * is safe to log or surface — it never contains API keys or patient data.
 */
export class GeminiProviderError extends Error {
  constructor(code, message) {
    super(message);
    this.name = 'GeminiProviderError';
    this.code = code;
  }
}

/**
 * Maps Gemini SDK / HTTP failures onto safe, actionable error codes.
 */
export const classifyGeminiError = (error) => {
  if (error instanceof GeminiProviderError) return error.code;

  const status = Number(error?.status ?? error?.statusCode ?? 0);
  const message = String(error?.message || '').toLowerCase();

  if (error?.name === 'AbortError' || message.includes('timeout') || message.includes('timed out')) {
    return GEMINI_ERROR_CODES.TIMEOUT;
  }
  if (status === 429 || message.includes('429') || message.includes('resource_exhausted') || message.includes('rate limit')) {
    return GEMINI_ERROR_CODES.RATE_LIMITED;
  }
  if (status === 401 || status === 403 || message.includes('api key') || message.includes('unauthenticated') || message.includes('permission_denied')) {
    return GEMINI_ERROR_CODES.AUTH_FAILED;
  }
  if (status === 400 || status === 422 || message.includes('invalid_argument')) {
    return GEMINI_ERROR_CODES.INVALID_REQUEST;
  }
  return GEMINI_ERROR_CODES.UNAVAILABLE;
};

/**
 * Resolves Gemini configuration from server-side environment variables only.
 * The API key must never be exposed to the frontend (no VITE_ prefix).
 */
export const resolveGeminiConfig = (env = process.env) => {
  const apiKey = String(env.GEMINI_API_KEY || '').trim();
  const model = String(env.GEMINI_MODEL || '').trim() || GEMINI_DEFAULT_MODEL;
  return { apiKey, model, isConfigured: apiKey.length > 0 };
};

export const isGeminiConfigured = (env = process.env) => resolveGeminiConfig(env).isConfigured;

const getGeminiClient = () => {
  const { apiKey, isConfigured } = resolveGeminiConfig();
  if (!isConfigured) return null;
  return new GoogleGenAI({ apiKey });
};

const extractResponseText = (response) => {
  if (!response) return '';
  if (typeof response.text === 'function') return response.text() || '';
  if (typeof response.text === 'string') return response.text;
  return '';
};
```

```
/**
 * Builds the shared Gemini request config. Thinking is disabled so the Flash
 * model returns structured triage JSON quickly and within the request timeout.
 */
const buildGeminiConfig = ({
  temperature = GEMINI_TEMPERATURE,
  maxOutputTokens = GEMINI_MAX_OUTPUT_TOKENS,
  systemInstruction = null,
  responseMimeType = null,
  responseSchema = null
} = {}) => {
  const config = {
    temperature,
    maxOutputTokens,
    thinkingConfig: { thinkingBudget: 0 }
  };
  if (systemInstruction) config.systemInstruction = systemInstruction;
  if (responseMimeType) config.responseMimeType = responseMimeType;
  if (responseSchema) config.responseSchema = responseSchema;
  return config;
};

// Load project disease & symptom taxonomies once at startup
let cachedDiseases = {};
let cachedSymptoms = [];

try {
  const diseasesPath = path.resolve(__dirname, '../data/diseases.json');
  const symptomsPath = path.resolve(__dirname, '../data/symptoms.json');
  if (fs.existsSync(diseasesPath)) {
    cachedDiseases = JSON.parse(fs.readFileSync(diseasesPath, 'utf8'));
  }
  if (fs.existsSync(symptomsPath)) {
    cachedSymptoms = JSON.parse(fs.readFileSync(symptomsPath, 'utf8'));
  }
} catch (err) {
  console.warn('[Gemini Service] Warning: Failed to load taxonomy data:', err.message);
}

export const resolveSymptomName = (symptomId) => {
  const found = cachedSymptoms.find((s) => s.id === symptomId);
  if (found) return found.name;
  return symptomId.replace(/_/g, ' ').replace(/\b\w/g, (c) => c.toUpperCase());
};

export const getDiseaseMetaFromCatalog = (diseaseName) => {
  const normalized = diseaseName?.toLowerCase().trim();
  for (const [name, meta] of Object.entries(cachedDiseases)) {
    if (name.toLowerCase() === normalized) {
      return { name, ...meta };
    }
  }
  return null;
};

export const computeSymptomCoverage = (symptoms = [], diseaseName = '') => {
  const meta = getDiseaseMetaFromCatalog(diseaseName);
  if (!meta || !Array.isArray(meta.coreSymptoms) || meta.coreSymptoms.length === 0) {
    return { matchingCount: 0, totalCore: 0, coverageRatio: 0 };
  }
  const symptomSet = new Set(symptoms.map((s) => s.toLowerCase().trim()));
  const matches = meta.coreSymptoms.filter((cs) => symptomSet.has(cs.toLowerCase().trim()));
  const coverageRatio = Math.round((matches.length / meta.coreSymptoms.length) * 100);
  return {
    matchingCount: matches.length,
    totalCore: meta.coreSymptoms.length,
    coverageRatio
  };
};

// ---------------------------------------------------------------------------
// Output validation & sanitization — model output is never trusted.
// Note: there is deliberately NO confidence sanitizer; Gemini output must not
// carry probability values at all (see CONFIDENCE_BASIS usage instead).
// ---------------------------------------------------------------------------
const sanitizeText = (value, maxLength) =>
  typeof value === 'string' ? value.trim().replace(/\s+/g, ' ').slice(0, maxLength) : '';

const sanitizeStringArray = (value, maxItems = GEMINI_MAX_ARRAY_ITEMS, maxItemLength = GEMINI_MAX_ARRAY_ITEM_LENGTH) => {
  if (!Array.isArray(value)) return [];
  return value
    .filter((item) => typeof item === 'string' && item.trim().length > 0)
    .slice(0, maxItems)
    .map((item) => item.trim().replace(/\s+/g, ' ').slice(0, maxItemLength));
};

const sanitizeUrgency = (value) => {
  const normalized = typeof value === 'string' ? value.trim().toLowerCase() : '';
  return URGENCY_LEVELS.includes(normalized) ? normalized : 'moderate';
};

// ---------------------------------------------------------------------------
// Confidence labelling — every displayed number is tied to what it actually is
// ---------------------------------------------------------------------------

/**
 * Human-readable interpretation of the confidence attached to an ML/heuristic
 * result, including measured validation metrics when the model provided them.
 */
export const buildConfidenceInterpretation = (mlResult = {}) => {
  const confidence = Number(mlResult.confidence ?? 0);
  const validation = mlResult.validation || null;
  const basis = mlResult.confidence_basis || CONFIDENCE_BASIS.NONE;

  if (basis === CONFIDENCE_BASIS.MODEL_PROBABILITY) {
```

```
    const heldOut = validation
      ? ` Held-out validation (n=${validation.n_test}): accuracy ${validation.accuracy_top1_pct}%, expected calibration error ${validation.ece_10bin} on the
     project's reference dataset.`
      : '';
    return `${confidence}% is the model's top-1 probability from the validated SymptomDiseaseClassifier.${heldOut} It is a model estimate, not a clinical
    probability or a confirmed diagnosis.`;
  }

  if (basis === CONFIDENCE_BASIS.HEURISTIC_OVERLAP) {
    return `${confidence}% is the share of this condition's core symptoms that you reported (heuristic symptom-overlap score) — a preliminary assessment,
    not a probability or a confirmed diagnosis.`;
  }

  if (basis === CONFIDENCE_BASIS.RULE_BASED) {
    return 'This result comes from a symptom-based rule, so no probability is available — treat it as a preliminary assessment only.';
  }

  if (basis === CONFIDENCE_BASIS.GEMINI_ESTIMATE) {
    return `${confidence}% is Gemini's preliminary AI estimate for the reported symptoms — a triage indication only, not a validated clinical probability or
     a confirmed diagnosis.`;
  }

  return 'No probability is available yet — more symptoms are needed before a reliable estimate can be made.';
};

/** Short method label shown next to the combined prediction. */
export const buildConfidenceMethod = (mlResult = {}) => {
  const basis = mlResult.confidence_basis || CONFIDENCE_BASIS.NONE;
  const validation = mlResult.validation || null;

  if (basis === CONFIDENCE_BASIS.MODEL_PROBABILITY) {
    return validation
      ? `SymptomDiseaseClassifier probability (held-out accuracy ${validation.accuracy_top1_pct}%)`
      : 'SymptomDiseaseClassifier probability';
  }
  if (basis === CONFIDENCE_BASIS.HEURISTIC_OVERLAP) {
    return 'Heuristic symptom-overlap score (preliminary — not a calibrated probability)';
  }
  if (basis === CONFIDENCE_BASIS.RULE_BASED) {
    return 'Rule-based preliminary assessment (no probability)';
  }
  if (basis === CONFIDENCE_BASIS.GEMINI_ESTIMATE) {
    return 'Gemini AI preliminary estimate (top-3 differential — not a validated probability)';
  }
  return 'Preliminary symptom assessment (insufficient evidence for a probability)';
};

/**
 * Parses and validates the JSON explanation payload returned by Gemini.
 * The schema contains NO confidence/probability fields by design — Gemini
 * explains the engine's candidate conditions, it never scores or chooses them.
 * Throws a typed GEMINI_INVALID_RESPONSE so callers can fall back to local
 * explanations instead of trusting junk.
 */
export const parseGeminiExplanationResponse = (rawText) => {
  if (typeof rawText !== 'string' || rawText.trim().length === 0) {
    throw new GeminiProviderError(GEMINI_ERROR_CODES.INVALID_RESPONSE, 'Gemini returned an empty response');
  }

  const cleanedText = rawText
    .replace(/^```json\s*/i, '')
    .replace(/^```\s*/i, '')
    .replace(/```\s*$/i, '')
    .trim();

  let parsed;
  try {
    parsed = JSON.parse(cleanedText);
  } catch (_error) {
    throw new GeminiProviderError(GEMINI_ERROR_CODES.INVALID_RESPONSE, 'Gemini returned a malformed JSON payload');
  }

  if (!parsed || typeof parsed !== 'object' || Array.isArray(parsed)) {
    throw new GeminiProviderError(GEMINI_ERROR_CODES.INVALID_RESPONSE, 'Gemini returned an unexpected payload shape');
  }

  const clinicalExplanation = sanitizeText(parsed.clinicalExplanation, GEMINI_MAX_TEXT_LENGTH);
  if (!clinicalExplanation) {
    throw new GeminiProviderError(GEMINI_ERROR_CODES.INVALID_RESPONSE, 'Gemini response did not include a clinical explanation');
  }

  const candidateAnalyses = (Array.isArray(parsed.candidateAnalyses) ? parsed.candidateAnalyses : [])
    .filter((item) => item && typeof item === 'object' && typeof item.condition === 'string' && item.condition.trim().length > 0)
    .slice(0, GEMINI_MAX_TOP_PREDICTIONS)
    .map((item) => ({
      condition: sanitizeText(item.condition, GEMINI_MAX_CONDITION_LENGTH),
      supportingSymptoms: sanitizeStringArray(item.supportingSymptoms),
      missingSymptoms: sanitizeStringArray(item.missingSymptoms),
      summary: sanitizeText(item.summary, GEMINI_MAX_SHORT_TEXT_LENGTH)
    }));

  return {
    clinicalExplanation,
    symptomPatternInsight: sanitizeText(parsed.symptomPatternInsight, GEMINI_MAX_SHORT_TEXT_LENGTH),
    differentialReasoning: sanitizeText(parsed.differentialReasoning, GEMINI_MAX_SHORT_TEXT_LENGTH),
    urgency: sanitizeUrgency(parsed.urgency),
    urgencyRationale: sanitizeText(parsed.urgencyRationale, GEMINI_MAX_SHORT_TEXT_LENGTH),
    selfCareSteps: sanitizeStringArray(parsed.selfCareSteps),
    warningSigns: sanitizeStringArray(parsed.warningSigns),
    whenToSeeDoctor: sanitizeText(parsed.whenToSeeDoctor, GEMINI_MAX_SHORT_TEXT_LENGTH),
    clarificationQuestion: sanitizeText(parsed.clarificationQuestion, GEMINI_MAX_SHORT_TEXT_LENGTH),
    candidateAnalyses
  };
};
```

```
/**
 * Structured-output schema constraining the shape of the model response at the
 * API level (in addition to our own server-side validation). No confidence
 * field exists in this schema on purpose.
 */
const buildExplanationResponseSchema = () => ({
  type: 'OBJECT',
  properties: {
    clinicalExplanation: {
      type: 'STRING',
      description: '2-3 plain-language sentences explaining the candidate conditions against the reported symptoms'
    },
    symptomPatternInsight: {
      type: 'STRING',
      description: '1 sentence on the reported symptom pattern'
    },
    differentialReasoning: {
      type: 'STRING',
      description: '1-2 sentences comparing the candidate conditions'
    },
    urgency: {
      type: 'STRING',
      enum: URGENCY_LEVELS,
      description: 'How urgently professional medical care is needed'
    },
    urgencyRationale: {
      type: 'STRING',
      description: '1 sentence explaining the urgency level'
    },
    selfCareSteps: {
      type: 'ARRAY',
      items: { type: 'STRING' },
      description: 'Up to 5 supportive self-care steps'
    },
    warningSigns: {
      type: 'ARRAY',
      items: { type: 'STRING' },
      description: 'Up to 5 red-flag warning signs that require medical attention'
    },
    whenToSeeDoctor: {
      type: 'STRING',
      description: 'Clear guidance on when to seek professional or emergency care'
    },
    clarificationQuestion: {
      type: 'STRING',
      description: 'Optional question to ask the user when symptom evidence is weak; empty string otherwise'
    },
    candidateAnalyses: {
      type: 'ARRAY',
      items: {
        type: 'OBJECT',
        properties: {
          condition: { type: 'STRING', description: 'One of the provided candidate conditions — never a new one' },
          supportingSymptoms: { type: 'ARRAY', items: { type: 'STRING' }, description: 'Reported symptoms supporting this candidate' },
          missingSymptoms: { type: 'ARRAY', items: { type: 'STRING' }, description: 'Core symptoms of this candidate that were NOT reported' },
          summary: { type: 'STRING', description: '1-2 sentence explanation for this candidate' }
        },
        required: ['condition', 'supportingSymptoms', 'missingSymptoms', 'summary']
      },
      description: 'Per-candidate supporting / missing symptom analysis'
    }
  },
  required: ['clinicalExplanation', 'urgency', 'whenToSeeDoctor', 'candidateAnalyses']
});

const buildExplanationSystemInstruction = () =>
  `You are MedInsight's clinical explanation assistant. You explain candidate conditions produced by a symptom-based model; you never choose or invent a
    diagnosis yourself. ` +
  `You never confirm a clinical diagnosis and never claim certainty, and you never output confidence or probability numbers. ` +
  `If the reported symptoms suggest a medical emergency (for example severe chest pain, difficulty breathing, uncontrolled bleeding, ` +
  `sudden neurological confusion, or signs of anaphylaxis), set urgency to "high" and advise contacting local emergency services immediately. ` +
  `Keep language plain, compassionate, and actionable. Respond with JSON only, matching the requested schema.`
;

// Privacy: only symptom names, duration, and the engine's candidate conditions
// (with their core symptoms) are ever sent to the Gemini API — no names,
// contact details, demographics, notes, or diagnosis history.

/**
 * Builds the candidate-explanation prompt. Each candidate already comes from
 * the symptom engine; supporting/missing core symptoms are computed server-side
 * so Gemini's job is strictly to explain, not to reason from scratch.
 */
const buildExplanationPrompt = ({ reportedNames, unrecognizedNames, duration, severity, candidates }) => {
  const candidateLines = candidates.map((candidate, idx) => {
    const lines = [
      `${idx + 1}. ${candidate.condition} — supporting symptoms reported: ${candidate.supporting.length > 0 ? candidate.supporting.join(', ') : 'none of its
     core symptoms were reported'}.`
    ];
    if (candidate.missing.length > 0) {
      lines.push(`   Core symptoms NOT reported: ${candidate.missing.join(', ')}.`);
    }
    return lines.join('\n');
  });

  return `
CANDIDATE CONDITION EXPLANATION — MedInsight
Reported symptoms: ${reportedNames.join(', ')}
${unrecognizedNames.length > 0 ? `Entries that were not recognized as symptoms: ${unrecognizedNames.join(', ')}\n` : ''}Duration: ${duration ||
    DEFAULT_SYMPTOM_DURATION}
Self-reported discomfort level: ${severity || 'unspecified'}

Candidate conditions from the symptom engine (already computed — do NOT add, replace, or re-rank them):
${candidateLines.join('\n')}
```

```
TASK:
Explain each candidate condition against the reported symptoms:
- clinicalExplanation: 2-3 sentences explaining the overall picture — which reported symptoms support the leading candidate and which of its core symptoms
    are missing.
- For EVERY candidate above, fill candidateAnalyses with its supportingSymptoms, missingSymptoms, and a 1-2 sentence summary. Use only the symptoms listed
    above.
- symptomPatternInsight: 1 sentence on the reported symptom pattern.
- differentialReasoning: 1-2 sentences comparing the candidates and what would distinguish them.
- urgency / urgencyRationale / selfCareSteps / warningSigns / whenToSeeDoctor: preliminary triage guidance (emergency-services advice when urgency is high).
- clarificationQuestion: if the reported symptoms are too weak to distinguish the candidates, ask one short question to collect the most useful missing
    symptom; otherwise use an empty string.

RULES:
- Explain ONLY the listed candidates. Never introduce a new diagnosis or condition name.
- Never output a confidence, probability, or percentage value.
- Frame everything as preliminary — never a confirmed diagnosis.

Return JSON only matching this shape:
{
  "clinicalExplanation": "2-3 sentences",
  "symptomPatternInsight": "1 sentence",
  "differentialReasoning": "1-2 sentences",
  "urgency": "low | moderate | high",
  "urgencyRationale": "1 sentence",
  "selfCareSteps": ["step", "step"],
  "warningSigns": ["red flag", "red flag"],
  "whenToSeeDoctor": "clear guidance, including emergency services advice when urgency is high",
  "clarificationQuestion": "optional short question, or empty string",
  "candidateAnalyses": [
    { "condition": "candidate name", "supportingSymptoms": ["symptom"], "missingSymptoms": ["symptom"], "summary": "1-2 sentences" }
  ]
}
`.trim();
};

/**
 * Builds the geminiInsights sub-document for an explanation of the engine's
 * candidates. The primary condition and confidence are ALWAYS taken from the
 * ML/heuristic result — Gemini contributes explanation text only.
 */
const buildExplanationInsights = ({ parsed, mlResult, model }) => ({
  clinicalExplanation: parsed.clinicalExplanation,
  symptomPatternInsight: parsed.symptomPatternInsight,
  differentialReasoning: parsed.differentialReasoning,
  urgencyAssessment: parsed.urgency,
  urgencyRationale: parsed.urgencyRationale || `Care urgency explained as ${parsed.urgency.toUpperCase()} for the reported symptom pattern.`,
  selfCareSteps: parsed.selfCareSteps,
  warningSignsToWatch: parsed.warningSigns,
  whenToSeeDoctor: parsed.whenToSeeDoctor,
  clarificationQuestion: parsed.clarificationQuestion,
  confidenceInterpretation: buildConfidenceInterpretation(mlResult),
  historicalContext: 'Preliminary AI-assisted explanation of symptom-model candidates.',
  geminiPredictedDisease: mlResult.primary_condition,
  geminiConfidence: mlResult.confidence,
  geminiTopPredictions: mlResult.top_predictions || [],
  candidateAnalyses: parsed.candidateAnalyses,
  accuracyMetrics: null,
  combinedPrediction: {
    predictedDisease: mlResult.primary_condition,
    confidence: mlResult.confidence,
    method: buildConfidenceMethod(mlResult)
  },
  poweredBy: model,
  geminiEnriched: true,
  generatedAt: new Date().toISOString()
});

/**
 * Explains the symptom engine's candidate conditions via Gemini.
 *
 * Contract:
 * - Never predicts or rewrites the condition (mlResult.primary_condition wins).
 * - Never returns a Gemini-generated confidence number.
 * - Limited / clarification / injury results (no candidates) and any Gemini
 *   failure degrade to local rule-based explanations — never to a Gemini guess.
 *
 * @returns {Promise<object>} geminiInsights-compatible sub-document
 */
export const enrichDiagnosisWithGemini = async ({
  symptoms = [],
  mlResult = {},
  duration = DEFAULT_SYMPTOM_DURATION
}) => {
  const localFallback = { ...buildLocalClinicalInsights(symptoms, mlResult), geminiEnriched: false };

  const candidates = Array.isArray(mlResult.top_predictions) ? mlResult.top_predictions : [];
  if (mlResult.needs_clarification || candidates.length === 0) {
    // Nothing to explain (insufficient symptoms / clarification / injury rule)
    return localFallback;
  }

  const { model, isConfigured } = resolveGeminiConfig();
  const client = getGeminiClient();
  if (!client || !isConfigured) {
    return { ...localFallback, enrichmentErrorCode: GEMINI_ERROR_CODES.NOT_CONFIGURED };
  }

  const reportedIds = (mlResult.recognized_symptoms || symptoms).map((s) => String(s).toLowerCase().trim());
  const reportedSet = new Set(reportedIds);
  const reportedNames = reportedIds.map(resolveSymptomName);
  const unrecognizedNames = (mlResult.unrecognized_symptoms || []).map(resolveSymptomName);

  const candidateDetails = candidates.slice(0, GEMINI_MAX_TOP_PREDICTIONS).map((candidate) => {
    const meta = getDiseaseMetaFromCatalog(candidate.disease);
    const core = Array.isArray(meta?.coreSymptoms) ? meta.coreSymptoms : [];
```

```
    const supporting = core.filter((id) => reportedSet.has(String(id).toLowerCase().trim())).map(resolveSymptomName);
    const missing = core.filter((id) => !reportedSet.has(String(id).toLowerCase().trim())).map(resolveSymptomName);
    return { condition: candidate.disease, supporting, missing };
  });

  const prompt = buildExplanationPrompt({
    reportedNames,
    unrecognizedNames,
    duration,
    severity: mlResult.severity || '',
    candidates: candidateDetails
  });

  let timeoutId;
  try {
    const timeoutPromise = new Promise((_, reject) => {
      timeoutId = setTimeout(
        () => reject(new GeminiProviderError(GEMINI_ERROR_CODES.TIMEOUT, 'Gemini request timed out')),
        GEMINI_ENRICHMENT_TIMEOUT_MS
      );
    });

    const callPromise = client.models.generateContent({
      model,
      contents: prompt,
      config: buildGeminiConfig({
        systemInstruction: buildExplanationSystemInstruction(),
        temperature: GEMINI_TEMPERATURE,
        maxOutputTokens: GEMINI_MAX_OUTPUT_TOKENS,
        responseSchema: buildExplanationResponseSchema()
      })
    });

    let response = await Promise.race([callPromise, timeoutPromise]);
    const parsed = parseGeminiExplanationResponse(extractResponseText(response));
    return buildExplanationInsights({ parsed, mlResult, model });
  } catch (error) {
    const code = classifyGeminiError(error);
    // Log the safe code only — never the raw SDK message, API key, or prompt.
    console.warn(`[Gemini Service] Candidate explanation failed (${code}). Using local explanations.`);
    return { ...localFallback, enrichmentErrorCode: code };
  } finally {
    clearTimeout(timeoutId);
  }
};

/**
 * Local (rule-based) clinical explanations used when Gemini is unavailable or
 * when there are no model candidates to explain (limited / clarification /
 * injury results). Purely derived from the project's own disease/symptom
 * taxonomy — no remote call, no invented conditions, no invented confidences:
 * the condition and confidence always come from mlResult.
 */
export const buildLocalClinicalInsights = (symptoms = [], mlResult = {}) => {
  const symptomNames = symptoms.map(resolveSymptomName);
  const lowerSymptoms = symptoms.map((s) => String(s).toLowerCase()).join(' ');

  const primaryDisease = mlResult.primary_condition || 'Preliminary Symptom Evaluation';
  const severity = mlResult.severity || 'moderate';
  let urgency = severity === 'severe' ? 'high' : severity === 'mild' ? 'low' : 'moderate';

  let selfCareSteps = [
    'Ensure adequate hydration and restful recovery.',
    'Monitor symptom trajectory closely over the next 24 to 48 hours.',
    'Avoid strenuous exertion and maintain a comfortable environment.'
  ];
  let warningSigns = [
    'Sudden onset of severe shortness of breath or acute chest pressure',
    'Uncontrolled high fever resistant to supportive measures',
    'Sudden neurological symptoms such as severe dizziness or confusion'
  ];
  let doctorAdvice = 'Consult a qualified physician if symptoms do not improve within 48 hours or if warning signs develop.';
  let explanation = '';

  if (mlResult.needs_clarification) {
    explanation = mlResult.clarification_message ||
      'The provided input could not be matched to known symptoms, so no assessment can be made yet.';
    doctorAdvice = 'Select your symptoms from the symptom list (or rephrase them) and run the assessment again.';
  } else if (INJURY_PATTERN.test(lowerSymptoms)) {
    urgency = 'high';
    explanation =
      `Your reported complaint (${symptomNames.join(', ')}) indicates possible acute physical trauma or suspected bone fracture. ` +
      'This requires radiographic X-ray imaging and clinical immobilization to assess structural alignment and prevent further injury.';
    selfCareSteps = [
      'Do NOT bear weight or put pressure on the injured limb.',
      'Immobilize and support the limb in a comfortable, neutral position.',
      'Apply an ice pack wrapped in a clean cloth for 15 minutes to reduce swelling.',
      'Keep the limb elevated above heart level if possible.'
    ];
    warningSigns = [
      'Visible bone deformity, unnatural angle, or bone piercing skin',
      'Loss of sensation, severe numbness, or cold pale toes/fingers',
      'Inability to move adjacent joints or severe unrelenting pain'
    ];
    doctorAdvice = 'Go to an urgent care clinic or emergency room immediately for X-ray imaging and orthopedic splinting.';
  } else if (primaryDisease === 'Limited symptom assessment') {
    explanation =
      'Your reported symptoms have been recorded, but a single symptom can have many causes. ' +
      'Add any other symptoms you are experiencing for a more useful preliminary assessment.';
    doctorAdvice =
      'Add more symptoms and run the assessment again. If this symptom is severe, worsening, or concerning, ' +
      'contact a healthcare professional; for an emergency, contact local emergency services immediately.';
  } else {
    const meta = getDiseaseMetaFromCatalog(primaryDisease);
    explanation = meta?.description
```

```
      ? `Based on clinical patterns in our reference taxonomy, ${primaryDisease} is associated with your reported symptoms (${symptomNames.join(', ')}).
    ${meta.description}`
      : `Your reported symptoms (${symptomNames.join(', ')}) were assessed against the project's reference taxonomy for ${primaryDisease}.`;
  }

  return {
    clinicalExplanation: explanation,
    symptomPatternInsight: `The observed presentation (${symptomNames.join(', ') || 'no symptoms listed'}) represents a preliminary clinical indication
    requiring targeted evaluation.`,
    differentialReasoning: `Candidate conditions were ranked by symptom overlap with the reference taxonomy; primary focus is directed at
    ${primaryDisease}.`,
    urgencyAssessment: urgency,
    urgencyRationale: `Urgency is evaluated as ${urgency.toUpperCase()} based on presenting symptom severity and physical risk indicators.`,
    selfCareSteps,
    warningSignsToWatch: warningSigns,
    whenToSeeDoctor: doctorAdvice,
    clarificationQuestion: '',
    confidenceInterpretation: buildConfidenceInterpretation(mlResult),
    historicalContext: 'Preliminary rule-based triage assessment.',
    geminiPredictedDisease: primaryDisease,
    geminiConfidence: Number(mlResult.confidence ?? 0),
    geminiTopPredictions: mlResult.top_predictions || [],
    candidateAnalyses: [],
    accuracyMetrics: null,
    combinedPrediction: {
      predictedDisease: primaryDisease,
      confidence: Number(mlResult.confidence ?? 0),
      method: buildConfidenceMethod(mlResult)
    },
    poweredBy: 'MedInsight Local Clinical Rules',
    geminiEnriched: false,
    generatedAt: new Date().toISOString()
  };
};

// ---------------------------------------------------------------------------
// Preliminary Gemini prediction — a real top-3 differential for ANY symptoms
// ---------------------------------------------------------------------------

const ASSESSMENT_LEVELS = ['low', 'moderate', 'high'];
const SEVERITY_BY_LEVEL = { low: 'mild', moderate: 'moderate', high: 'severe' };

const sanitizeProbability = (value) => {
  const numeric = Number(value);
  if (!Number.isFinite(numeric)) return 0;
  return Math.min(100, Math.max(0, Math.round(numeric * 10) / 10));
};

const sanitizeAssessmentLevel = (value) => {
  const normalized = typeof value === 'string' ? value.trim().toLowerCase() : '';
  return ASSESSMENT_LEVELS.includes(normalized) ? normalized : 'moderate';
};

/**
 * Parses and validates the preliminary prediction JSON returned by Gemini.
 * Strips ``` fences, parses inside try/catch, clamps probabilities to 0-100
 * and coerces urgency/impact onto the allow-list. Throws a typed
 * GEMINI_INVALID_RESPONSE so callers can fall back to the symptom engine
 * instead of trusting junk or crashing.
 */
export const parseGeminiPredictionResponse = (rawText) => {
  if (typeof rawText !== 'string' || rawText.trim().length === 0) {
    throw new GeminiProviderError(GEMINI_ERROR_CODES.INVALID_RESPONSE, 'Gemini returned an empty prediction response');
  }

  const cleanedText = rawText
    .replace(/^```json\s*/i, '')
    .replace(/^```\s*/i, '')
    .replace(/```\s*$/i, '')
    .trim();

  let parsed;
  try {
    parsed = JSON.parse(cleanedText);
  } catch (_error) {
    throw new GeminiProviderError(GEMINI_ERROR_CODES.INVALID_RESPONSE, 'Gemini returned a malformed prediction payload');
  }

  if (!parsed || typeof parsed !== 'object' || Array.isArray(parsed)) {
    throw new GeminiProviderError(GEMINI_ERROR_CODES.INVALID_RESPONSE, 'Gemini returned an unexpected prediction shape');
  }
  if (typeof parsed.valid !== 'boolean') {
    throw new GeminiProviderError(GEMINI_ERROR_CODES.INVALID_RESPONSE, 'Gemini prediction did not include a valid flag');
  }

  const topPredictions = (Array.isArray(parsed.top_predictions) ? parsed.top_predictions : [])
    .filter((item) => item && typeof item === 'object' && typeof item.condition === 'string' && item.condition.trim().length > 0)
    .slice(0, GEMINI_MAX_TOP_PREDICTIONS)
    .map((item) => ({
      condition: sanitizeText(item.condition, GEMINI_MAX_CONDITION_LENGTH),
      probability: sanitizeProbability(item.probability),
      reason: sanitizeText(item.reason, GEMINI_MAX_SHORT_TEXT_LENGTH)
    }));

  if (topPredictions.length === 0) {
    throw new GeminiProviderError(GEMINI_ERROR_CODES.INVALID_RESPONSE, 'Gemini prediction did not include any top predictions');
  }

  const explanation = sanitizeText(parsed.explanation, GEMINI_MAX_TEXT_LENGTH);
  if (!explanation) {
    throw new GeminiProviderError(GEMINI_ERROR_CODES.INVALID_RESPONSE, 'Gemini prediction did not include an explanation');
  }

  return {
    valid: parsed.valid,
```

```
    top_predictions: topPredictions,
    urgency: sanitizeAssessmentLevel(parsed.urgency),
    impact: sanitizeAssessmentLevel(parsed.impact),
    explanation,
    recommended_actions: sanitizeStringArray(parsed.recommended_actions)
  };
};

/** Structured-output schema constraining the prediction JSON at the API level. */
const buildPredictionResponseSchema = () => ({
  type: 'OBJECT',
  properties: {
    valid: { type: 'BOOLEAN', description: 'False only when the input is random letters with no plausible health meaning' },
    top_predictions: {
      type: 'ARRAY',
      items: {
        type: 'OBJECT',
        properties: {
          condition: { type: 'STRING', description: 'Most likely condition name' },
          probability: { type: 'NUMBER', description: 'Estimated probability from 0 to 100; keep values low when symptom evidence is thin' },
          reason: { type: 'STRING', description: 'One sentence linking this condition to the reported symptoms' }
        },
        required: ['condition', 'probability', 'reason']
      },
      description: 'Exactly the top 3 most likely conditions, most likely first'
    },
    urgency: { type: 'STRING', enum: ['Low', 'Moderate', 'High'], description: 'How urgently professional medical care is needed' },
    impact: { type: 'STRING', enum: ['Low', 'Moderate', 'High'], description: 'Expected impact on daily life' },
    explanation: { type: 'STRING', description: '2-3 sentence short clinical explanation of the overall picture' },
    recommended_actions: { type: 'ARRAY', items: { type: 'STRING' }, description: '3-5 concrete self-care and next-step actions' }
  },
  required: ['valid', 'top_predictions', 'urgency', 'impact', 'explanation', 'recommended_actions']
});

const buildPredictionSystemInstruction = () =>
  `You are MedInsight's preliminary triage assistant. You provide careful, preliminary assessments of reported symptoms — never confirmed clinical
    diagnoses. ` +
  `You keep language plain, compassionate, and actionable, you stay conservative with urgency, and safety comes first: when reported symptoms suggest a
    medical emergency ` +
  `(severe chest pain, difficulty breathing, uncontrolled bleeding, sudden neurological confusion), set urgency to "High" and advise contacting local
    emergency services immediately. ` +
  `With few symptoms you stay humble: probabilities stay low and common conditions lead the list. ` +
  `You always respond with JSON only, matching the requested schema.`;

/**
 * Privacy: only the reported symptom entries (and their catalog display
 * names), duration, and self-reported severity are ever sent to the Gemini
 * API — no names, contact details, demographics, notes, or history.
 */
const buildPredictionPrompt = ({ submitted, duration, severity }) => {
  const symptomLines = submitted.map((entry) => {
    const resolvedId = resolveSymptomId(entry);
    return resolvedId
      ? `- "${entry}" → known symptom "${resolveSymptomName(resolvedId)}"`
      : `- "${entry}" (not in the reference taxonomy — interpret it as best you can)`;
  }).join('\n');

  return `
PRELIMINARY SYMPTOM ASSESSMENT — MedInsight
Reported symptoms (${submitted.length}):
${symptomLines}
Duration: ${duration || DEFAULT_SYMPTOM_DURATION}
Self-reported discomfort level: ${severity || 'unspecified'}

TASK:
Based only on the reported symptoms, return your best preliminary differential assessment:
- top_predictions: exactly the TOP 3 most likely conditions, each with condition (name), probability (0-100) and reason (one sentence tying it to the
    reported symptoms).
- valid: false only if every entry is random letters with no plausible health meaning; health terms, disease names, injuries, and misspellings are valid.
- With few symptoms (1-2) or unusual/unrecognized entries, keep every probability LOW and prefer COMMON conditions; state the uncertainty in each reason.
- urgency: how urgently professional care is needed (Low | Moderate | High).
- impact: expected impact on daily life (Low | Moderate | High).
- explanation: 2-3 sentence short clinical explanation of the overall picture.
- recommended_actions: 3-5 concrete self-care / next-step actions.

RULES:
- This is preliminary triage guidance, NOT a clinical diagnosis; never claim certainty.
- Return ONLY valid JSON matching this shape (no extra text):
{
  "valid": true,
  "top_predictions": [ { "condition": "...", "probability": 0-100, "reason": "..." } ],
  "urgency": "Low | Moderate | High",
  "impact": "Low | Moderate | High",
  "explanation": "short clinical explanation",
  "recommended_actions": ["...", "..."]
}
`.trim();
};

/**
 * Maps a sanitized Gemini prediction onto the exact field shapes the rest of
 * the pipeline (controllers, persistence, UI) already consumes: an
 * mlResult-style prediction plus a geminiInsights-compatible insights
 * document. The mandatory medical disclaimer travels with every result.
 */
export const buildGeminiPredictionResult = ({ parsed, symptoms = [] }) => {
  const submitted = (Array.isArray(symptoms) ? symptoms : [])
    .filter((entry) => typeof entry === 'string' && entry.trim().length > 0)
    .map((entry) => entry.trim());

  const recognized = [];
  const unrecognized = [];
  for (const entry of submitted) {
    const resolvedId = resolveSymptomId(entry);
    if (resolvedId) {
```

```
      if (!recognized.includes(resolvedId)) recognized.push(resolvedId);
    } else {
      const lowerEntry = entry.toLowerCase();
      if (!unrecognized.includes(lowerEntry)) unrecognized.push(lowerEntry);
    }
  }

  // Severity (the UI's "impact" label) reflects the WORSE of urgency/impact.
  const severityRank = { low: 0, moderate: 1, high: 2 };
  const worstLevel = severityRank[parsed.urgency] >= severityRank[parsed.impact] ? parsed.urgency : parsed.impact;
  const severity = SEVERITY_BY_LEVEL[worstLevel] || 'moderate';

  const topPredictions = parsed.top_predictions.map((candidate) => ({
    disease: candidate.condition,
    condition: candidate.condition,
    confidence: candidate.probability,
    severity,
    description: candidate.reason,
    reason: candidate.reason
  }));
  const primary = topPredictions[0];

  const prediction = {
    success: true,
    primary_condition: primary.condition,
    confidence: primary.confidence,
    confidence_basis: CONFIDENCE_BASIS.GEMINI_ESTIMATE,
    severity,
    description: parsed.explanation,
    recommendation: parsed.recommended_actions.join(' '),
    top_predictions: topPredictions,
    recognized_symptoms: recognized,
    unrecognized_symptoms: unrecognized,
    disclaimer: MANDATORY_DISCLAIMER,
    isFallback: false
  };

  const insights = {
    clinicalExplanation: parsed.explanation,
    symptomPatternInsight: `Reported pattern (${submitted.map((entry) => resolveSymptomName(resolveSymptomId(entry) || entry.toLowerCase())).join(', ')})
    assessed preliminarily by Gemini.`,
    differentialReasoning: topPredictions.map((candidate) => `${candidate.condition}: ${candidate.reason}`).join(' | '),
    urgencyAssessment: parsed.urgency,
    urgencyRationale: `Care urgency assessed as ${parsed.urgency.toUpperCase()} and impact as ${parsed.impact.toUpperCase()} for the reported symptom
    pattern.`,
    selfCareSteps: parsed.recommended_actions,
    warningSignsToWatch: [],
    whenToSeeDoctor: parsed.recommended_actions.join(' '),
    clarificationQuestion: '',
    confidenceInterpretation: buildConfidenceInterpretation(prediction),
    historicalContext: 'Preliminary Gemini AI estimate of the reported symptom pattern.',
    geminiPredictedDisease: primary.condition,
    geminiConfidence: primary.confidence,
    geminiTopPredictions: topPredictions,
    candidateAnalyses: topPredictions.map((candidate) => ({
      condition: candidate.condition,
      supportingSymptoms: recognized.map((id) => resolveSymptomName(id)),
      missingSymptoms: [],
      summary: candidate.reason
    })),
    accuracyMetrics: null,
    combinedPrediction: {
      predictedDisease: primary.condition,
      confidence: primary.confidence,
      method: 'Gemini AI preliminary estimate (top-3 differential — not a validated probability)'
    },
    poweredBy: 'Gemini',
    geminiEnriched: true,
    generatedAt: new Date().toISOString()
  };

  return { prediction, insights };
};

/**
 * Generates a preliminary top-3 differential via Gemini for 1+ submitted
 * symptoms. Returns null when Gemini is not configured (caller falls back
 * silently); throws a typed GeminiProviderError on any call/parse failure so
 * runPrediction can fall back to the symptom-grounded engine without
 * crashing.
 */
export const generateGeminiPrediction = async ({
  symptoms = [],
  duration = DEFAULT_SYMPTOM_DURATION,
  severity = ''
}) => {
  const submitted = (Array.isArray(symptoms) ? symptoms : [])
    .filter((entry) => typeof entry === 'string' && entry.trim().length > 0)
    .map((entry) => entry.trim());
  if (submitted.length === 0) return null;

  const { model, isConfigured } = resolveGeminiConfig();
  const client = getGeminiClient();
  if (!client || !isConfigured) return null;

  const prompt = buildPredictionPrompt({ submitted, duration, severity });

  let timeoutId;
  try {
    let lastError;
    for (let attempt = 0; attempt < 2; attempt += 1) {
      try {
        const timeoutPromise = new Promise((_, reject) => {
          timeoutId = setTimeout(
            () => reject(new GeminiProviderError(GEMINI_ERROR_CODES.TIMEOUT, 'Gemini prediction request timed out')),
```

```
          GEMINI_PREDICTION_TIMEOUT_MS
        );
      });
      const callPromise = client.models.generateContent({
        model,
        contents: attempt === 0 ? prompt : `${prompt}\n\nReturn valid:true for any plausible symptom, injury, or disease name. valid:false is only for
    random letters.`,
        config: buildGeminiConfig({
          systemInstruction: buildPredictionSystemInstruction(),
          temperature: GEMINI_TEMPERATURE,
          maxOutputTokens: GEMINI_MAX_OUTPUT_TOKENS,
          responseMimeType: 'application/json',
          responseSchema: buildPredictionResponseSchema()
        })
      });
      const response = await Promise.race([callPromise, timeoutPromise]);
      const parsed = parseGeminiPredictionResponse(extractResponseText(response));
      if (parsed.valid === false) {
        throw new GeminiProviderError(GEMINI_ERROR_CODES.INVALID_RESPONSE, 'Gemini identified non-health random input');
      }
      return buildGeminiPredictionResult({ parsed, symptoms: submitted });
    } catch (error) {
      lastError = error;
      if (attempt === 1) throw error;
    }
  }
  throw lastError;
 } finally {
  clearTimeout(timeoutId);
 }
};

const CHAT_DISCLAIMER = 'This is not a medical diagnosis; please consult a doctor.';

const buildChatFallback = () => `Rest, drink fluids, and contact a doctor if symptoms worsen or do not improve.\n\n${CHAT_DISCLAIMER}`;

/** Conversational health guidance. Retries once and always returns a safe reply. */
export const generateGeminiChatReply = async ({
  userMessage,
  detectedSymptoms = [],
  conversationHistory = [],
  isEmergency = false
}) => {
  const fallback = buildChatFallback();
  if (isEmergency) return fallback;
  try {
    const { model, isConfigured } = resolveGeminiConfig();
    const client = getGeminiClient();
    if (!client || !isConfigured) return fallback;

    const symptomNames = detectedSymptoms.map(resolveSymptomName);
    const historyContents = conversationHistory.slice(-GEMINI_CHAT_HISTORY_TURNS).map((turn) => ({
      role: turn.role === 'assistant' || turn.role === 'bot' ? 'model' : 'user',
      parts: [{ text: turn.content || turn.text || '' }]
    }));
    const prompt = `User message: "${userMessage}". Detected symptoms: ${symptomNames.join(', ') || 'None yet'}.\nFor any disease or symptom, include: a
    short explanation; simple home care; relevant diet/lifestyle tips; warning signs for a doctor or emergency care; and which type of doctor to consult. If
     this is not health-related, politely say you can only help with health questions. Do not diagnose. End with a short note that this is not a medical
    diagnosis and to consult a doctor.`;
    const systemInstruction = 'You are a careful health assistant. Give general educational guidance only. Do not prescribe medicine names or doses. Keep
    the answer short, clear and in simple language. Always reply in simple English, whatever language the user writes in. Understand Bengali and Banglish
    input, but answer only in English.';

    for (let attempt = 0; attempt < 2; attempt += 1) {
      let timeoutId;
      try {
        const timeoutPromise = new Promise((_, reject) => {
          timeoutId = setTimeout(
            () => reject(new GeminiProviderError(GEMINI_ERROR_CODES.TIMEOUT, 'Gemini chat request timed out')),
            GEMINI_CHAT_TIMEOUT_MS
          );
        });
        const callPromise = client.models.generateContent({
          model,
          contents: [...historyContents, { role: 'user', parts: [{ text: prompt }] }],
          config: buildGeminiConfig({
            systemInstruction,
            temperature: GEMINI_CHAT_TEMPERATURE,
            maxOutputTokens: GEMINI_CHAT_MAX_OUTPUT_TOKENS
          })
        });
        const response = await Promise.race([callPromise, timeoutPromise]);
        const reply = extractResponseText(response).trim();
        if (reply) {
          const disclaimerPresent = /not a medical diagnosis/i.test(reply);
          const disclaimer = disclaimerPresent ? '' : `\n\n${CHAT_DISCLAIMER}`;
          return `${reply}${disclaimer}`;
        }
      } catch (_error) {
        // Retry once; preserve a safe generic answer after the second failure.
      } finally {
        clearTimeout(timeoutId);
      }
    }
    return fallback;
  } catch (_error) {
    return fallback;
  } finally {
    // No caller should receive an empty reply, even if provider setup fails.
  }
};

export default {
  GeminiProviderError,
  classifyGeminiError,
```

```
  resolveGeminiConfig,
  isGeminiConfigured,
  parseGeminiExplanationResponse,
  enrichDiagnosisWithGemini,
  generateGeminiChatReply,
  resolveSymptomName,
  getDiseaseMetaFromCatalog,
  computeSymptomCoverage,
  buildConfidenceInterpretation,
  buildConfidenceMethod,
  buildLocalClinicalInsights
};
```

backend/services/mlService.js

Node symptom normalization, Python service bridge, and local heuristic fallback.

```
import fs from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';
import {
  CONFIDENCE_BASIS,
  SYMPTOMS_NOT_RECOGNIZED_CODE,
  SYMPTOMS_NOT_RECOGNIZED_MESSAGE,
  NO_VALID_SYMPTOMS_MESSAGE,
  SYMPTOMS_NO_MATCH_MESSAGE
} from '../config/constants.js';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const ML_SERVICE_URL = process.env.ML_SERVICE_URL || 'http://localhost:8000';
const ML_TIMEOUT_MS = 3000;

// Kept in sync with SYMPTOM_ALIASES in ml-service/app.py so the Node bridge,
// the heuristic fallback and the Python service resolve synonyms identically.
const SYMPTOM_ALIASES = {
  fever: 'high_fever',
  'high fever': 'high_fever',
  'low fever': 'mild_fever',
  'mild fever': 'mild_fever',
  flu: 'high_fever',
  temperature: 'high_fever',
  'body fever': 'high_fever',
  feverish: 'high_fever',
  cough: 'cough',
  coughing: 'cough',
  sneeze: 'continuous_sneezing',
  sneezing: 'continuous_sneezing',
  nausea: 'nausea',
  nauseated: 'nausea',
  vomit: 'vomiting',
  vomiting: 'vomiting',
  diarrhea: 'diarrhoea',
  diarrhoea: 'diarrhoea',
  loose_motion: 'diarrhoea',
  loose_stools: 'diarrhoea',
  dizzy: 'dizziness',
  tired: 'fatigue',
  tiredness: 'fatigue',
  exhausted: 'fatigue',
  sleepy: 'fatigue',
  weak: 'fatigue',
  weakness: 'fatigue',
  no_energy: 'fatigue',
  low_energy: 'fatigue',
  cold: 'chills',
  shivering: 'chills',
  rash: 'skin_rash',
  itchy: 'itching',
  itch: 'itching',
  breathless: 'breathlessness',
  sob: 'breathlessness',
  shortness_of_breath: 'breathlessness',
  difficulty_breathing: 'breathlessness',
  yellow: 'yellowish_skin',
  jaundice: 'yellowish_skin',
  runny: 'runny_nose',
  blocked_nose: 'runny_nose',
  stuffy_nose: 'runny_nose',
  nasal_congestion: 'runny_nose',
  constipated: 'constipation',
  sweat: 'sweating',
  backache: 'back_pain',
  belly: 'stomach_pain',
  tummy: 'stomach_pain',
  stomach_ache: 'stomach_pain',
  stomach_aches: 'stomach_pain',
  tummy_pain: 'stomach_pain',
  belly_ache: 'stomach_pain',
  bloating: 'abdominal_pain',
  palpitation: 'palpitations',
  anxious: 'anxiety',
  depressed: 'depression',
  sore_throat: 'throat_irritation',
  throat_pain: 'throat_irritation',
  head_ache: 'headache',
  headaches: 'headache',
  head_pain: 'headache',
  chest_tightness: 'chest_pain',
  chest_pressure: 'chest_pain',
  body_aches: 'muscle_pain',
```

```
  body_pain: 'muscle_pain',
  body_ache: 'muscle_pain',
  aching_body: 'muscle_pain',
  leg_pain: 'joint_pain',
  leg_break: 'joint_pain',
  leg_ache: 'joint_pain',
  broken_leg: 'joint_pain',
  knee_pain: 'joint_pain',
  ankle_pain: 'joint_pain',
  arm_pain: 'joint_pain',
  shoulder_pain: 'joint_pain',
  hip_pain: 'joint_pain',
  fracture: 'joint_pain',
  sprain: 'joint_pain',
  stomach_problem: 'stomach_pain',
  chest_problem: 'chest_pain',
  back_problem: 'back_pain',
  throat_problem: 'throat_irritation',
  eye_pain: 'pain_behind_the_eyes',
  ear_pain: 'ear_ache'
};

export const MANDATORY_DISCLAIMER =
  'MedInsight provides preliminary, AI-assisted symptom triage guidance for informational purposes only. ' +
  'It is not a clinical medical diagnosis. In case of emergency or severe discomfort, please consult a certified doctor ' +
  'or contact your local emergency services immediately.';

// Reported orthopedic trauma wording gets a dedicated urgent-care pathway
// instead of a disease label (evidence-based rule on reported symptoms).
// Shared with geminiService.js for explanation content.
export const INJURY_PATTERN = /(fractur|broken|break|dislocat|sprain|tibia|fibula|femur|bone)/i;

// In-memory cache for symptoms and disease definitions
let cachedSymptoms = [];
let cachedDiseases = {};
let cachedCatalogIds = null;

// Load local reference datasets for caching and heuristic fallback
try {
  const symptomsPath = path.resolve(__dirname, '../data/symptoms.json');
  const diseasesPath = path.resolve(__dirname, '../data/diseases.json');

  if (fs.existsSync(symptomsPath)) {
    cachedSymptoms = JSON.parse(fs.readFileSync(symptomsPath, 'utf8'));
  }
  if (fs.existsSync(diseasesPath)) {
    cachedDiseases = JSON.parse(fs.readFileSync(diseasesPath, 'utf8'));
  }
} catch (err) {
  console.warn('[ML Service Bridge] Warning: Could not pre-load fallback dataset files:', err.message);
}

const normalizeSymptom = (raw) => {
  const cleaned = raw
    .trim()
    .toLowerCase()
    .replace(/[^a-z0-9]+/g, '_')
    .replace(/_+/g, '_')
    .replace(/^_|_$/g, '');
  return SYMPTOM_ALIASES[cleaned] || cleaned;
};

/**
 * Returns master list of symptoms (cached in Node memory)
 */
export const getSymptomsList = async () => {
  if (cachedSymptoms.length > 0) {
    return cachedSymptoms;
  }

  // Fallback to querying ML service
  try {
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), ML_TIMEOUT_MS);
    const response = await fetch(`${ML_SERVICE_URL}/symptoms`, { signal: controller.signal });
    clearTimeout(timeoutId);

    if (response.ok) {
      cachedSymptoms = await response.json();
      return cachedSymptoms;
    }
  } catch (_e) {
    // Return empty array if both fail
  }
  return [];
};

// ---------------------------------------------------------------------------
// Typo-tolerant catalog resolution (mirrors match_feature in ml-service/app.py)
// ---------------------------------------------------------------------------

const levenshtein = (a, b) => {
  if (a === b) return 0;
  if (!a) return b.length;
  if (!b) return a.length;
  let prev = Array.from({ length: b.length + 1 }, (_, i) => i);
  for (let i = 1; i <= a.length; i++) {
    const cur = [i];
    for (let j = 1; j <= b.length; j++) {
      cur[j] = Math.min(prev[j] + 1, cur[j - 1] + 1, prev[j - 1] + (a[i - 1] === b[j - 1] ? 0 : 1));
    }
    prev = cur;
  }
  return prev[b.length];
};
```

```
const getCatalogIds = () => {
  if (cachedCatalogIds) return cachedCatalogIds;
  const ids = new Set();
  for (const symptom of cachedSymptoms) {
    if (symptom && typeof symptom.id === 'string') ids.add(symptom.id);
  }
  // Guarantee every symptom referenced by a disease profile is resolvable
  for (const meta of Object.values(cachedDiseases)) {
    for (const core of meta?.coreSymptoms || []) ids.add(String(core).toLowerCase().trim());
  }
  cachedCatalogIds = [...ids];
  return cachedCatalogIds;
};

/**
 * Resolve one normalized input to a single catalog symptom id.
 * Tier 1 exact → Tier 2 unique whole word → Tier 3 unique readable containment
 * → Tier 4 edit-distance fuzzy. Ambiguous or unrelated input returns null so
 * callers can ask the user to clarify instead of guessing.
 */
export const resolveSymptomId = (symptom) => {
  if (!symptom) return null;
  const ids = getCatalogIds();

  if (ids.includes(symptom)) return symptom;

  const words = symptom.split('_').filter(Boolean);
  if (words.length > 0) {
    const wordHits = new Set();
    for (const id of ids) {
      if (id.split('_').some((w) => words.includes(w))) wordHits.add(id);
    }
    if (wordHits.size === 1) return [...wordHits][0];
  }

  const readable = symptom.replace(/_/g, ' ');
  const tier3 = ids.filter((id) => {
    const idReadable = id.replace(/_/g, ' ');
    return readable.includes(idReadable) || idReadable.includes(readable);
  });
  if (tier3.length === 1) return tier3[0];

  if (symptom.length < 4) return null;
  const maxDistance = symptom.length >= 6 ? 2 : 1;
  let best = null;
  let bestDistance = maxDistance + 1;
  for (const id of ids) {
    const distance = levenshtein(symptom, id);
    if (distance < bestDistance) {
      best = id;
      bestDistance = distance;
    }
  }
  return bestDistance <= maxDistance ? best : null;
};

const resolveSymptomSet = (normalizedSymptoms) => {
  const recognized = [];
  const unrecognized = [];
  for (const symptom of normalizedSymptoms) {
    const match = resolveSymptomId(symptom);
    if (match) {
      if (!recognized.includes(match)) recognized.push(match);
    } else if (!unrecognized.includes(symptom)) {
      unrecognized.push(symptom);
    }
  }
  return { recognized, unrecognized };
};

/** Nearest catalog symptom ids for unrecognized input (clarification help). */
const nearestCatalogIds = (inputs, limit = 3) => {
  const ids = getCatalogIds();
  const suggestions = [];
  for (const input of inputs) {
    const scored = ids
      .map((id) => ({ id, distance: levenshtein(input, id) }))
      .sort((a, b) => a.distance - b.distance);
    for (const { id } of scored.slice(0, limit)) {
      if (!suggestions.includes(id)) suggestions.push(id);
    }
  }
  return suggestions.slice(0, 5);
};

/**
 * Clarification result: returned instead of a disease whenever the input
 * cannot be grounded in the symptom catalog. Never carries a condition.
 */
const buildClarificationResult = ({ message, recognized = [], unrecognized = [], isFallback = false, suggestions = [] }) => ({
  success: false,
  needs_clarification: true,
  code: SYMPTOMS_NOT_RECOGNIZED_CODE,
  clarification_message: message,
  primary_condition: '',
  confidence: 0,
  confidence_basis: CONFIDENCE_BASIS.NONE,
  severity: 'moderate',
  description: '',
  top_predictions: [],
  recognized_symptoms: recognized,
  unrecognized_symptoms: unrecognized,
  suggestions,
  disclaimer: MANDATORY_DISCLAIMER,
```

```
  isFallback
});

/**
 * Heuristic fallback triage engine used when the ML service times out or is
 * unreachable. Scores candidate conditions ONLY by symptom overlap between the
 * recognized symptoms and each disease profile's core symptoms — there is no
 * hardcoded default condition, so unrelated input yields a clarification
 * request instead of the same disease every time.
 *
 * The returned "confidence" is a symptom-coverage score labelled
 * `heuristic_overlap` — a preliminary assessment, never a probability.
 */
const runFallbackTriage = (recognized, unrecognized) => {
  const symptomSet = new Set(recognized);
  const diseaseScores = [];

  for (const [disease, meta] of Object.entries(cachedDiseases)) {
    const coreSymptoms = (meta.coreSymptoms || []).map((s) => s.toLowerCase().trim());
    const matchCount = coreSymptoms.filter((s) => symptomSet.has(s)).length;

    if (matchCount > 0) {
      const coreLength = Math.max(coreSymptoms.length, 1);
      diseaseScores.push({
        disease,
        matchCount,
        matchRatio: matchCount / coreLength,
        severity: meta.severity || 'moderate',
        description: meta.description || 'A medical condition characterized by the recorded symptoms.'
      });
    }
  }

  if (diseaseScores.length === 0) {
    // Symptom strings resolved, but no disease profile matches: ask for
    // different/additional symptoms rather than inventing a condition.
    return buildClarificationResult({
      message: SYMPTOMS_NO_MATCH_MESSAGE,
      recognized,
      unrecognized,
      isFallback: true
    });
  }

  // Sort by match count descending, then by coverage descending
  diseaseScores.sort((a, b) => b.matchCount - a.matchCount || b.matchRatio - a.matchRatio);

  const topPredictions = diseaseScores.slice(0, 3).map((item) => ({
    disease: item.disease,
    confidence: Math.round(item.matchRatio * 1000) / 10,
    severity: item.severity,
    description: item.description
  }));

  const primary = topPredictions[0];
  const isLowConfidence = primary.confidence < 30;

  const result = {
    success: true,
    primary_condition: primary.disease,
    confidence: primary.confidence,
    confidence_basis: CONFIDENCE_BASIS.HEURISTIC_OVERLAP,
    severity: primary.severity,
    description: primary.description,
    top_predictions: topPredictions,
    recognized_symptoms: recognized,
    unrecognized_symptoms: unrecognized,
    disclaimer: MANDATORY_DISCLAIMER,
    isFallback: true
  };

  if (isLowConfidence) {
    result.low_confidence = true;
    result.low_confidence_message =
      'Only a small portion of this condition's core symptoms was reported — ' +
      'this is a preliminary estimate. Add more symptoms for a more useful assessment.';
  }

  return result;
};

/**
 * Predicts disease from the user-provided symptoms array.
 *
 * Flow: normalize (synonyms) → resolve against the symptom catalog (exact /
 * unique word / containment / typo-fuzzy) →
 *   - 0 recognized  → clarification request (no condition returned)
 *   - ≥1 recognized → Python ML service model prediction over the resolved
 *     symptom subset (single symptoms included); a 422 from the healthy
 *     service also maps to a clarification request (never a disease).
 * Only when the ML service itself is unreachable does the local heuristic
 * overlap engine run — and it too returns a clarification when nothing matches.
 *
 * @param {string[]} rawSymptoms
 * @returns {Promise<object>} prediction or clarification result
 */
export const predictDiagnosis = async (rawSymptoms) => {
  const cleanedInput = (Array.isArray(rawSymptoms) ? rawSymptoms : [])
    .filter((s) => typeof s === 'string' && s.trim().length > 0)
    .map((s) => s.trim());
  const normalized = cleanedInput.map(normalizeSymptom);

  if (normalized.length === 0) {
    return buildClarificationResult({ message: NO_VALID_SYMPTOMS_MESSAGE });
  }
```

```
  // Reported orthopedic trauma → dedicated urgent-care pathway (rule applied
  // to the user's own reported complaint, never a disease-name echo).
  const injuryText = cleanedInput.join(' ').toLowerCase();
  if (INJURY_PATTERN.test(injuryText)) {
    const { recognized, unrecognized } = resolveSymptomSet(normalized);
    return {
      success: true,
      primary_condition: 'Suspected Bone Fracture / Musculoskeletal Trauma',
      confidence: 0,
      confidence_basis: CONFIDENCE_BASIS.RULE_BASED,
      severity: 'severe',
      description:
        `Your reported complaint (${cleanedInput.join(', ')}) indicates possible acute physical trauma. ` +
        'This requires clinical evaluation and imaging (X-ray) to assess structural alignment. ' +
        'This is a preliminary injury triage assessment, not a confirmed diagnosis.',
      recommendation:
        'Seek urgent care or emergency evaluation immediately for imaging. Do not bear weight on or ' +
        'move the injured area, and keep it immobilized while arranging care.',
      top_predictions: [],
      recognized_symptoms: recognized,
      unrecognized_symptoms: unrecognized,
      disclaimer: MANDATORY_DISCLAIMER,
      isFallback: false
    };
  }

  const { recognized, unrecognized } = resolveSymptomSet(normalized);

  if (recognized.length === 0) {
    return buildClarificationResult({
      message: SYMPTOMS_NOT_RECOGNIZED_MESSAGE,
      unrecognized,
      suggestions: nearestCatalogIds(unrecognized)
    });
  }

  // Every recognized symptom count proceeds to the engine — including a
  // single symptom (the "limited assessment" rule was removed; only
  // unrecognized/empty input asks the user to clarify above).
  try {
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), ML_TIMEOUT_MS);

    const response = await fetch(`${ML_SERVICE_URL}/predict`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ symptoms: normalized }),
      signal: controller.signal
    });

    clearTimeout(timeoutId);

    if (response.ok) {
      const data = await response.json();
      return {
        ...data,
        confidence_basis: data.confidence_basis || CONFIDENCE_BASIS.MODEL_PROBABILITY,
        isFallback: false
      };
    }

    if (response.status === 422) {
      // The ML service is healthy but could not recognize the input:
      // ask the user to clarify instead of inventing a condition.
      let body = null;
      try {
        body = await response.json();
      } catch (_parseError) {
        body = null;
      }
      return buildClarificationResult({
        message: body?.detail || SYMPTOMS_NOT_RECOGNIZED_MESSAGE,
        recognized: body?.recognized_symptoms || [],
        unrecognized: body?.unrecognized_symptoms || unrecognized,
        suggestions: Array.isArray(body?.suggestions) ? body.suggestions : []
      });
    }

    console.warn(`[ML Service Bridge] ML service returned status ${response.status}. Engaging fallback.`);
  } catch (err) {
    console.warn(`[ML Service Bridge] ML service call failed (${err.name === 'AbortError' ? 'Timeout 3000ms' : err.message}). Engaging fallback.`);
  }

  // Gracefully fallback to the internal heuristic overlap engine
  return runFallbackTriage(recognized, unrecognized);
};

export default {
  getSymptomsList,
  predictDiagnosis,
  resolveSymptomId,
  MANDATORY_DISCLAIMER
};
```

backend/services/chatEngine.js

Symptom extraction, emergency flags, conversational branches, and response structure.

```
 import fs from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';
```

```
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// Load symptoms catalog & disease definitions
let symptomsData = [];
let diseasesData = {};

try {
  const symptomsPath = path.resolve(__dirname, '../data/symptoms.json');
  const diseasesPath = path.resolve(__dirname, '../data/diseases.json');
  if (fs.existsSync(symptomsPath)) {
    symptomsData = JSON.parse(fs.readFileSync(symptomsPath, 'utf8'));
  }
  if (fs.existsSync(diseasesPath)) {
    diseasesData = JSON.parse(fs.readFileSync(diseasesPath, 'utf8'));
  }
} catch (err) {
  console.warn('[Chat Engine] Warning: Failed to load taxonomy for chat engine:', err.message);
}

// Comprehensive synonym mapping to multi-hot symptom IDs
const SYMPTOM_SYNONYMS = {
  high_fever: ['fever', 'high fever', 'high temperature', 'burning up', 'hot forehead', 'febrile', 'pyrexia'],
  mild_fever: ['mild fever', 'low grade fever', 'warm forehead', 'slight temperature'],
  headache: ['headache', 'head ache', 'migraine', 'throbbing head', 'head hurts', 'pain in head'],
  cough: ['cough', 'coughing', 'dry cough', 'wet cough', 'hacking cough', 'hacking'],
  fatigue: ['fatigue', 'tired', 'tiredness', 'exhausted', 'exhaustion', 'worn out', 'weak', 'weakness', 'low energy', 'lethargic', 'sluggish'],
  chills: ['chills', 'feeling cold', 'shivering', 'goosebumps', 'cold sweats', 'teeth chattering'],
  nausea: ['nausea', 'nauseous', 'feel sick', 'sick to stomach', 'queasy'],
  vomiting: ['vomit', 'vomiting', 'throwing up', 'puking', 'threw up'],
  breathlessness: ['breathless', 'shortness of breath', 'short of breath', 'hard to breathe', 'difficulty breathing', 'wheezing', 'can’t breathe',
    'cant breathe', 'gasping'],
  chest_pain: ['chest pain', 'tight chest', 'chest pressure', 'chest ache', 'pain in chest', 'chest discomfort'],
  throat_irritation: ['sore throat', 'scratchy throat', 'throat hurts', 'throat irritation', 'throat pain', 'swollen throat'],
  stomach_pain: ['stomach pain', 'belly ache', 'abdominal pain', 'tummy ache', 'gut pain', 'cramps in stomach', 'pain in stomach'],
  joint_pain: ['joint pain', 'aching joints', 'joints hurt', 'pain in joints', 'swollen joints'],
  muscle_pain: ['muscle pain', 'body ache', 'muscle ache', 'aching muscles', 'myalgia'],
  dizziness: ['dizziness', 'dizzy', 'lightheaded', 'feeling faint', 'spinning head', 'vertigo'],
  skin_rash: ['skin rash', 'rash', 'itchy rash', 'red spots', 'breakout', 'hives'],
  itching: ['itching', 'itchy', 'scratching', 'pruritus'],
  continuous_sneezing: ['sneezing', 'sneezes', 'continuous sneezing', 'frequent sneezing'],
  diarrhoea: ['diarrhea', 'diarrhoea', 'loose stools', 'watery stool'],
  acidity: ['acidity', 'acid reflux', 'heartburn', 'sour stomach', 'burning chest'],
  loss_of_appetite: ['loss of appetite', 'no appetite', 'not hungry', 'don’t want to eat', 'cant eat'],
  sweating: ['sweating', 'excessive sweating', 'night sweats', 'perspiration'],
  back_pain: ['back pain', 'backache', 'aching back', 'lower back pain'],
  neck_pain: ['neck pain', 'stiff neck', 'neck stiffness', 'pain in neck'],
  yellowish_skin: ['yellow skin', 'yellowish skin', 'jaundice'],
  runny_nose: ['runny nose', 'congested', 'nasal congestion', 'stuffy nose', 'blocked nose'],
  ulcers_on_tongue: ['mouth ulcers', 'ulcers on tongue', 'canker sores'],
  hair_loss: ['hair fall', 'hair loss', 'hair shedding']
};

// Emergency red flags
const EMERGENCY_KEYWORDS = [
  'chest pain', 'difficulty breathing', 'severe bleeding', 'unconsciousness',
  'fracture', 'suicidal thoughts', 'suicidal', 'want to kill myself',
  'crushing chest pain',
  'heart attack',
  'cannot breathe',
  'can not breathe',
  'cant breathe',
  'choking',
  'coughing blood',
  'vomiting blood',
  'unconscious',
  'passed out',
  'seizure',
  'severe allergic reaction',
  'anaphylaxis',
  'slurred speech',
  'stroke'
];

/**
 * Extracts recognized symptoms from user message text
 */
export const extractSymptomsFromText = (text, existingSymptoms = []) => {
  const normalizedText = text.toLowerCase();
  const detected = new Set(existingSymptoms);

  // Check synonym map
  for (const [symptomId, phrases] of Object.entries(SYMPTOM_SYNONYMS)) {
    for (const phrase of phrases) {
      // Word boundary match
      const regex = new RegExp(`\\b${phrase}\\b`, 'i');
      if (regex.test(normalizedText)) {
        detected.add(symptomId);
        break;
      }
    }
  }

  // Check all official symptoms
  for (const s of symptomsData) {
    const rawName = s.name.toLowerCase();
    const idName = s.id.replace(/_/g, ' ').toLowerCase();
    const regex1 = new RegExp(`\\b${rawName}\\b`, 'i');
    const regex2 = new RegExp(`\\b${idName}\\b`, 'i');
    if (regex1.test(normalizedText) || regex2.test(normalizedText)) {
      detected.add(s.id);
    }
  }
```

```
  return Array.from(detected);
};

/**
 * Checks for critical emergency red flags
 */
export const checkEmergencySigns = (text) => {
  const normalized = text.toLowerCase();
  return EMERGENCY_KEYWORDS.some((kw) => normalized.includes(kw)) ||
    /          |               |            |      |         |         /.test(normalized);
};

const isBengaliMessage = (text) => /[\u0980-\u09FF]/.test(text) ||
  /\b(jor|matha betha|buk betha|shash kosto|rokto porche|osustho)\b/i.test(text);

const isHealthRelatedMessage = (text, detectedSymptoms) => {
  if (detectedSymptoms.length > 0 || isBengaliMessage(text)) return true;
  const normalized = text.toLowerCase();
  const knownDisease = Object.keys(diseasesData).some((name) => {
    const escaped = name.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
    return new RegExp(`\\b${escaped}\\b`, 'i').test(normalized);
  });
  return knownDisease || /\b(diabetes|typhoid|dengue|hair fall|hair
    loss|fever|pain|infection|disease|illness|symptom|sick|health|rash|vomit|cough)\b/i.test(normalized);
};

/**
 * Finds suggested related symptoms based on co-occurrence in diseases
 */
export const getRelatedSymptomSuggestions = (currentSymptoms = []) => {
  if (currentSymptoms.length === 0) {
    return [
      { id: 'high_fever', name: 'High Fever' },
      { id: 'headache', name: 'Headache' },
      { id: 'cough', name: 'Cough' },
      { id: 'fatigue', name: 'Fatigue' }
    ];
  }

  const currentSet = new Set(currentSymptoms);
  const candidateCounts = {};

  for (const disease of Object.values(diseasesData)) {
    const coreSymptoms = disease.coreSymptoms || [];
    const hasOverlap = coreSymptoms.some((s) => currentSet.has(s.toLowerCase()));

    if (hasOverlap) {
      for (const s of coreSymptoms) {
        const cleanS = s.toLowerCase();
        if (!currentSet.has(cleanS)) {
          candidateCounts[cleanS] = (candidateCounts[cleanS] || 0) + 1;
        }
      }
    }
  }

  // Sort candidate symptoms by frequency
  const sorted = Object.entries(candidateCounts)
    .sort((a, b) => b[1] - a[1])
    .map(([id]) => {
      const found = symptomsData.find((item) => item.id === id);
      return {
        id,
        name: found ? found.name : id.replace(/_/g, ' ').replace(/\b\w/g, (c) => c.toUpperCase())
      };
    })
    .slice(0, 5);

  return sorted;
};

/**
 * Formats a list of symptom names nicely for chat replies
 */
const formatSymptomNames = (symptomIds) => {
  return symptomIds.map((id) => {
    const found = symptomsData.find((s) => s.id === id);
    return found ? found.name : id.replace(/_/g, ' ');
  });
};

/**
 * Generates an empathetic and clinically structured response
 */
export const processChatMessage = async ({ message, conversationHistory = [], currentSymptoms = [] }) => {
  const isEmergency = checkEmergencySigns(message);
  const updatedSymptoms = extractSymptomsFromText(message, currentSymptoms);
  const newlyDetected = updatedSymptoms.filter((s) => !currentSymptoms.includes(s));
  const isHealthRelated = isHealthRelatedMessage(message, updatedSymptoms);

  // Determine intent
  const lowerMsg = message.toLowerCase().trim();
  const isGreeting = /^(hi|hello|hey|greetings|good morning|good afternoon|good evening)\b/i.test(lowerMsg) && updatedSymptoms.length === 0;
  const isRequestingDiagnosis = /(diagnos|what do i have|predict|check my|evaluat|analyze|results|what is wrong)/i.test(lowerMsg);
  const isAffirmative = /^(yes|yeah|yep|i do|correct|sure|definitely)\b/i.test(lowerMsg);
  const isNegative = /^(no|nope|neither|none|negative)\b/i.test(lowerMsg);

  // Suggestions for next turn
  const suggestions = getRelatedSymptomSuggestions(updatedSymptoms);

  let replyText = '';
  let shouldRunInference = false;

  if (isEmergency) {
    replyText = '   **Emergency:** Seek emergency care now. Call your local emergency number or go to the nearest emergency department. Do not wait
```

```
  for an online assessment.\n\nThis is not a medical diagnosis; please consult a doctor.';
  return {
    reply: replyText,
    detectedSymptoms: updatedSymptoms,
    suggestedSymptoms: [],
    isEmergency: true,
    isHealthRelated: true,
    shouldRunInference: false
  };
}

if (!isHealthRelated) {
  return {
    reply: 'I can only help with health questions. Tell me about a symptom or condition, and I'll share general guidance. This is not a medical
  diagnosis; please consult a doctor.',
    detectedSymptoms: updatedSymptoms,
    suggestedSymptoms: [],
    isEmergency: false,
    isHealthRelated: false,
    shouldRunInference: false
  };
}

if (isGreeting) {
  replyText =
    'Hello! I am **MedInsight**, your AI Health Triage Assistant.  \n\n' +
    'I am here to help you understand your symptoms, evaluate potential health conditions, and provide supportive care guidance.\n\n' +
    'Tell me, **what symptoms or discomfort are you experiencing today?** You can describe how you feel in your own natural words.';
  return {
    reply: replyText,
    detectedSymptoms: updatedSymptoms,
    suggestedSymptoms: suggestions,
    isEmergency: false,
    isHealthRelated: true,
    shouldRunInference: false
  };
}

if (isRequestingDiagnosis) {
  if (updatedSymptoms.length === 0) {
    replyText =
      'Before I can evaluate potential conditions, please share at least one or two symptoms you are experiencing (e.g. fever, headache, cough,
  stomach pain).';
  } else {
    replyText = `Understood. Analyzing your recorded symptoms: **${formatSymptomNames(updatedSymptoms).join(', ')}** with our diagnostic
  classifier...`;
    shouldRunInference = true;
  }
  return {
    reply: replyText,
    detectedSymptoms: updatedSymptoms,
    suggestedSymptoms: suggestions,
    isEmergency: false,
    isHealthRelated: true,
    shouldRunInference
  };
}

// Symptom intake handling
if (newlyDetected.length > 0) {
  const formattedNew = formatSymptomNames(newlyDetected);
  const allFormatted = formatSymptomNames(updatedSymptoms);

  if (updatedSymptoms.length >= 3) {
    replyText =
      `I've noted your symptoms: **${allFormatted.join(', ')}**.\n\n` +
      `We now have sufficient clinical indicators to evaluate potential differential diagnoses. ` +
      `Would you like me to run the **diagnostic evaluation** now, or do you have additional symptoms like **${suggestions.slice(0, 2).map((s) =>
  s.name).join('** or **')}**?`;
  } else {
    replyText =
      `I've recorded **${formattedNew.join('**, **')}** in your current assessment.\n\n` +
      `To help narrow down the diagnosis:\n` +
      `• How long have you felt this way (e.g., today, 1–3 days, or over a week)?\n` +
      `• Are you also experiencing any of these common accompanying symptoms: **${suggestions.slice(0, 3).map((s) => s.name).join('**, **')}**?`;
  }
} else if (isAffirmative && suggestions.length > 0) {
  replyText =
    `Thanks for clarifying. You can tap on any of the suggested symptom pills below to add them to your assessment, ` +
    `or say **"Evaluate my symptoms"** when you are ready for diagnostic analysis.`;
} else if (isNegative) {
  if (updatedSymptoms.length > 0) {
    replyText =
      `Understood, no additional symptoms noted. You currently have: **${formatSymptomNames(updatedSymptoms).join(', ')}** recorded.\n\n` +
      `Would you like me to evaluate your health prediction now?`;
  } else {
    replyText = `Okay. Please tell me about any other discomfort or symptoms you'd like to check today.`;
  }
} else {
  // General conversational turn
  if (updatedSymptoms.length > 0) {
    replyText =
      `Thank you for providing that context. We currently have **${formatSymptomNames(updatedSymptoms).join(', ')}** recorded.\n\n` +
      `You can add any other symptoms by clicking the chips below, or click **"Run Diagnostic Evaluation"** whenever you're ready.`;
  } else {
    replyText =
      `I hear you. Could you tell me more specifically about any physical symptoms you are having, such as fever, cough, aches, nausea, or
  headache?`;
  }
}

return {
  reply: replyText,
  detectedSymptoms: updatedSymptoms,
  suggestedSymptoms: suggestions,
```

```
    isEmergency: false,
    isHealthRelated: true,
    shouldRunInference: false
  };
};

export default {
  extractSymptomsFromText,
  checkEmergencySigns,
  getRelatedSymptomSuggestions,
  processChatMessage
};
```

Mongoose models

backend/models/User.js

User account and profile schema.

```
import mongoose from 'mongoose';

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
      minlength: [2, 'Name must be at least 2 characters'],
      maxlength: [50, 'Name cannot exceed 50 characters']
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email address']
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: [8, 'Password must be at least 8 characters'],
      select: false
    },
    age: {
      type: Number,
      min: [0, 'Age cannot be negative'],
      max: [120, 'Age cannot exceed 120']
    },
    gender: {
      type: String,
      enum: {
        values: ['male', 'female', 'other', 'prefer_not_to_say'],
        message: '{VALUE} is not a supported gender'
      },
      lowercase: true
    },
    dateOfBirth: {
      type: Date
    },
    bloodType: {
      type: String,
      trim: true
    },
    allergies: [{
      type: String,
      trim: true
    }]
  },
  { timestamps: true }
);

export default mongoose.model('User', userSchema);
```

backend/models/Doctor.js

Doctor profile and availability schema.

```
import mongoose from 'mongoose';
import { DEFAULT_DOCTOR_AVAILABILITY_SLOTS, DEFAULT_DOCTOR_CONSULTATION_FEE } from '../config/constants.js';

const doctorSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    specialty: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true },
    available: { type: Boolean, default: true },
    rating: { type: Number, min: 0, max: 5, default: 5 },
    consultationFee: { type: Number, min: 0, default: DEFAULT_DOCTOR_CONSULTATION_FEE },
    description: { type: String, trim: true },
    availabilitySlots: {
      type: [String],
      default: DEFAULT_DOCTOR_AVAILABILITY_SLOTS
    }
  },
  { timestamps: true }
);

export default mongoose.model('Doctor', doctorSchema);
```

backend/models/Appointment.js

Appointment schema and references.

```
import mongoose from 'mongoose';
import { APPOINTMENT_STATUSES } from '../config/constants.js';

const appointmentSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
      index: true
    },
    doctor: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Doctor',
      required: true,
      index: true
    },
    doctorName: {
      type: String,
      required: true,
      trim: true
    },
    specialty: {
      type: String,
      required: true,
      trim: true
    },
    appointmentDate: {
      type: Date,
      required: true
    },
    timeSlot: {
      type: String,
      required: true,
      trim: true
    },
    reason: {
      type: String,
      trim: true,
      maxlength: 300
    },
    status: {
      type: String,
      enum: APPOINTMENT_STATUSES,
      default: APPOINTMENT_STATUSES[0]
    }
  },
  { timestamps: true }
);

appointmentSchema.index({ user: 1, appointmentDate: -1 });

export default mongoose.model('Appointment', appointmentSchema);
```

backend/models/Diagnosis.js

Diagnosis, candidate, and insight schemas.

```
import mongoose from 'mongoose';

const topPredictionSchema = new mongoose.Schema(
  {
    disease: { type: String, required: true },
    confidence: { type: Number, required: true },
    severity: { type: String, enum: ['mild', 'moderate', 'severe'], default: 'moderate' },
    description: { type: String, default: '' }
  },
  { _id: false }
);

// Accuracy and dual-model validation metrics schema
const accuracyMetricsSchema = new mongoose.Schema(
  {
    consensusLevel: { type: String, default: 'moderate' },
    agreementScore: { type: Number, default: 0 },
    symptomCoverageRatio: { type: Number, default: 0 },
    modelAgreement: { type: Boolean, default: false },
    validationStatus: { type: String, default: '' },
    mlConfidence: { type: Number, default: 0 },
    geminiConfidence: { type: Number, default: 0 },
    combinedConfidence: { type: Number, default: 0 }
  },
  { _id: false }
);

// Per-candidate supporting/missing symptom analysis produced by Gemini
// explanations (never a diagnosis choice — the model grounds candidates).
const candidateAnalysisSchema = new mongoose.Schema(
  {
    condition: { type: String, default: '' },
    supportingSymptoms: { type: [String], default: [] },
    missingSymptoms: { type: [String], default: [] },
    summary: { type: String, default: '' }
  },
  { _id: false }
);
```

```
// Gemini AI enrichment sub-document schema
const geminiInsightsSchema = new mongoose.Schema(
  {
    clinicalExplanation: { type: String, default: '' },
    symptomPatternInsight: { type: String, default: '' },
    differentialReasoning: { type: String, default: '' },
    urgencyAssessment: { type: String, enum: ['low', 'moderate', 'high'], default: 'moderate' },
    urgencyRationale: { type: String, default: '' },
    selfCareSteps: { type: [String], default: [] },
    warningSignsToWatch: { type: [String], default: [] },
    whenToSeeDoctor: { type: String, default: '' },
    clarificationQuestion: { type: String, default: '' },
    confidenceInterpretation: { type: String, default: '' },
    historicalContext: { type: String, default: '' },
    geminiPredictedDisease: { type: String, default: '' },
    geminiConfidence: { type: Number, default: 0 },
    geminiTopPredictions: { type: [topPredictionSchema], default: [] },
    candidateAnalyses: { type: [candidateAnalysisSchema], default: [] },
    accuracyMetrics: { type: accuracyMetricsSchema, default: null },
    combinedPrediction: {
      predictedDisease: { type: String, default: '' },
      confidence: { type: Number, default: 0 },
      method: { type: String, default: 'Preliminary symptom assessment' }
    },
    poweredBy: { type: String, default: 'Gemini' },
    generatedAt: { type: Date, default: null }
  },
  { _id: false }
);

const diagnosisSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
      index: true
    },
    symptoms: [
      {
        type: String,
        required: true,
        trim: true
      }
    ],
    predictedDisease: {
      type: String,
      required: true,
      trim: true
    },
    confidence: {
      type: Number,
      required: true,
      min: 0,
      max: 100
    },
    // What the confidence number represents: 'model_probability' (validated
    // classifier), 'gemini_estimate' (preliminary Gemini AI estimate),
    // 'heuristic_overlap' (symptom coverage score), 'rule_based' or 'none'.
    confidenceBasis: {
      type: String,
      enum: ['model_probability', 'gemini_estimate', 'heuristic_overlap', 'rule_based', 'none'],
      default: 'none'
    },
    severity: {
      type: String,
      enum: ['mild', 'moderate', 'severe'],
      default: 'moderate'
    },
    topPredictions: {
      type: [topPredictionSchema],
      default: []
    },
    description: {
      type: String,
      default: ''
    },
    duration: {
      type: String,
      default: '1–3 days'
    },
    notes: {
      type: String,
      trim: true,
      default: ''
    },
    recommendation: {
      type: String,
      default: ''
    },
    disclaimer: {
      type: String,
      default:
        'MedInsight provides preliminary, AI-assisted symptom triage guidance for informational purposes only. ' +
        'It is not a clinical medical diagnosis. In case of emergency or severe discomfort, please consult a certified doctor ' +
        'or contact your local emergency services immediately.'
    },
    isFallback: {
      type: Boolean,
      default: false
    },
    unrecognizedSymptoms: {
      type: [String],
      default: []
    },
```

```
    predictionProvider: {
      type: String,
      enum: ['gemini', 'ml', 'heuristic'],
      default: null
    },
    providerFallbackReason: {
      type: String,
      default: null
    },
    status: {
      type: String,
      enum: ['completed', 'reviewing'],
      default: 'completed'
    },
    geminiInsights: {
      type: geminiInsightsSchema,
      default: null
    },
    geminiEnriched: {
      type: Boolean,
      default: false
    }
  },
  {
    timestamps: true,
    toJSON: { virtuals: true },
    toObject: { virtuals: true }
  }
);

// Compound index for fast paginated history queries per user
diagnosisSchema.index({ user: 1, createdAt: -1 });

// Virtual for alias `userId`
diagnosisSchema.virtual('userId').get(function () {
  return this.user;
});

// Backward compatibility virtual for `possibleConditions`
diagnosisSchema.virtual('possibleConditions').get(function () {
  if (this.topPredictions && this.topPredictions.length > 0) {
    return this.topPredictions.map((tp) => ({
      name: tp.disease,
      likelihood: `${tp.confidence}%`
    }));
  }
  return [{ name: this.predictedDisease, likelihood: `${this.confidence}%` }];
});

export default mongoose.model('Diagnosis', diagnosisSchema);
```

Routes and middleware

backend/routes/authRoutes.js

Registration and login validation and routes.

```
import { Router } from 'express';
import { body } from 'express-validator';
import { register, signup, login } from '../controllers/authController.js';
import { validate } from '../middleware/validateMiddleware.js';
import { authLimiter } from '../middleware/rateLimiter.js';
import { PASSWORD_REGEX, PASSWORD_ERROR_MESSAGE } from '../config/constants.js';

const router = Router();

const registerValidation = [
  body('name')
    .trim()
    .isLength({ min: 2, max: 50 })
    .withMessage('Name must be between 2 and 50 characters'),
  body('email')
    .isEmail()
    .normalizeEmail()
    .withMessage('Please provide a valid email address'),
  body('password')
    .matches(PASSWORD_REGEX)
    .withMessage(PASSWORD_ERROR_MESSAGE),
  body('age')
    .optional({ checkFalsy: true })
    .isInt({ min: 0, max: 120 })
    .withMessage('Age must be between 0 and 120'),
  body('gender')
    .optional({ checkFalsy: true })
    .isIn(['male', 'female', 'other', 'prefer_not_to_say'])
    .withMessage('Gender must be one of: male, female, other, prefer_not_to_say'),
  validate
];

const loginValidation = [
  body('email')
    .isEmail()
    .normalizeEmail()
    .withMessage('Please provide a valid email address'),
  body('password')
    .notEmpty()
    .withMessage('Password is required'),
  validate
];

// Register routes
```

```
router.post('/register', authLimiter, registerValidation, register);
router.post('/signup', authLimiter, registerValidation, signup);

// Login route
router.post('/login', authLimiter, loginValidation, login);

export default router;
```

backend/routes/userRoutes.js

Protected profile endpoints.

```
import { Router } from 'express';
import { body } from 'express-validator';
import protect from '../middleware/authMiddleware.js';
import { getProfile, updateProfile } from '../controllers/userController.js';
import { validate } from '../middleware/validateMiddleware.js';

const router = Router();

const updateValidation = [
  body('name')
    .optional()
    .trim()
    .isLength({ min: 2, max: 50 })
    .withMessage('Name must be between 2 and 50 characters'),
  body('age')
    .optional({ checkFalsy: true })
    .isInt({ min: 0, max: 120 })
    .withMessage('Age must be between 0 and 120'),
  body('gender')
    .optional({ checkFalsy: true })
    .isIn(['male', 'female', 'other', 'prefer_not_to_say'])
    .withMessage('Gender must be one of: male, female, other, prefer_not_to_say'),
  validate
];

router.use(protect);

// Support both PRD standard (/me) and alias (/profile)
router.get('/me', getProfile);
router.put('/me', updateValidation, updateProfile);

router.get('/profile', getProfile);
router.put('/profile', updateValidation, updateProfile);

export default router;
```

backend/routes/diagnosisRoutes.js

Protected diagnosis lifecycle and public symptom endpoint.

```
import { Router } from 'express';
import protect from '../middleware/authMiddleware.js';
import {
  createDiagnosis,
  getDiagnoses,
  getDiagnosis,
  deleteDiagnosis,
  getSymptoms
} from '../controllers/diagnosisController.js';

const router = Router();

// Publicly accessible symptom taxonomy for selection
router.get('/symptoms', getSymptoms);

// Protected diagnosis lifecycle endpoints
router.use(protect);

router.post('/predict', createDiagnosis);
router.get('/history', getDiagnoses);

router.route('/')
  .get(getDiagnoses)
  .post(createDiagnosis);

router.route('/:id')
  .get(getDiagnosis)
  .delete(deleteDiagnosis);

export default router;
```

backend/routes/doctorRoutes.js

Protected doctor and appointment endpoints.

```
import { Router } from 'express';
import protect from '../middleware/authMiddleware.js';
import {
  createAppointment,
  getAppointments,
  getDoctors
} from '../controllers/doctorController.js';

const router = Router();

router.use(protect);
```

```
router.get('/doctors', getDoctors);
router.get('/appointments', getAppointments);
router.post('/appointments', createAppointment);

export default router;
```

backend/routes/chatRoutes.js

Chat route, optional auth, history normalization, and request validation.

```
import { Router } from 'express';
import jwt from 'jsonwebtoken';
import { body } from 'express-validator';
import User from '../models/User.js';
import { handleChatMessage } from '../controllers/chatController.js';
import { chatLimiter } from '../middleware/rateLimiter.js';
import { validate } from '../middleware/validateMiddleware.js';
import {
  CHAT_MAX_HISTORY_ENTRIES,
  CHAT_MAX_MESSAGE_LENGTH,
  CHAT_MAX_SYMPTOMS
} from '../config/constants.js';

const router = Router();

/**
 * Optional-auth middleware.
 * If a valid Bearer token is present, attaches req.user so the diagnosis
 * can be persisted. If no token or invalid token → continues anonymously.
 */
const optionalAuth = async (req, _res, next) => {
  const authHeader = req.headers.authorization;
  if (authHeader?.startsWith('Bearer ')) {
    try {
      const token = authHeader.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = await User.findById(decoded.id).lean();
    } catch (_e) {
      // Invalid token → proceed without user (anonymous mode)
    }
  }
  next();
};

const normalizeConversationHistory = (req, _res, next) => {
  if (!req.body || typeof req.body !== 'object' || Array.isArray(req.body)) req.body = {};
  const rawHistory = req.body?.conversationHistory;
  if (!Array.isArray(rawHistory)) {
    req.body.conversationHistory = [];
    return next();
  }

  const normalizedHistory = rawHistory.flatMap((entry) => {
    if (!entry || typeof entry !== 'object' || Array.isArray(entry)) return [];

    const role = String(entry.role || '').trim().toLowerCase();
    if (!['user', 'assistant', 'bot', 'model', 'ai'].includes(role)) return [];

    const text = [entry.text, entry.content, entry.message, entry.parts]
      .map((value) => Array.isArray(value)
        ? value.map((part) => typeof part === 'string' ? part : part?.text || '').join('')
        : typeof value === 'string'
          ? value
          : value && typeof value === 'object'
            ? String(value.text || '')
            : '')
      .find((value) => value.trim()) || '';
    const content = text.trim();

    if (!content || /^ /u.test(content) || /invalid/i.test(content)) return [];

    return [{
      role: role === 'user' ? 'user' : 'assistant',
      content: content.slice(0, 1500)
    }];
  });

  req.body.conversationHistory = normalizedHistory.slice(-CHAT_MAX_HISTORY_ENTRIES);
  next();
};

/**
 * POST /api/chat/message
 * Works both for logged-in users (persists diagnosis to DB)
 * and anonymous visitors (stateless evaluation only).
 */
router.post(
  '/message',
  chatLimiter,
  normalizeConversationHistory,
  [
    body().custom((payload) => payload && typeof payload === 'object' && !Array.isArray(payload))
      .withMessage('Request body must be an object'),
    body('action').optional().isIn(['evaluate']).withMessage('Action must be evaluate'),
    body('message').custom((value, { req }) => {
      if (req.body.action === 'evaluate' && (value === undefined || value === '')) return true;
      return typeof value === 'string' &&
        value.trim().length > 0 &&
        value.trim().length <= CHAT_MAX_MESSAGE_LENGTH;
    }).withMessage(`Message must be between 1 and ${CHAT_MAX_MESSAGE_LENGTH} characters`),
    body('currentSymptoms').optional().isArray({ max: CHAT_MAX_SYMPTOMS })
      .withMessage(`At most ${CHAT_MAX_SYMPTOMS} symptoms may be submitted`)
```

```
      .bail()
      .custom((symptoms) => symptoms.every((symptom) =>
        typeof symptom === 'string' && symptom.length > 0 && symptom.length <= 80
      ))
      .withMessage('Symptoms must be non-empty strings of at most 80 characters'),
    body('duration').optional().isString().bail().trim().isLength({ max: 100 })
      .withMessage('Duration must be at most 100 characters'),
    body('notes').optional().isString().bail().trim().isLength({ max: 500 })
      .withMessage('Notes must be at most 500 characters'),
    validate
  ],
  optionalAuth,
  handleChatMessage
);

export default router;
```

backend/middleware/authMiddleware.js

JWT protection for authenticated routes.

```
import jwt from 'jsonwebtoken';
import User from '../models/User.js';
import { sendError } from '../utils/apiResponse.js';

const protect = async (req, res, next) => {
  const authHeader = req.headers.authorization;
  const token = authHeader?.startsWith('Bearer ')
    ? authHeader.split(' ')[1]
    : null;

  if (!token) {
    return sendError(res, 401, 'Authentication token is required', {
      code: 'AUTH_TOKEN_MISSING'
    });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id);
    if (!req.user) {
      return sendError(res, 401, 'User associated with this token no longer exists', {
        code: 'AUTH_USER_NOT_FOUND'
      });
    }
    next();
  } catch (err) {
    return sendError(res, 401, 'Invalid or expired authentication token', {
      code: 'AUTH_TOKEN_INVALID',
      details: err.message
    });
  }
};

export default protect;
```

backend/middleware/errorHandler.js

Central API exception handling.

```
import { sendError } from '../utils/apiResponse.js';

/**
 * Global Express Error Handling Middleware
 */
export const errorHandler = (err, _req, res, _next) => {
  const statusCode = err.statusCode || err.status || 500;
  const message = err.message || 'Internal server error';

  if (process.env.NODE_ENV !== 'production') {
    console.error('SERVER ERROR:', err);
  }

  const errorDetails = {
    code: err.code || 'INTERNAL_ERROR',
    ...(process.env.NODE_ENV !== 'production' && { stack: err.stack })
  };

  return sendError(res, statusCode, message, errorDetails);
};
```

backend/middleware/rateLimiter.js

Auth and chat rate limits.

```
import rateLimit from 'express-rate-limit';
import {
  CHAT_RATE_LIMIT_MAX,
  CHAT_RATE_LIMIT_WINDOW_MS,
  RATE_LIMIT_AUTH_WINDOW_MS,
  RATE_LIMIT_AUTH_MAX
} from '../config/constants.js';

export const authLimiter = rateLimit({
  windowMs: RATE_LIMIT_AUTH_WINDOW_MS,
  max: RATE_LIMIT_AUTH_MAX,
  standardHeaders: true,
  legacyHeaders: false,
```

```
  handler: (_req, res) => {
    return res.status(429).json({
      success: false,
      message:
        'Too many authentication attempts. Please try again after 15 minutes.',
      data: null,
      error: { code: 'AUTH_RATE_LIMIT_EXCEEDED' }
    });
  }
});

export const chatLimiter = rateLimit({
  windowMs: CHAT_RATE_LIMIT_WINDOW_MS,
  max: CHAT_RATE_LIMIT_MAX,
  standardHeaders: true,
  legacyHeaders: false,
  handler: (_req, res) => {
    return res.status(429).json({
      success: false,
      message: 'Too many chat requests. Please try again later.',
      data: null,
      error: { code: 'CHAT_RATE_LIMIT_EXCEEDED' }
    });
  }
});
```

backend/middleware/validateMiddleware.js

Express-validator response handling.

```
import { validationResult } from 'express-validator';
import { sendError } from '../utils/apiResponse.js';

export const validate = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    const formattedErrors = errors.array().map((err) => ({
      field: err.path || err.param,
      message: err.msg
    }));

    return sendError(res, 400, formattedErrors[0]?.message || 'Validation failed', {
      details: formattedErrors
    });
  }
  next();
};
```

backend/config/constants.js

Shared application and provider constants; excludes secret values.

```
export const JWT_EXPIRES_IN = '7d';
export const BCRYPT_SALT_ROUNDS = 12;

export const PASSWORD_REGEX = /^(?=.*\d).{8,}$/;
export const PASSWORD_ERROR_MESSAGE = 'Password must be at least 8 characters long and contain at least one number';

export const RATE_LIMIT_AUTH_WINDOW_MS = 50 * 60 * 1000; // 50 minutes
export const RATE_LIMIT_AUTH_MAX = 50; // Max 50 attempts per window

export const RATE_LIMIT_GLOBAL_WINDOW_MS = 50 * 60 * 1000;
export const RATE_LIMIT_GLOBAL_MAX = 200;

export const CHAT_RATE_LIMIT_WINDOW_MS = 15 * 60 * 1000;
export const CHAT_RATE_LIMIT_MAX = 30;
export const CHAT_MAX_MESSAGE_LENGTH = 1000;
export const CHAT_MAX_HISTORY_ENTRIES = 10;
export const CHAT_MAX_SYMPTOMS = 30;

export const DEFAULT_PAGE_SIZE = 10;
export const MAX_PAGE_SIZE = 50;

export const APPOINTMENT_STATUSES = ['pending', 'confirmed', 'completed', 'cancelled'];
export const DEFAULT_DOCTOR_AVAILABILITY_SLOTS = ['09:00', '10:30', '14:00', '16:30'];
export const DEFAULT_DOCTOR_CONSULTATION_FEE = 49;

// ---------------------------------------------------------------------------
// Prediction provider configuration (Gemini preliminary prediction with
// symptom-grounded ML/heuristic fallback; Gemini also supplies explanations)
// ---------------------------------------------------------------------------
// 'gemini' (default): Gemini generates a preliminary top-3 differential for
//   ANY submitted symptom(s) — including a single symptom or unusual entries
//   the taxonomy cannot resolve. Any Gemini failure (or the orthopedic-trauma
//   safety rule) falls back to the Python ML engine / local heuristic rules,
//   which also explain candidates via Gemini or local clinical rules.
// 'ml': symptom-engine predictions with local rule-based explanations only —
//   no external Gemini calls.
export const PREDICTION_PROVIDER_GEMINI = 'gemini';
export const PREDICTION_PROVIDER_ML = 'ml';
export const PREDICTION_PROVIDER_HEURISTIC = 'heuristic';
export const PREDICTION_PROVIDER_ENV_DEFAULT = PREDICTION_PROVIDER_GEMINI;

// What a displayed confidence number actually represents. Probabilities are
// only shown for the validated model basis; the Gemini basis is a preliminary
// AI estimate, and every other basis is labelled as a preliminary assessment
// in the UI.
export const CONFIDENCE_BASIS = {
  MODEL_PROBABILITY: 'model_probability',
  HEURISTIC_OVERLAP: 'heuristic_overlap',
```

```
  RULE_BASED: 'rule_based',
  GEMINI_ESTIMATE: 'gemini_estimate',
  NONE: 'none'
};

// Returned when none of the submitted inputs resolve to known symptoms.
export const SYMPTOMS_NOT_RECOGNIZED_CODE = 'SYMPTOMS_NOT_RECOGNIZED';
export const SYMPTOMS_NOT_RECOGNIZED_MESSAGE =
  'We could not recognize those entries as symptoms. Please select symptoms from the symptom list or describe what you are feeling (for example:
    headache, cough, fever).';
export const NO_VALID_SYMPTOMS_MESSAGE =
  'No valid symptoms were submitted. Please enter at least one symptom before requesting an assessment.';
export const SYMPTOMS_NO_MATCH_MESSAGE =
  'The provided symptoms do not match any condition profile in the reference dataset, so no reliable assessment can be made yet. Please add a few
    different or additional symptoms, or select them from the symptom list.';

// Current stable Gemini Flash model verified against the live Gemini API and
// docs (https://ai.google.dev/gemini-api/docs/models). Note: gemini-2.5-flash
// now returns 404 for new API keys, and gemini-3.8-flash can return 503 during
// demand spikes — override per environment with the GEMINI_MODEL env var.
export const GEMINI_DEFAULT_MODEL = 'gemini-3.6-flash';
export const GEMINI_PREDICTION_TIMEOUT_MS = 12000;
export const GEMINI_ENRICHMENT_TIMEOUT_MS = 6500;
export const GEMINI_MAX_OUTPUT_TOKENS = 1024;
export const GEMINI_TEMPERATURE = 0.2;
export const GEMINI_CHAT_MAX_OUTPUT_TOKENS = 350;
export const GEMINI_CHAT_TEMPERATURE = 0.4;
export const GEMINI_CHAT_TIMEOUT_MS = 5000;
export const GEMINI_CHAT_HISTORY_TURNS = 10;

// Generative models never produce clinical probabilities; Gemini output is
// used for explanations only and must not carry confidence values at all.
// Kept limits for top-candidate selection & output sanitization.
export const GEMINI_MAX_TOP_PREDICTIONS = 3;
export const GEMINI_MAX_ARRAY_ITEMS = 5;

// Output sanitization limits for model responses
export const GEMINI_MAX_CONDITION_LENGTH = 120;
export const GEMINI_MAX_TEXT_LENGTH = 800;
export const GEMINI_MAX_SHORT_TEXT_LENGTH = 400;
export const GEMINI_MAX_ARRAY_ITEM_LENGTH = 200;

export const URGENCY_LEVELS = ['low', 'moderate', 'high'];
export const SEVERITY_LEVELS = ['mild', 'moderate', 'severe'];
export const DEFAULT_SYMPTOM_DURATION = '1–3 days';
export const LOW_CONFIDENCE_SYMPTOM_THRESHOLD = 2;
export const LOW_CONFIDENCE_MESSAGE =
  'Only one symptom was provided — this is a preliminary estimate. Add any other symptoms you are experiencing for a more useful assessment.';

export const GEMINI_API_KEY_DOCS_URL = 'https://aistudio.google.com/apikey';

export const CARE_RECOMMENDATIONS = {
  mild: 'Rest, hydrate, and monitor your symptoms. Consider over-the-counter support if appropriate, and consult a doctor if symptoms worsen or
    persist past a few days.',
  moderate: 'Consider scheduling a consultation with a healthcare professional within 24 to 48 hours for clinical evaluation and targeted therapy.',
  severe: 'Please seek prompt clinical or emergency medical evaluation. If experiencing difficulty breathing, high unyielding fever, or intense chest
    pain, contact emergency medical services immediately.'
};

export const GEMINI_ERROR_CODES = {
  NOT_CONFIGURED: 'GEMINI_NOT_CONFIGURED',
  AUTH_FAILED: 'GEMINI_AUTH_FAILED',
  INVALID_REQUEST: 'GEMINI_INVALID_REQUEST',
  RATE_LIMITED: 'GEMINI_RATE_LIMITED',
  TIMEOUT: 'GEMINI_TIMEOUT',
  UNAVAILABLE: 'GEMINI_UNAVAILABLE',
  INVALID_RESPONSE: 'GEMINI_INVALID_RESPONSE'
};
```

Python ML service and classifier

ml-service/app.py

Flask inference API, payload checks, feature encoding, and classifier call.

```
import os
import sys
import json
import pickle
import re
from flask import Flask, request, jsonify

# Ensure classifier class is resolvable during unpickling
base_dir = os.path.dirname(os.path.abspath(__file__))
model_dir = os.path.join(base_dir, 'model')
sys.path.insert(0, model_dir)
from classifier import SymptomDiseaseClassifier  # noqa: E402

app = Flask(__name__)

# Model and metadata holders
model = None
features = []
feature_index_map = {}
symptoms_metadata = []
diseases_metadata = {}
validation_metrics = {}
is_loaded = False

MANDATORY_DISCLAIMER = (
    "MedInsight provides preliminary, AI-assisted symptom triage guidance for informational purposes only. "
```

```
    "It is not a clinical medical diagnosis. In case of emergency or severe discomfort, please consult a certified doctor "
    "or contact your local emergency services immediately."
)

SYMPTOM_ALIASES = {
    "fever": "high_fever",
    "high_fever": "high_fever",
    "high_fever_": "high_fever",
    "low_fever": "mild_fever",
    "mild_fever": "mild_fever",
    "flu": "high_fever",
    "temperature": "high_fever",
    "feverish": "high_fever",
    "coughing": "cough",
    "sneeze": "continuous_sneezing",
    "sneezing": "continuous_sneezing",
    "nauseated": "nausea",
    "vomit": "vomiting",
    "diarrhea": "diarrhoea",
    # --- extended common words ---
    "dizzy": "dizziness",
    "tired": "fatigue",
    "tiredness": "fatigue",
    "exhausted": "fatigue",
    "sleepy": "fatigue",
    "weak": "fatigue",
    "cold": "chills",
    "shivering": "chills",
    "rash": "skin_rash",
    "itchy": "itching",
    "itch": "itching",
    "breathless": "breathlessness",
    "sob": "breathlessness",
    "shortness_of_breath": "breathlessness",
    "difficulty_breathing": "breathlessness",
    "yellow": "yellowish_skin",
    "jaundice": "yellowish_skin",
    "runny": "runny_nose",
    "constipated": "constipation",
    "sweat": "sweating",
    "backache": "back_pain",
    "belly": "stomach_pain",
    "tummy": "stomach_pain",
    "stomach_ache": "stomach_pain",
    "stomach_aches": "stomach_pain",
    "tummy_pain": "stomach_pain",
    "belly_ache": "stomach_pain",
    "bloating": "abdominal_pain",
    "palpitation": "palpitations",
    "anxious": "anxiety",
    "depressed": "depression",
    "loss_of_smell": "loss_of_smell",
    "no_smell": "loss_of_smell",
    "no_taste": "loss_of_taste",
    "sore_throat": "throat_irritation",
    "throat_pain": "throat_irritation",
    "head_ache": "headache",
    "headaches": "headache",
    "head_pain": "headache",
    "chest_tightness": "chest_pain",
    "chest_pressure": "chest_pain",
    "body_aches": "muscle_pain",
    "aching_body": "muscle_pain",
    "loose_motion": "diarrhoea",
    "loose_stools": "diarrhoea",
    "blocked_nose": "runny_nose",
    "stuffy_nose": "runny_nose",
    "nasal_congestion": "runny_nose",
    # --- body part / injury aliases ---
    "leg_pain": "joint_pain",
    "leg_break": "joint_pain",
    "leg_ache": "joint_pain",
    "broken_leg": "joint_pain",
    "knee_pain": "joint_pain",
    "ankle_pain": "joint_pain",
    "arm_pain": "joint_pain",
    "shoulder_pain": "joint_pain",
    "hip_pain": "joint_pain",
    "leg": "joint_pain",
    "arm": "muscle_pain",
    "fracture": "joint_pain",
    "sprain": "joint_pain",
    "swollen": "swelling_joints",
    "stomach_problem": "stomach_pain",
    "chest_problem": "chest_pain",
    "back_problem": "back_pain",
    "throat_problem": "throat_irritation",
    "eye_pain": "pain_behind_the_eyes",
    "ear_pain": "ear_ache",
    "body_pain": "muscle_pain",
    "weakness": "fatigue",
    "no_energy": "fatigue",
    "low_energy": "fatigue",
}


def normalize_symptom(raw_symptom):
    cleaned = raw_symptom.strip().lower()
    cleaned = re.sub(r"[^a-z0-9]+", "_", cleaned)
    cleaned = re.sub(r"_+", "_", cleaned).strip("_")
    return SYMPTOM_ALIASES.get(cleaned, cleaned)


def levenshtein(a, b):
    """Classic edit-distance (iterative two-row implementation)."""
```

```
    if a == b:
        return 0
    if not a:
        return len(b)
    if not b:
        return len(a)
    prev = list(range(len(b) + 1))
    for i, ca in enumerate(a, 1):
        cur = [i]
        for j, cb in enumerate(b, 1):
            cur.append(min(prev[j] + 1, cur[j - 1] + 1, prev[j - 1] + (ca != cb)))
        prev = cur
    return prev[-1]


def match_feature(symptom, features, feature_index_map):
    """
    Resolve a normalized symptom string to exactly one dataset feature.

    Tier 1: exact feature match.
    Tier 2: whole-word match — the symptom must equal one word of exactly one
            feature (ambiguous words like bare "pain" stay unresolved instead
            of silently mapping to an arbitrary condition).
    Tier 3: readable display-name containment, only when unique.
    Tier 4: typo-tolerant fuzzy match (edit distance within length-scaled bound).

    Returns the matched feature name or None.
    """
    # Tier 1
    if symptom in feature_index_map:
        return symptom

    # Tier 2: unique whole-word match (multiple hits are ambiguous, e.g. a
    # bare "pain" touching every *_pain feature, and stay unresolved here)
    symptom_words = [w for w in symptom.split("_") if w]
    if symptom_words:
        word_hits = set()
        for feat in features:
            if any(w == fw for w in symptom_words for fw in feat.split("_")):
                word_hits.add(feat)
        if len(word_hits) == 1:
            return next(iter(word_hits))

    # Tier 3: readable containment, only when unique
    readable = symptom.replace("_", " ")
    tier3_hits = [
        f for f in features
        if readable in f.replace("_", " ") or f.replace("_", " ") in readable
    ]
    if len(tier3_hits) == 1:
        return tier3_hits[0]

    # Tier 4: typo tolerance (edit distance). Skips very short inputs so that
    # random short words cannot collide with dataset features.
    if len(symptom) < 4:
        return None
    max_distance = 2 if len(symptom) >= 6 else 1
    best, best_distance = None, max_distance + 1
    for feat in features:
        distance = levenshtein(symptom, feat)
        if distance < best_distance:
            best, best_distance = feat, distance
    if best is not None and best_distance <= max_distance:
        return best
    return None


def nearest_features(symptom, features, limit=3):
    """Closest dataset feature names for an unrecognized input (clarification help)."""
    if not symptom:
        return []
    scored = sorted(
        ((levenshtein(symptom, feat), feat) for feat in features),
        key=lambda item: item[0]
    )
    return [feat for _, feat in scored[:limit]]


def score_disease_overlap(disease_name, recognized_symptoms):
    disease_entry = diseases_metadata.get(disease_name, {})
    core_symptoms = disease_entry.get('coreSymptoms', [])
    if not core_symptoms:
        return 0.0

    normalized_core = {normalize_symptom(symptom) for symptom in core_symptoms}
    overlap = sum(1 for symptom in recognized_symptoms if symptom in normalized_core)
    return overlap / max(len(core_symptoms), 1)


def select_primary_prediction(top_predictions, recognized_symptoms):
    if not top_predictions:
        return None

    scored = []
    for candidate in top_predictions:
        disease_name = candidate["disease"]
        overlap_score = score_disease_overlap(disease_name, recognized_symptoms)
        confidence = candidate["confidence"]
        severity_boost = 5 if candidate.get("severity") == "severe" else 0
        scored.append({
            **candidate,
            "overlap_score": overlap_score,
            "final_score": (confidence * 0.7) + (overlap_score * 100 * 0.3) + severity_boost
        })
```

```
    scored.sort(key=lambda item: item["final_score"], reverse=True)
    return scored[0]


def load_artifacts():
    global model, features, feature_index_map, symptoms_metadata, diseases_metadata, validation_metrics, is_loaded

    model_path = os.path.join(model_dir, 'model.pkl')
    features_path = os.path.join(model_dir, 'features.json')
    symptoms_path = os.path.join(model_dir, 'symptoms.json')
    diseases_path = os.path.join(model_dir, 'diseases.json')
    validation_path = os.path.join(model_dir, 'validation_metrics.json')

    if not os.path.exists(model_path):
        raise FileNotFoundError(f"Model file not found at {model_path}. Run train.py first.")

    with open(model_path, 'rb') as f:
        model = pickle.load(f)

    with open(features_path, 'r', encoding='utf-8') as f:
        features = json.load(f)

    feature_index_map = {feat: idx for idx, feat in enumerate(features)}

    if os.path.exists(symptoms_path):
        with open(symptoms_path, 'r', encoding='utf-8') as f:
            symptoms_metadata = json.load(f)

    if os.path.exists(diseases_path):
        with open(diseases_path, 'r', encoding='utf-8') as f:
            diseases_metadata = json.load(f)

    # Held-out validation metrics produced at training time (evaluate.py /
    # train.py). Served with each prediction so confidence values are always
    # reported together with their measured accuracy and calibration.
    if os.path.exists(validation_path):
        try:
            with open(validation_path, 'r', encoding='utf-8') as f:
                validation_metrics = json.load(f)
        except (OSError, ValueError):
            validation_metrics = {}
    else:
        validation_metrics = {}

    is_loaded = True
    print(f"[ML Service] Loaded model with {len(features)} features and {len(model.classes_)} classes.")

# Load on module import
try:
    load_artifacts()
except Exception as e:
    print(f"[ML Service] Warning: Failed to load model on startup: {e}")

@app.route('/health', methods=['GET'])
def health_check():
    """Liveness probe reporting model status and feature counts."""
    return jsonify({
        "status": "ok",
        "service": "ml-inference-microservice",
        "model_loaded": is_loaded,
        "features_count": len(features),
        "classes_count": len(model.classes_) if model else 0
    }), 200

@app.route('/symptoms', methods=['GET'])
def get_symptoms():
    """Returns all available symptoms for frontend multi-select."""
    if symptoms_metadata:
        return jsonify(symptoms_metadata), 200

    formatted = [
        {"id": feat, "name": feat.replace('_', ' ').title()}
        for feat in features
    ]
    return jsonify(formatted), 200

@app.route('/predict', methods=['POST'])
def predict_diagnosis():
    """
    Ingests patient symptoms and runs inference against the trained model.

    Contract:
    - 422 when NO input resolves to a known symptom (caller must ask the user
      to clarify — never invent a condition for unrelated input).
    - 200 model prediction for any recognized symptom count (single symptom
      included), with `confidence_basis` describing what the confidence
      number actually is (validated model probability).
    """
    if not is_loaded or model is None:
        return jsonify({"detail": "Model is not loaded or unavailable."}), 503

    data = request.get_json(silent=True) or {}
    raw_symptoms = data.get('symptoms')

    if not raw_symptoms or not isinstance(raw_symptoms, list) or len(raw_symptoms) == 0:
        return jsonify({"detail": "At least one symptom must be provided in 'symptoms' array."}), 422

    # Step 1: Normalize + alias resolution
    normalized = []
    for symptom in raw_symptoms:
        if not isinstance(symptom, str) or not symptom.strip():
            continue
        normalized.append(normalize_symptom(symptom))

    # Step 2: Resolve each symptom to a dataset feature (exact → unique word →
```

```
    # readable containment → typo-tolerant fuzzy match)
    recognized = []
    unrecognized = []
    vector = [0] * len(features)

    for symptom in normalized:
        match = match_feature(symptom, features, feature_index_map)
        if match:
            vector[feature_index_map[match]] = 1
            if match not in recognized:
                recognized.append(match)
        else:
            unrecognized.append(symptom)

    # Nothing resolved: ask the caller to clarify instead of guessing.
    if not recognized:
        suggestions = []
        for symptom in unrecognized:
            for candidate in nearest_features(symptom, features):
                if candidate not in suggestions:
                    suggestions.append(candidate)
        return jsonify({
            "success": False,
            "code": "UNRECOGNIZED_SYMPTOMS",
            "detail": (
                "The symptom(s) you entered were not found in the diagnostic dataset. "
                "Please try selecting from the symptom list, or rephrase your symptom."
            ),
            "unrecognized_symptoms": unrecognized,
            "suggestions": suggestions[:5]
        }), 422

    # Step 3: Run model inference
    probas = model.predict_proba([vector])[0]
    classes = model.classes_

    top_indices = sorted(range(len(probas)), key=lambda i: probas[i], reverse=True)[:3]

    top_predictions = []
    for idx in top_indices:
        disease_name = classes[idx]
        conf = float(probas[idx] * 100)
        info = diseases_metadata.get(disease_name, {})
        top_predictions.append({
            "disease": disease_name,
            "confidence": round(conf, 1),
            "severity": info.get('severity', 'moderate'),
            "description": info.get('description', 'A medical condition matching the recorded symptoms.')
        })

    # Step 4: Select primary — absolute fallback ensures we always get one
    primary = select_primary_prediction(top_predictions, recognized)
    if primary is None:
        best_idx = top_indices[0]
        disease_name = classes[best_idx]
        info = diseases_metadata.get(disease_name, {})
        primary = {
            "disease": disease_name,
            "confidence": round(float(probas[best_idx] * 100), 1),
            "severity": info.get('severity', 'moderate'),
            "description": info.get('description', 'A medical condition matching the recorded symptoms.'),
            "overlap_score": 0.0
        }

    # Step 5: Determine if low-confidence advisory should be shown
    is_low_confidence = (
        primary["confidence"] < 30 or
        (primary.get("overlap_score", 1.0) < 0.15 and len(recognized) <= 2)
    )

    response = {
        "success": True,
        "primary_condition": primary["disease"],
        "confidence": primary["confidence"],
        "confidence_basis": "model_probability",
        "validation": validation_metrics,
        "severity": primary.get("severity", "moderate"),
        "description": primary.get("description", "A medical condition matching the recorded symptoms."),
        "top_predictions": top_predictions,
        "recognized_symptoms": recognized,
        "unrecognized_symptoms": unrecognized,
        "disclaimer": MANDATORY_DISCLAIMER
    }

    if is_low_confidence:
        response["low_confidence"] = True
        response["low_confidence_message"] = (
            f"{len(recognized)} symptom(s) provided — "
            "this is a preliminary estimate. Add more symptoms for a more accurate assessment."
        )

    return jsonify(response), 200


if __name__ == '__main__':
    port = int(os.environ.get('PORT', 8000))
    print(f"[ML Service] Running Flask server on port {port}...")
    app.run(host='0.0.0.0', port=port, debug=False)
```

ml-service/model/classifier.py

Classifier wrapper used by the ML service.

```
import math
import json
import os

class SymptomDiseaseClassifier:
    """
    Interpretable Probabilistic Classifier for Multi-Hot Symptom Vectors.
    Employs Multivariate Bernoulli Naive Bayes with Laplace smoothing,
    designed to produce calibrated top-K differential diagnosis probabilities
    without relying on native C-extension LAPACK DLLs.
    """
    def __init__(self, alpha=1.0):
        self.alpha = alpha  # Laplace smoothing parameter
        self.classes_ = []
        self.feature_names_ = []
        self.class_priors_ = {}
        self.feature_probs_ = {}  # {class: {feature: P(feature=1 | class)}}

    def fit(self, X, y, feature_names):
        """
        Train the classifier.
        X: list of lists (binary 0/1 vectors)
        y: list of disease strings
        feature_names: list of feature name strings
        """
        self.feature_names_ = list(feature_names)
        self.classes_ = sorted(list(set(y)))
        n_samples = len(y)
        n_features = len(self.feature_names_)

        # Calculate class priors P(C)
        class_counts = {c: 0 for c in self.classes_}
        class_samples = {c: [] for c in self.classes_}

        for i, label in enumerate(y):
            class_counts[label] += 1
            class_samples[label].append(X[i])

        self.class_priors_ = {c: math.log(count / n_samples) for c, count in class_counts.items()}

        # Calculate feature probabilities P(x_i = 1 | C) with Laplace smoothing
        self.feature_probs_ = {}
        for c in self.classes_:
            samples = class_samples[c]
            n_c = len(samples)
            self.feature_probs_[c] = []

            for f_idx in range(n_features):
                # Count how many times this feature was active for class c
                count_active = sum(row[f_idx] for row in samples)
                # Smoothed probability
                p = (count_active + self.alpha) / (n_c + 2.0 * self.alpha)
                self.feature_probs_[c].append(p)

        return self

    def predict_proba(self, X):
        """
        Compute calibrated probability distributions across all classes.
        Returns: list of probability distributions (each sum to 1.0)
        """
        all_probas = []

        for row in X:
            log_scores = []
            for c in self.classes_:
                log_prob = self.class_priors_[c]
                probs_for_c = self.feature_probs_[c]

                for f_idx, val in enumerate(row):
                    p = probs_for_c[f_idx]
                    if val == 1:
                        log_prob += math.log(p)
                    else:
                        log_prob += math.log(1.0 - p)

                log_scores.append(log_prob)

            # Softmax normalization for numerical stability
            max_log = max(log_scores)
            exp_scores = [math.exp(s - max_log) for s in log_scores]
            total_exp = sum(exp_scores)
            probas = [s / total_exp for s in exp_scores]
            all_probas.append(probas)

        return all_probas

    def predict(self, X):
        """Returns the most probable class for each sample."""
        probas = self.predict_proba(X)
        predictions = []
        for p_row in probas:
            best_idx = max(range(len(p_row)), key=lambda i: p_row[i])
            predictions.append(self.classes_[best_idx])
        return predictions

    def score(self, X, y):
        """Computes prediction accuracy."""
        preds = self.predict(X)
        correct = sum(1 for true_val, pred_val in zip(y, preds) if true_val == pred_val)
        return correct / len(y) if y else 0.0
```

ml-service/model/train.py

Model training pipeline.

```
import os
import csv
import json
import pickle
import random
import sys

# Ensure local imports work
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from classifier import SymptomDiseaseClassifier

def load_data(csv_path):
    with open(csv_path, 'r', encoding='utf-8') as f:
        reader = csv.reader(f)
        header = next(reader)
        features = header[:-1]

        X = []
        y = []
        for row in reader:
            if not row:
                continue
            X.append([int(v) for v in row[:-1]])
            y.append(row[-1].strip('"'))

    return X, y, features

def train():
    current_dir = os.path.dirname(os.path.abspath(__file__))
    data_path = os.path.join(current_dir, '..', 'data', 'dataset.csv')
    model_output_path = os.path.join(current_dir, 'model.pkl')
    features_output_path = os.path.join(current_dir, 'features.json')

    print(f"[Training] Loading dataset from: {data_path}")
    X, y, feature_cols = load_data(data_path)
    print(f"[Training] Loaded {len(X)} samples with {len(feature_cols)} features and {len(set(y))} unique conditions.")

    # Stratified 80/20 train/test split
    data = list(zip(X, y))
    random.seed(42)
    random.shuffle(data)

    split_idx = int(len(data) * 0.8)
    train_data = data[:split_idx]
    test_data = data[split_idx:]

    X_train, y_train = zip(*train_data)
    X_test, y_test = zip(*test_data)

    print("[Training] Fitting SymptomDiseaseClassifier...")
    clf = SymptomDiseaseClassifier(alpha=0.5)
    clf.fit(list(X_train), list(y_train), feature_cols)

    acc = clf.score(list(X_test), list(y_test))
    print(f"\n[Training] Evaluation Accuracy: {acc * 100:.2f}%")

    if acc < 0.90:
        raise ValueError(f"Model accuracy {acc * 100:.2f}% is below the 90% threshold.")

    # Save model.pkl
    with open(model_output_path, 'wb') as f:
        pickle.dump(clf, f)
    print(f"[Training] Saved model to: {model_output_path}")

    # Save feature names list for inference vectorization
    with open(features_output_path, 'w', encoding='utf-8') as f:
        json.dump(feature_cols, f, indent=2)
    print(f"[Training] Saved feature ordering to: {features_output_path}")

    # Held-out discrimination + calibration metrics, stored next to the model
    # so the API can always report confidence values together with their
    # measured accuracy and calibration (evaluate.py is the audit entry point).
    try:
        from evaluate import compute_metrics

        diseases_path = os.path.join(current_dir, 'diseases.json')
        with open(diseases_path, 'r', encoding='utf-8') as f:
            diseases_metadata = json.load(f)
        metrics = compute_metrics(clf, list(X_test), list(y_test), feature_cols, diseases_metadata)
        metrics_output_path = os.path.join(current_dir, 'validation_metrics.json')
        with open(metrics_output_path, 'w', encoding='utf-8') as f:
            json.dump(metrics, f, indent=2)
        print(f"[Training] Saved validation metrics to: {metrics_output_path}")
        print(f"[Training] Held-out top-1 {metrics['accuracy_top1_pct']}% | "
              f"top-3 {metrics['accuracy_top3_pct']}% | ECE {metrics['ece_10bin']}")
    except Exception as metrics_error:
        print(f"[Training] Warning: could not compute validation metrics: {metrics_error}")

    print("\n[Training] Training and artifact serialization completed successfully!")

if __name__ == '__main__':
    train()
```

ml-service/model/evaluate.py

Evaluation script and metric generation.

```
"""Held-out evaluation of the shipped SymptomDiseaseClassifier artifact.

Reproduces the exact train/test split used by train.py (seed 42, 80/20) and
computes discrimination and calibration metrics on the held-out 20% that the
model never saw during fitting. Also measures the effect of the /predict
endpoint's primary-selection re-ranking on accuracy.

Usage:
    python evaluate.py
"""
import csv
import json
import math
import os
import pickle
import random
import sys

base_dir = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, base_dir)
from classifier import SymptomDiseaseClassifier  # noqa: E402

data_path = os.path.join(base_dir, '..', 'data', 'dataset.csv')
model_path = os.path.join(base_dir, 'model.pkl')
diseases_path = os.path.join(base_dir, 'diseases.json')

N_BINS = 10


def load_data(csv_path):
    with open(csv_path, 'r', encoding='utf-8') as f:
        reader = csv.reader(f)
        header = next(reader)
        features = header[:-1]
        X, y = [], []
        for row in reader:
            if not row:
                continue
            X.append([int(v) for v in row[:-1]])
            y.append(row[-1].strip('"'))
    return X, y, features


def expected_calibration_error(probas, labels, n_bins=N_BINS):
    """Standard equal-width-bin ECE over top-1 probabilities."""
    bins = [[] for _ in range(n_bins)]
    for conf, label in zip(probas, labels):
        idx = min(int(conf * n_bins), n_bins - 1)
        bins[idx].append((conf, label))
    ece = 0.0
    total = len(probas)
    for bucket in bins:
        if not bucket:
            continue
        acc = sum(1 for c, l in bucket if l) / len(bucket)
        avg_conf = sum(c for c, _ in bucket) / len(bucket)
        ece += (len(bucket) / total) * abs(acc - avg_conf)
    return ece


def compute_metrics(clf, X_test, y_test, feature_cols, diseases_metadata=None):
    """
    Held-out discrimination + calibration metrics for a fitted classifier.

    Returns a JSON-serializable dict; used by evaluate.py (artifact audit) and
    train.py (metrics written next to the model at training time).
    """
    probas = clf.predict_proba(X_test)
    classes = clf.classes_

    top1_correct = 0
    top3_correct = 0
    top1_probs = []
    top1_correct_flags = []
    nll = 0.0
    for row, truth, p_row in zip(X_test, y_test, probas):
        ranked = sorted(range(len(p_row)), key=lambda i: p_row[i], reverse=True)
        pred = classes[ranked[0]]
        if pred == truth:
            top1_correct += 1
        if truth in [classes[i] for i in ranked[:3]]:
            top3_correct += 1
        top1_probs.append(p_row[ranked[0]])
        top1_correct_flags.append(1 if pred == truth else 0)
        p_truth = p_row[classes.index(truth)]
        nll -= math.log(max(p_truth, 1e-12))

    n = len(y_test)
    accuracy = top1_correct / n
    top3_accuracy = top3_correct / n
    mean_nll = nll / n
    ece = expected_calibration_error(top1_probs, top1_correct_flags)
    brier = sum((p - f) ** 2 for p, f in zip(top1_probs, top1_correct_flags)) / n

    metrics = {
        'n_test': n,
        'n_classes': len(classes),
        'accuracy_top1': round(accuracy, 4),
        'accuracy_top3': round(top3_accuracy, 4),
        'accuracy_top1_pct': round(accuracy * 100, 2),
        'accuracy_top3_pct': round(top3_accuracy * 100, 2),
        'nll': round(mean_nll, 4),
        'brier_score': round(brier, 4),
        'ece_10bin': round(ece, 4),
        'mean_top1_probability': round(sum(top1_probs) / n, 4)
```

```
    }

    # --- Effect of app.py's select_primary_prediction re-ranking ---
    if diseases_metadata is not None:
        sys.path.insert(0, os.path.join(base_dir, '..'))
        from app import score_disease_overlap  # noqa: E402

        rerank_correct = 0
        rerank_flips_better = 0
        rerank_flips_worse = 0
        for row, truth, p_row in zip(X_test, y_test, probas):
            ranked = sorted(range(len(p_row)), key=lambda i: p_row[i], reverse=True)[:3]
            candidates = []
            for idx in ranked:
                name = classes[idx]
                info = diseases_metadata.get(name, {})
                overlap = score_disease_overlap(
                    name, [feature_cols[i] for i, v in enumerate(row) if v == 1]
                )
                candidates.append({
                    'name': name,
                    'final': (p_row[idx] * 100) * 0.7 + overlap * 100 * 0.3
                             + (5 if info.get('severity') == 'severe' else 0)
                })
            candidates.sort(key=lambda c: c['final'], reverse=True)
            best_before = classes[ranked[0]]
            after = candidates[0]['name']
            if after == truth:
                rerank_correct += 1
            if after != best_before:
                if after == truth:
                    rerank_flips_better += 1
                elif best_before == truth:
                    rerank_flips_worse += 1

        metrics['rerank_accuracy_pct'] = round((rerank_correct / n) * 100, 2)
        metrics['rerank_flips_better'] = rerank_flips_better
        metrics['rerank_flips_worse'] = rerank_flips_worse

    return metrics


def main():
    X, y, feature_cols = load_data(data_path)
    with open(model_path, 'rb') as f:
        clf = pickle.load(f)

    # Exact split replication from train.py
    data = list(zip(X, y))
    random.seed(42)
    random.shuffle(data)
    split_idx = int(len(data) * 0.8)
    test_data = data[split_idx:]
    X_test, y_test = zip(*test_data)
    X_test, y_test = list(X_test), list(y_test)

    with open(diseases_path, 'r', encoding='utf-8') as f:
        diseases_metadata = json.load(f)

    metrics = compute_metrics(clf, X_test, y_test, feature_cols, diseases_metadata)
    metrics['n_total'] = len(y_test) + int(split_idx)
    print(json.dumps(metrics, indent=2))

    out_path = os.path.join(base_dir, 'validation_metrics.json')
    with open(out_path, 'w', encoding='utf-8') as f:
        json.dump(metrics, f, indent=2)
    print(f'[Evaluate] Wrote {out_path}')


if __name__ == '__main__':
    main()
```

Safe control-file examples

backend/config/doctors-control.example.json

Only the first sample entry is shown.

```
{
  "Dr. Maya Patel": true
}
```

backend/config/bookings-control.example.json

Only the checked-in generic sample booking is shown.

```
{
  "sample-booking-id": {
    "bookingId": "sample-booking-id",
    "patientName": "Sample Patient",
    "doctorName": "Dr. Maya Patel",
    "specialty": "General Physician",
    "date": "2026-10-12",
    "time": "09:00",
    "reason": "Sample consultation reason",
    "status": "pending"
  }
}
```

10. Testing

The repository’s actual automated checks and this report-generation run are summarized below. Tests were executed from their own service directories. The aggregate Node `npm test` command was attempted, but this environment returned `spawn EPERM` when the Node test runner tried to start separate test-file processes; executing each actual test file directly worked.

Node backend tests

• `backend/tests/geminiService.test.js`: 20 passed, 0 failed.

• `backend/tests/mlService.test.js`: 8 passed, 0 failed. The local ML service was not running; its expected connection failure exercised fallback behavior.

• `backend/tests/predictionService.test.js`: 15 passed, 0 failed.

• Direct Node test total: 43 passed, 0 failed. Aggregate `npm test`: runner failed at process spawn (`EPERM`), not an assertion failure.

Python ML tests

• `python -m pytest -q` in `ml-service`: 12 passed. Pytest emitted a cache write warning for the workspace; assertions completed successfully.

Frontend checks

The repository does not define a frontend test script or a frontend test suite. No browser screenshot or UI test is claimed in this report.

Test coverage represented in source

The checked-in Node tests cover Gemini response parsing and sanitization, confidence interpretation, provider error classification, symptom normalization, ML service outage fallback, prediction-provider orchestration, and fallback behavior. Python tests exercise the ML prediction service. These tests do not establish clinical performance or production readiness.

backend/tests/geminiService.test.js

Verbatim checked-in test source.

```
import test from 'node:test';
import assert from 'node:assert/strict';
import {
  GeminiProviderError,
  classifyGeminiError,
  resolveGeminiConfig,
  isGeminiConfigured,
  parseGeminiExplanationResponse,
  parseGeminiPredictionResponse,
  buildConfidenceInterpretation,
  buildConfidenceMethod
} from '../services/geminiService.js';
import { GEMINI_DEFAULT_MODEL, GEMINI_ERROR_CODES, GEMINI_MAX_ARRAY_ITEMS, CONFIDENCE_BASIS } from '../config/constants.js';

// ---------------------------------------------------------------------------
// Preliminary prediction parsing (safe JSON, clamped probabilities)
// ---------------------------------------------------------------------------

test('parseGeminiPredictionResponse accepts a valid payload and clamps values', () => {
  const payload = JSON.stringify({
    valid: true,
    top_predictions: [
      { condition: 'Common Cold', probability: 42.55, reason: 'Cough and fever fit.' },
      { condition: 'Flu', probability: -5, reason: 'Fever fits.' },
      { condition: 'Allergy', probability: 250, reason: 'Less likely.' },
      { condition: 'Extra', probability: 5, reason: 'Ignored beyond top 3.' }
    ],
    urgency: 'Low',
    impact: 'moderate',
    explanation: 'Overall picture looks mild.',
    recommended_actions: ['Rest', 'Hydrate']
  });

  const parsed = parseGeminiPredictionResponse(payload);
  assert.equal(parsed.top_predictions.length, 3, 'capped at the top 3 candidates');
  assert.equal(parsed.top_predictions[0].probability, 42.6);
  assert.equal(parsed.top_predictions[1].probability, 0, 'probabilities clamp to 0-100');
  assert.equal(parsed.top_predictions[2].probability, 100);
  assert.equal(parsed.urgency, 'low', 'levels are normalized to lowercase');
  assert.equal(parsed.impact, 'moderate');
  assert.equal(parsed.explanation, 'Overall picture looks mild.');
  assert.deepEqual(parsed.recommended_actions, ['Rest', 'Hydrate']);
});

test('parseGeminiPredictionResponse strips markdown code fences', () => {
  const fenced = '```json\n{"valid":true,"top_predictions":[{"condition":"Migraine","probability":35,"reason":"Headache."}],"urgency":"Low","impact":"Lo
```

```
    w","explanation":"exp","recommended_actions":["Rest"]}\n```';
  const parsed = parseGeminiPredictionResponse(fenced);
  assert.equal(parsed.top_predictions[0].condition, 'Migraine');
});

test('parseGeminiPredictionResponse rejects malformed or empty payloads', () => {
  assert.throws(() => parseGeminiPredictionResponse(''), GeminiProviderError);
  assert.throws(() => parseGeminiPredictionResponse('{not json'), GeminiProviderError);
  assert.throws(
    () => parseGeminiPredictionResponse(JSON.stringify({ urgency: 'Low', explanation: 'x' })),
    GeminiProviderError
  );
  assert.throws(
    () => parseGeminiPredictionResponse(JSON.stringify({ top_predictions: [], explanation: 'x' })),
    GeminiProviderError
  );
});

test('buildConfidenceInterpretation labels Gemini estimates as preliminary', () => {
  const text = buildConfidenceInterpretation({ confidence: 40, confidence_basis: CONFIDENCE_BASIS.GEMINI_ESTIMATE });
  assert.match(text, /Gemini's preliminary AI estimate/);
  assert.match(text, /not a validated clinical probability/);
  const method = buildConfidenceMethod({ confidence_basis: CONFIDENCE_BASIS.GEMINI_ESTIMATE });
  assert.match(method, /Gemini AI preliminary estimate/);
});

// ---------------------------------------------------------------------------
// Provider configuration
// ---------------------------------------------------------------------------

test('resolveGeminiConfig defaults to the current stable Gemini Flash model', () => {
  const config = resolveGeminiConfig({ GEMINI_API_KEY: '[REDACTED]' });
  assert.equal(config.model, GEMINI_DEFAULT_MODEL);
  assert.equal(config.isConfigured, true);
  assert.equal(config.apiKey, 'test-key');
});

test('resolveGeminiConfig honors GEMINI_MODEL override and trims whitespace', () => {
  const config = resolveGeminiConfig({ GEMINI_API_KEY: '[REDACTED]', GEMINI_MODEL: ' gemini-custom-model ' });
  assert.equal(config.model, 'gemini-custom-model');
  assert.equal(config.isConfigured, true);
  assert.equal(config.apiKey, 'test-key');
});

test('resolveGeminiConfig reports unconfigured when the key is missing or blank', () => {
  assert.equal(resolveGeminiConfig({}).isConfigured, false);
  assert.equal(resolveGeminiConfig({ GEMINI_API_KEY: '[REDACTED]' }).isConfigured, false);
  assert.equal(isGeminiConfigured({}), false);
});

// ---------------------------------------------------------------------------
// Error classification (safe, actionable failure codes)
// ---------------------------------------------------------------------------

test('classifyGeminiError maps rate limits to GEMINI_RATE_LIMITED', () => {
  assert.equal(classifyGeminiError({ status: 429, message: 'Too many requests' }), GEMINI_ERROR_CODES.RATE_LIMITED);
  assert.equal(classifyGeminiError({ message: 'RESOURCE_EXHAUSTED: quota exceeded' }), GEMINI_ERROR_CODES.RATE_LIMITED);
});

test('classifyGeminiError maps auth failures to GEMINI_AUTH_FAILED', () => {
  assert.equal(classifyGeminiError({ status: 401, message: 'Unauthorized' }), GEMINI_ERROR_CODES.AUTH_FAILED);
  assert.equal(classifyGeminiError({ status: 403, message: 'Forbidden' }), GEMINI_ERROR_CODES.AUTH_FAILED);
  assert.equal(classifyGeminiError({ message: 'API key not valid' }), GEMINI_ERROR_CODES.AUTH_FAILED);
});

test('classifyGeminiError maps invalid requests and timeouts', () => {
  assert.equal(classifyGeminiError({ status: 400, message: 'Bad request' }), GEMINI_ERROR_CODES.INVALID_REQUEST);
  assert.equal(classifyGeminiError({ name: 'AbortError', message: 'aborted' }), GEMINI_ERROR_CODES.TIMEOUT);
  assert.equal(classifyGeminiError({ message: 'request timed out' }), GEMINI_ERROR_CODES.TIMEOUT);
});

test('classifyGeminiError defaults outages to GEMINI_UNAVAILABLE and preserves typed codes', () => {
  assert.equal(classifyGeminiError({ status: 503, message: 'Service unavailable' }), GEMINI_ERROR_CODES.UNAVAILABLE);
  assert.equal(classifyGeminiError({ message: 'something odd' }), GEMINI_ERROR_CODES.UNAVAILABLE);
  const typed = new GeminiProviderError(GEMINI_ERROR_CODES.INVALID_RESPONSE, 'bad payload');
  assert.equal(classifyGeminiError(typed), GEMINI_ERROR_CODES.INVALID_RESPONSE);
});

// ---------------------------------------------------------------------------
// Explanation parsing & validation (Gemini explains — it never scores)
// ---------------------------------------------------------------------------

test('parseGeminiExplanationResponse accepts a valid payload', () => {
  const payload = JSON.stringify({
    clinicalExplanation: 'Reported cough, fever and chills support the Common Cold candidate.',
    symptomPatternInsight: 'Upper respiratory symptom pattern.',
    differentialReasoning: 'Allergy is less likely without sneezing or watery eyes.',
    urgency: 'low',
    urgencyRationale: 'Symptoms are mild and self-limiting.',
    selfCareSteps: ['Rest', 'Hydrate'],
    warningSigns: ['High fever'],
    whenToSeeDoctor: 'If symptoms worsen.',
    clarificationQuestion: '',
    candidateAnalyses: [
      { condition: 'Common Cold', supportingSymptoms: ['Cough', 'High Fever'], missingSymptoms: ['Runny Nose'], summary: 'Consistent.' }
    ]
  });

  const parsed = parseGeminiExplanationResponse(payload);
  assert.equal(parsed.urgency, 'low');
  assert.equal(parsed.candidateAnalyses.length, 1);
  assert.equal(parsed.candidateAnalyses[0].condition, 'Common Cold');
  assert.deepEqual(parsed.candidateAnalyses[0].supportingSymptoms, ['Cough', 'High Fever']);
});
```

```
test('parseGeminiExplanationResponse never extracts a confidence value from Gemini output', () => {
  const payload = JSON.stringify({
    clinicalExplanation: 'Explanation text.',
    urgency: 'moderate',
    whenToSeeDoctor: 'See a doctor.',
    // Model tried to sneak scores in — they must not survive parsing
    estimatedConfidence: 91,
    confidence: 91,
    geminiConfidence: 91
  });

  const parsed = parseGeminiExplanationResponse(payload);
  assert.equal(Object.hasOwn(parsed, 'estimatedConfidence'), false);
  assert.equal(Object.hasOwn(parsed, 'confidence'), false);
  assert.equal(Object.hasOwn(parsed, 'geminiConfidence'), false);
});

test('parseGeminiExplanationResponse strips markdown code fences', () => {
  const fenced = '```json\n{"clinicalExplanation": "text", "urgency": "low", "whenToSeeDoctor": "see a doctor"}\n```';
  const parsed = parseGeminiExplanationResponse(fenced);
  assert.equal(parsed.clinicalExplanation, 'text');
});

test('parseGeminiExplanationResponse coerces invalid urgency onto the safe allow-list', () => {
  const payload = JSON.stringify({ clinicalExplanation: 'text', urgency: 'CATASTROPHIC' });
  assert.equal(parseGeminiExplanationResponse(payload).urgency, 'moderate');
});

test('parseGeminiExplanationResponse sanitizes arrays and candidate analyses', () => {
  const payload = JSON.stringify({
    clinicalExplanation: 'text',
    selfCareSteps: ['a', 42, null, 'b', 'c', 'd', 'e', 'f', 'g'],
    candidateAnalyses: [
      { condition: 'Common Cold', supportingSymptoms: ['Cough'], missingSymptoms: [], summary: 'ok' },
      'not-an-object',
      { condition: '' }
    ]
  });
  const parsed = parseGeminiExplanationResponse(payload);
  assert.equal(parsed.selfCareSteps.length, GEMINI_MAX_ARRAY_ITEMS);
  assert.deepEqual(parsed.selfCareSteps, ['a', 'b', 'c', 'd', 'e']);
  assert.equal(parsed.candidateAnalyses.length, 1);
  assert.equal(parsed.candidateAnalyses[0].condition, 'Common Cold');
});

test('parseGeminiExplanationResponse rejects malformed, empty, or incomplete payloads', () => {
  assert.throws(
    () => parseGeminiExplanationResponse('I am feeling unwell, doctor'),
    (error) => error instanceof GeminiProviderError && error.code === GEMINI_ERROR_CODES.INVALID_RESPONSE
  );
  assert.throws(
    () => parseGeminiExplanationResponse(''),
    (error) => error.code === GEMINI_ERROR_CODES.INVALID_RESPONSE
  );
  assert.throws(
    () => parseGeminiExplanationResponse('["array"]'),
    (error) => error.code === GEMINI_ERROR_CODES.INVALID_RESPONSE
  );
  assert.throws(
    () => parseGeminiExplanationResponse(JSON.stringify({ urgency: 'high', whenToSeeDoctor: 'now' })),
    (error) => error.code === GEMINI_ERROR_CODES.INVALID_RESPONSE
  );
});

// ---------------------------------------------------------------------------
// Confidence labelling (probability only from the validated model)
// ---------------------------------------------------------------------------

test('buildConfidenceInterpretation reports measured validation for model probabilities', () => {
  const text = buildConfidenceInterpretation({
    confidence: 84.5,
    confidence_basis: CONFIDENCE_BASIS.MODEL_PROBABILITY,
    validation: { n_test: 960, accuracy_top1_pct: 99.69, ece_10bin: 0.003 }
  });
  assert.match(text, /84\.5%/);
  assert.match(text, /99\.69/);
  assert.match(text, /n=960/);
  assert.match(text, /not a clinical probability/);
});

test('buildConfidenceInterpretation labels heuristic scores as preliminary, not probabilities', () => {
  const text = buildConfidenceInterpretation({ confidence: 40, confidence_basis: CONFIDENCE_BASIS.HEURISTIC_OVERLAP });
  assert.match(text, /overlap/i);
  assert.match(text, /not a probability/);
});

test('buildConfidenceMethod never claims a probability without the validated model basis', () => {
  assert.match(buildConfidenceMethod({ confidence_basis: CONFIDENCE_BASIS.HEURISTIC_OVERLAP }), /not a calibrated probability/);
  assert.match(buildConfidenceMethod({ confidence_basis: CONFIDENCE_BASIS.NONE }), /insufficient evidence/);
  assert.doesNotMatch(buildConfidenceMethod({ confidence_basis: CONFIDENCE_BASIS.RULE_BASED }), /^SymptomDiseaseClassifier/);
  assert.match(
    buildConfidenceMethod({ confidence_basis: CONFIDENCE_BASIS.MODEL_PROBABILITY, validation: { accuracy_top1_pct: 99.69 } }),
    /99\.69/
  );
});
```

backend/tests/mlService.test.js

Verbatim checked-in test source.

```
import test from 'node:test';
import assert from 'node:assert/strict';
```

```
import fs from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';
import { predictDiagnosis, resolveSymptomId } from '../services/mlService.js';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const diseaseCatalog = JSON.parse(
  fs.readFileSync(path.resolve(__dirname, '../data/diseases.json'), 'utf8')
);
const catalogDiseaseNames = new Set(Object.keys(diseaseCatalog));

const okResponse = (data) => ({ ok: true, status: 200, json: async () => data });
const errorResponse = (status, data) => ({ ok: false, status, json: async () => data });

/** Replaces global fetch for the duration of a test (restored automatically). */
const mockFetch = (t, impl) => {
  const original = globalThis.fetch;
  globalThis.fetch = impl;
  t.after(() => {
    globalThis.fetch = original;
  });
};

// ---------------------------------------------------------------------------
// Symptom resolution (synonyms + misspellings + unrelated input)
// ---------------------------------------------------------------------------

test('resolveSymptomId resolves exact, misspelled and unrelated inputs', () => {
  assert.equal(resolveSymptomId('headache'), 'headache');
  assert.equal(resolveSymptomId('hedache'), 'headache', 'typo within edit distance 1');
  assert.equal(resolveSymptomId('hig_fever'), 'high_fever', 'typo within edit distance 1');
  assert.equal(resolveSymptomId('best_pizza_place'), null, 'unrelated input must not resolve');
  assert.equal(resolveSymptomId('pain'), null, 'ambiguous bare word must not silently map to one condition');
});

// ---------------------------------------------------------------------------
// Clarification paths — no disease is ever invented
// ---------------------------------------------------------------------------

test('predictDiagnosis returns a clarification for whitespace-only input without calling the ML service', async (t) => {
  mockFetch(t, async () => {
    throw new Error('ML service must not be called');
  });

  const result = await predictDiagnosis(['   ', '']);
  assert.equal(result.needs_clarification, true);
  assert.equal(result.primary_condition, '');
  assert.equal(result.top_predictions.length, 0);
  assert.match(result.clarification_message, /at least one symptom/i);
});

test('predictDiagnosis returns a clarification (not a disease) for unrelated input without touching the model', async (t) => {
  let fetchCalled = false;
  mockFetch(t, async () => {
    fetchCalled = true;
    return errorResponse(422, {
      success: false,
      code: 'UNRECOGNIZED_SYMPTOMS',
      detail: 'not found in the diagnostic dataset',
      unrecognized_symptoms: ['best_pizza_place', 'stock_market'],
      suggestions: ['headache', 'chest_pain']
    });
  });

  const result = await predictDiagnosis(['best_pizza_place', 'stock_market']);
  assert.equal(fetchCalled, false, 'unrelated input is rejected locally before any model call');
  assert.equal(result.needs_clarification, true);
  assert.equal(result.primary_condition, '', 'no condition may be returned for unrelated input');
  assert.equal(result.top_predictions.length, 0);
  assert.deepEqual(result.unrecognized_symptoms, ['best_pizza_place', 'stock_market']);
  assert.ok(result.suggestions.length > 0, 'nearest catalog suggestions help the user rephrase');
  // Regression guard: the old hardcoded trio must never come back
  assert.ok(!catalogDiseaseNames.has('Unspecified Viral Syndrome'));
  assert.notEqual(result.primary_condition, 'Unspecified Viral Syndrome');
});

test('predictDiagnosis returns a clarification when the ML service is down and input is unrelated', async (t) => {
  mockFetch(t, async () => {
    throw new Error('connect ECONNREFUSED 127.0.0.1:8000');
  });

  const result = await predictDiagnosis(['best_pizza_place', 'stock_market']);
  assert.equal(result.needs_clarification, true);
  assert.equal(result.primary_condition, '');
  assert.equal(result.top_predictions.length, 0, 'heuristic fallback must not invent a default disease');
});

test('predictDiagnosis forwards a single recognized symptom to the ML service (no limited-assessment gate)', async (t) => {
  let requestBody = null;
  mockFetch(t, async (_url, options) => {
    requestBody = JSON.parse(options.body);
    return okResponse({
      success: true,
      primary_condition: 'Migraine',
      confidence: 62.5,
      confidence_basis: 'model_probability',
      severity: 'moderate',
      description: 'model output',
      top_predictions: [{ disease: 'Migraine', confidence: 62.5, severity: 'moderate', description: '' }],
      recognized_symptoms: ['high_fever'],
      unrecognized_symptoms: [],
      disclaimer: 'd'
    });
  });
```

```
  const result = await predictDiagnosis(['fever']);
  assert.deepEqual(requestBody.symptoms, ['high_fever'], 'alias resolution still applies');
  assert.equal(result.primary_condition, 'Migraine');
  assert.equal(result.confidence_basis, 'model_probability');
  assert.equal(result.isFallback, false);
});

// ---------------------------------------------------------------------------
// Symptom-grounded predictions (ML pass-through and heuristic fallback)
// ---------------------------------------------------------------------------

test('predictDiagnosis resolves misspellings and forwards them to the ML service', async (t) => {
  let requestBody = null;
  mockFetch(t, async (_url, options) => {
    requestBody = JSON.parse(options.body);
    return okResponse({
      success: true,
      primary_condition: 'Migraine',
      confidence: 91.2,
      confidence_basis: 'model_probability',
      validation: { n_test: 960, accuracy_top1_pct: 99.69, ece_10bin: 0.003 },
      severity: 'moderate',
      description: 'model output',
      top_predictions: [{ disease: 'Migraine', confidence: 91.2, severity: 'moderate', description: '' }],
      recognized_symptoms: ['headache', 'high_fever'],
      unrecognized_symptoms: [],
      disclaimer: 'd'
    });
  });

  const result = await predictDiagnosis(['hedache', 'hig fever']);
  assert.deepEqual(requestBody.symptoms, ['hedache', 'hig_fever'], 'raw normalized input is sent; both engines resolve it');
  assert.equal(result.primary_condition, 'Migraine');
  assert.equal(result.confidence_basis, 'model_probability');
  assert.equal(result.isFallback, false);
  assert.ok(result.validation && result.validation.accuracy_top1_pct, 'validation metrics travel with the probability');
});

test('predictDiagnosis falls back to symptom-overlap scoring (never a hardcoded disease) when the ML service is down', async (t) => {
  mockFetch(t, async () => {
    throw new Error('connect ECONNREFUSED 127.0.0.1:8000');
  });

  const result = await predictDiagnosis(['cough', 'high_fever']);
  assert.equal(result.isFallback, true);
  assert.equal(result.confidence_basis, 'heuristic_overlap');
  assert.ok(result.top_predictions.length > 0 && result.top_predictions.length <= 3);
  for (const candidate of result.top_predictions) {
    assert.ok(catalogDiseaseNames.has(candidate.disease), `${candidate.disease} must come from the disease catalog`);
    assert.ok(candidate.confidence >= 0 && candidate.confidence <= 100);
  }
  assert.deepEqual(result.recognized_symptoms, ['cough', 'high_fever']);
});

test('predictDiagnosis uses the urgent-care rule for reported orthopedic trauma without calling the ML service', async (t) => {
  mockFetch(t, async () => {
    throw new Error('ML service must not be called for the injury rule');
  });

  const result = await predictDiagnosis(['leg break']);
  assert.equal(result.primary_condition, 'Suspected Bone Fracture / Musculoskeletal Trauma');
  assert.equal(result.confidence_basis, 'rule_based');
  assert.equal(result.confidence, 0, 'rule-based results never claim a probability');
  assert.equal(result.severity, 'severe');
  assert.match(result.recommendation, /urgent care|emergency/i);
});
```

backend/tests/predictionService.test.js

Verbatim checked-in test source.

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { resolvePredictionProvider, runPrediction, validatePredictionEnvironment } from '../services/predictionService.js';
import { GeminiProviderError } from '../services/geminiService.js';
import {
  GEMINI_ERROR_CODES,
  PREDICTION_PROVIDER_GEMINI,
  PREDICTION_PROVIDER_ML,
  PREDICTION_PROVIDER_HEURISTIC
} from '../config/constants.js';

// Symptom-grounded engine result (what the ML/heuristic engine returns)
const fakeMlPrediction = {
  success: true,
  primary_condition: 'Common Cold',
  confidence: 80,
  confidence_basis: 'model_probability',
  severity: 'mild',
  description: 'ML description',
  top_predictions: [
    { disease: 'Common Cold', confidence: 80, severity: 'mild', description: '' },
    { disease: 'Allergy', confidence: 12, severity: 'mild', description: '' },
    { disease: 'Bronchial Asthma', confidence: 8, severity: 'severe', description: '' }
  ],
  recognized_symptoms: ['cough', 'high_fever'],
  unrecognized_symptoms: [],
  disclaimer: 'Preliminary guidance only.',
  isFallback: false
};

const fakeClarification = {
```

```
  success: false,
  needs_clarification: true,
  clarification_message: 'We could not recognize those entries as symptoms.',
  primary_condition: '',
  confidence: 0,
  confidence_basis: 'none',
  top_predictions: [],
  recognized_symptoms: [],
  unrecognized_symptoms: ['best_pizza_place'],
  disclaimer: 'Preliminary guidance only.',
  isFallback: false
};

const fakeExplanationInsights = {
  clinicalExplanation: 'Reported cough and fever support Common Cold.',
  urgencyAssessment: 'low',
  accuracyMetrics: null,
  geminiEnriched: true,
  combinedPrediction: { predictedDisease: 'Common Cold', confidence: 80, method: 'SymptomDiseaseClassifier probability' }
};

// ---------------------------------------------------------------------------
// Provider configuration
// ---------------------------------------------------------------------------

test('resolvePredictionProvider defaults to gemini (explanation mode)', () => {
  assert.equal(resolvePredictionProvider(undefined), PREDICTION_PROVIDER_GEMINI);
  assert.equal(resolvePredictionProvider(''), PREDICTION_PROVIDER_GEMINI);
});

test('resolvePredictionProvider is case-insensitive and trims whitespace', () => {
  assert.equal(resolvePredictionProvider(' ML '), PREDICTION_PROVIDER_ML);
  assert.equal(resolvePredictionProvider('Gemini'), PREDICTION_PROVIDER_GEMINI);
});

test('resolvePredictionProvider falls back to gemini for unknown values', () => {
  assert.equal(resolvePredictionProvider('openai'), PREDICTION_PROVIDER_GEMINI);
});

// ---------------------------------------------------------------------------
// Orchestration: engine grounds the prediction, Gemini only explains
// ---------------------------------------------------------------------------

test('runPrediction grounds the condition in the ML engine and uses Gemini only to explain', async () => {
  let enricherInput = null;
  const result = await runPrediction(
    { symptoms: ['cough', 'high_fever'], duration: '1–3 days', severity: 'moderate' },
    {
      geminiPredictor: async () => null,
      mlPredictor: async () => fakeMlPrediction,
      enricher: async (input) => {
        enricherInput = input;
        return fakeExplanationInsights;
      }
    }
  );

  // The prediction object is the engine result — never a Gemini-chosen disease
  assert.equal(result.prediction, fakeMlPrediction);
  assert.equal(result.provider, PREDICTION_PROVIDER_ML);
  assert.equal(result.insights, fakeExplanationInsights);
  assert.equal(result.fallbackReason, null);
  // The enricher received the engine's result (to explain), not raw control
  assert.equal(enricherInput.mlResult, fakeMlPrediction);
});

test('runPrediction returns a low-confidence fallback for unrecognized input', async () => {
  let enricherCalled = false;
  const result = await runPrediction(
    { symptoms: ['best_pizza_place', 'stock_market'] },
    {
      geminiPredictor: async () => null,
      mlPredictor: async () => fakeClarification,
      enricher: async () => {
        enricherCalled = true;
        return fakeExplanationInsights;
      }
    }
  );

  assert.equal(result.prediction.needs_clarification, undefined);
  assert.match(result.prediction.primary_condition, /best_pizza_place/);
  assert.ok(result.insights);
  assert.equal(result.fallbackReason, null);
  assert.equal(enricherCalled, false);
});

test('health terms and random letters always produce a result when Gemini and ML cannot classify', async () => {
  for (const term of ['hair fall', 'diabetes', 'headache', 'leg break', 'asdfgh']) {
    const result = await runPrediction({ symptoms: [term] }, {
      geminiPredictor: async () => null,
      mlPredictor: async () => ({ ...fakeClarification, unrecognized_symptoms: [term] }),
      provider: PREDICTION_PROVIDER_ML
    });
    assert.ok(result.prediction.primary_condition, `${term} has a prediction`);
    assert.ok(result.prediction.top_predictions.length > 0, `${term} has a top prediction`);
    assert.ok(result.prediction.disclaimer);
  }
});

test('runPrediction degrades to local explanations when Gemini fails, recording the safe reason', async () => {
  const result = await runPrediction(
    { symptoms: ['cough', 'high_fever'] },
    {
      geminiPredictor: async () => null,
```

```
      mlPredictor: async () => fakeMlPrediction,
      enricher: async () => {
        throw new GeminiProviderError(GEMINI_ERROR_CODES.RATE_LIMITED, 'Gemini explanation failed (GEMINI_RATE_LIMITED)');
      }
    }
  );

  // Condition still comes from the engine even when explanations fail
  assert.equal(result.prediction.primary_condition, 'Common Cold');
  assert.equal(result.provider, PREDICTION_PROVIDER_ML);
  assert.equal(result.fallbackReason, GEMINI_ERROR_CODES.RATE_LIMITED);
  assert.ok(result.insights, 'local insights should be generated on the fallback path');
  assert.equal(result.insights.accuracyMetrics, null, 'no consensus metrics are invented on the fallback path');
  assert.equal(result.insights.geminiEnriched, false);
});

test('runPrediction picks up the safe reason when the enricher degrades gracefully', async () => {
  const result = await runPrediction(
    { symptoms: ['cough', 'high_fever'] },
    {
      geminiPredictor: async () => null,
      mlPredictor: async () => fakeMlPrediction,
      enricher: async () => ({
        ...fakeExplanationInsights,
        geminiEnriched: false,
        enrichmentErrorCode: GEMINI_ERROR_CODES.AUTH_FAILED
      })
    }
  );

  assert.equal(result.fallbackReason, GEMINI_ERROR_CODES.AUTH_FAILED);
  assert.equal(result.prediction.primary_condition, 'Common Cold');
});

test('runPrediction labels the candidate engine as heuristic when the ML service is also down', async () => {
  const result = await runPrediction(
    { symptoms: ['cough', 'high_fever'] },
    {
      geminiPredictor: async () => null,
      mlPredictor: async () => ({ ...fakeMlPrediction, isFallback: true, confidence_basis: 'heuristic_overlap' }),
      enricher: async () => fakeExplanationInsights
    }
  );

  assert.equal(result.provider, PREDICTION_PROVIDER_HEURISTIC);
  assert.equal(result.prediction.isFallback, true);
});

test('runPrediction in ml mode uses local explanations without calling the enricher', async () => {
  let enrichmentCalled = false;
  const result = await runPrediction(
    { symptoms: ['cough'] },
    {
      provider: PREDICTION_PROVIDER_ML,
      mlPredictor: async () => fakeMlPrediction,
      enricher: async () => {
        enrichmentCalled = true;
        return fakeExplanationInsights;
      }
    }
  );

  assert.equal(result.provider, PREDICTION_PROVIDER_ML);
  assert.equal(result.fallbackReason, null);
  assert.ok(result.insights, 'local rule-based insights are always produced');
  assert.equal(result.insights.accuracyMetrics, null);
  assert.equal(enrichmentCalled, false, 'ml mode never calls the external Gemini enricher');
});

// ---------------------------------------------------------------------------
// Gemini preliminary prediction (1+ symptoms, mapped to the UI fields)
// ---------------------------------------------------------------------------

test('runPrediction uses a Gemini preliminary prediction for any symptoms (1+) when available', async () => {
  const fakeGeminiPrediction = {
    success: true,
    primary_condition: 'Tension headache',
    confidence: 42,
    confidence_basis: 'gemini_estimate',
    severity: 'mild',
    top_predictions: [{ disease: 'Tension headache', condition: 'Tension headache', confidence: 42, severity: 'mild', description: 'r1' }],
    recognized_symptoms: ['headache'],
    unrecognized_symptoms: [],
    disclaimer: 'Preliminary guidance only.',
    isFallback: false
  };
  const fakeGeminiInsights = {
    clinicalExplanation: 'Single-symptom preliminary estimate.',
    urgencyAssessment: 'low',
    accuracyMetrics: null,
    geminiEnriched: true,
    combinedPrediction: { predictedDisease: 'Tension headache', confidence: 42, method: 'Gemini AI preliminary estimate' }
  };
  let mlCalled = false;
  let enricherCalled = false;
  const result = await runPrediction(
    { symptoms: ['headache'] },
    {
      geminiPredictor: async () => ({ prediction: fakeGeminiPrediction, insights: fakeGeminiInsights }),
      mlPredictor: async () => {
        mlCalled = true;
        return fakeMlPrediction;
      },
      enricher: async () => {
        enricherCalled = true;
```

```
        return fakeExplanationInsights;
      }
    }
  );

  assert.equal(result.prediction, fakeGeminiPrediction, 'a single symptom gets the Gemini prediction');
  assert.equal(result.insights, fakeGeminiInsights);
  assert.equal(mlCalled, false, 'the symptom engine is not consulted when Gemini succeeds');
  assert.equal(enricherCalled, false, 'no separate explanation pass is needed');
  assert.equal(result.provider, PREDICTION_PROVIDER_ML, 'provider keeps the UI dual-consensus layout');
  assert.equal(result.fallbackReason, null);
});

test('runPrediction falls back to the symptom engine when the Gemini prediction fails', async () => {
  const result = await runPrediction(
    { symptoms: ['headache'] },
    {
      geminiPredictor: async () => {
        throw new GeminiProviderError(GEMINI_ERROR_CODES.UNAVAILABLE, 'service unavailable');
      },
      mlPredictor: async () => fakeMlPrediction,
      enricher: async () => fakeExplanationInsights
    }
  );

  assert.equal(result.prediction.primary_condition, 'Common Cold', 'fallback still yields a real prediction');
  assert.equal(result.provider, PREDICTION_PROVIDER_ML);
  assert.equal(result.fallbackReason, null);
});

// ---------------------------------------------------------------------------
// Startup environment validation
// ---------------------------------------------------------------------------

test('validatePredictionEnvironment warns when the Gemini key is missing', () => {
  const result = validatePredictionEnvironment({ PREDICTION_PROVIDER: 'gemini' });
  assert.equal(result.provider, PREDICTION_PROVIDER_GEMINI);
  assert.equal(result.warnings.length, 1);
  assert.match(result.warnings[0], /GEMINI_API_KEY/);
});

test('validatePredictionEnvironment reports no warnings when the key is set', () => {
  const result = validatePredictionEnvironment({ PREDICTION_PROVIDER: 'gemini', GEMINI_API_KEY: '[REDACTED]' });
  assert.equal(result.warnings.length, 0);
});

test('validatePredictionEnvironment does not warn about a missing key in ml mode', () => {
  const result = validatePredictionEnvironment({ PREDICTION_PROVIDER: 'ml' });
  assert.equal(result.provider, PREDICTION_PROVIDER_ML);
  assert.equal(result.warnings.length, 0);
});
```

ml-service/tests/test_predict.py

Verbatim checked-in test source.

```
import pytest
import sys
import os

# Add parent directory to path so app can be imported
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from app import app, is_loaded

@pytest.fixture
def client():
    app.config['TESTING'] = True
    with app.test_client() as client:
        yield client

def test_health_endpoint(client):
    response = client.get('/health')
    assert response.status_code == 200
    data = response.get_json()
    assert data["status"] == "ok"
    assert data["model_loaded"] is True
    assert data["features_count"] >= 130

def test_get_symptoms(client):
    response = client.get('/symptoms')
    assert response.status_code == 200
    data = response.get_json()
    assert isinstance(data, list)
    assert len(data) >= 130
    assert "id" in data[0]
    assert "name" in data[0]

def test_predict_common_cold(client):
    payload = {
        "symptoms": ["chills", "continuous_sneezing", "cough", "runny_nose"]
    }
    response = client.post('/predict', json=payload)
    assert response.status_code == 200
    data = response.get_json()
    assert data["success"] is True
    assert "Common Cold" in [p["disease"] for p in data["top_predictions"]]
    assert data["confidence"] > 0
    assert len(data["top_predictions"]) == 3
    assert "not a clinical medical diagnosis" in data["disclaimer"]


def test_predict_single_symptom_returns_model_prediction(client):
```

```
    payload = {"symptoms": ["fever"]}
    response = client.post('/predict', json=payload)
    assert response.status_code == 200
    data = response.get_json()
    assert data["primary_condition"] != "Limited symptom assessment"
    assert data["confidence"] > 0
    assert data["confidence_basis"] == "model_probability"
    assert len(data["top_predictions"]) == 3
    assert data["recognized_symptoms"] == ["high_fever"]
    assert "not a clinical medical diagnosis" in data["disclaimer"]


def test_predict_with_unrecognized_symptoms(client):
    payload = {
        "symptoms": ["chills", "cough", "fictional_alien_symptom"]
    }
    response = client.post('/predict', json=payload)
    assert response.status_code == 200
    data = response.get_json()
    assert "fictional_alien_symptom" in data["unrecognized_symptoms"]
    assert "chills" in data["recognized_symptoms"]
    assert len(data["top_predictions"]) == 3

def test_predict_all_unrecognized(client):
    payload = {
        "symptoms": ["completely_invented_symptom_123"]
    }
    response = client.post('/predict', json=payload)
    assert response.status_code == 422

def test_predict_unrecognized_body_includes_code_and_suggestions(client):
    payload = {"symptoms": ["best_pizza_place", "stock_market"]}
    response = client.post('/predict', json=payload)
    assert response.status_code == 422
    data = response.get_json()
    assert data["code"] == "UNRECOGNIZED_SYMPTOMS"
    assert data["unrecognized_symptoms"] == ["best_pizza_place", "stock_market"]
    assert len(data["suggestions"]) > 0


def test_predict_ambiguous_bare_word_is_not_silently_matched(client):
    # "pain" touches every *_pain feature — it must clarify, not guess.
    payload = {"symptoms": ["pain"]}
    response = client.post('/predict', json=payload)
    assert response.status_code == 422


def test_predict_misspelled_symptoms_are_resolved(client):
    payload = {"symptoms": ["hedache", "hig fever"]}
    response = client.post('/predict', json=payload)
    assert response.status_code == 200
    data = response.get_json()
    assert data["success"] is True
    assert data["recognized_symptoms"] == ["headache", "high_fever"]
    assert data["unrecognized_symptoms"] == []


def test_predict_returns_confidence_basis_and_validation_metrics(client):
    payload = {"symptoms": ["chills", "continuous_sneezing", "cough", "runny_nose"]}
    response = client.post('/predict', json=payload)
    assert response.status_code == 200
    data = response.get_json()
    assert data["confidence_basis"] == "model_probability"
    assert "validation" in data
    assert data["validation"]["n_test"] > 0
    assert data["validation"]["accuracy_top1_pct"] > 0
    assert "ece_10bin" in data["validation"]


def test_predict_single_symptom_uses_model_probability_basis(client):
    payload = {"symptoms": ["fever"]}
    response = client.post('/predict', json=payload)
    assert response.status_code == 200
    data = response.get_json()
    assert data["confidence_basis"] == "model_probability"


def test_predict_empty_symptoms(client):
    payload = {
        "symptoms": []
    }
    response = client.post('/predict', json=payload)
    assert response.status_code == 422
```

11. Conclusion

Online Health Diagnosis System demonstrates a full-stack implementation connecting user-facing symptom workflows, protected APIs, database-backed records, provider integrations, and a separate machine-learning service. Its modular structure and tests support continued development. It remains a preliminary triage and educational tool; a qualified healthcare professional must assess medical concerns.

Future Scope

- Add comprehensive frontend component and browser end-to-end tests, including accessibility review.
- Add structured audit trails, stronger appointment conflict handling, and clearer operational monitoring.

- Evaluate the classifier on independently sourced, representative data and publish transparent validation methodology.
- Add clinician-reviewed content and localized emergency information after jurisdiction-specific review.
- Complete deployment, privacy, threat-model, and regulatory assessments before any real clinical or patient-facing use.

Bibliography

- Project source repository: `frontend/`, `backend/`, and `ml-service/` (implementation inspected for this report).
- Project requirements: `Docs/PRD.md`; architecture baseline: `Docs/ARCHITECTURE.md`; code rules and standards: `Docs/RULES.md` and `Docs/CODING_STANDARDS.md`.
- React documentation: https://react.dev/
- Express documentation: https://expressjs.com/
- Mongoose documentation: https://mongoosejs.com/docs/
- Flask documentation: https://flask.palletsprojects.com/
- scikit-learn documentation: https://scikit-learn.org/stable/
- Google Gemini API integration reference: https://ai.google.dev/gemini-api/docs (provider configured outside this report; no key included).

Submission placeholders to complete: [Student Full Name], [Mentor Name], [Company Name], [College/University Name], [Academic Year], signature lines, date, place, and all marked screenshots.